



Universidade Federal
do Rio de Janeiro

Escola Politécnica

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

Projeto de Graduação apresentado ao
Curso de Engenharia Mecânica da Escola
Politécnica, Universidade Federal do Rio
de Janeiro, como parte dos requisitos ne-
cessários à obtenção do título de Engenheiro
Mecânico.

Orientador: Gustavo Rabello dos Anjos

Rio de Janeiro
Fevereiro de 2026

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO
DE ENGENHARIA MECÂNICA DA ESCOLA POLITÉCNICA DA UNIVERSI-
DADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NE-
CESSÁRIOS PARA A OBTENÇÃO DO GRAU DE ENGENHEIRO MECÂNICO

Autor:

Luiz Gustavo Santos Farias

Orientador:

Prof. Gustavo Rabello dos Anjos, D. Sc.

Examinador:

Prof. Examinador 1, D. Sc.

Examinador:

Prof. Examinador 2, D. Sc.

Rio de Janeiro

Fevereiro de 2026

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Escola Politécnica - Departamento de Engenharia Mecânica

Centro de Tecnologia, bloco G, sala G-204, Cidade Universitária

Rio de Janeiro - RJ CEP 21941-909

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmear ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

DEDICATÓRIA

Dedico este trabalho, primeiramente, a Deus, Senhor da minha vida, que sustentou cada etapa desta caminhada e me concedeu graça, perseverança e propósito quando as forças pareciam insuficientes.

À minha esposa Aretha, companheira fiel, amiga e apoio constante em todos os momentos, cujo amor, paciência e incentivo tornaram possível a conclusão desta jornada.

À minha filha Marina, que mesmo sem compreender totalmente o significado deste trabalho, foi diariamente a maior motivação para não desistir. Que este diploma seja também um testemunho para você de que vale a pena terminar o que começamos.

Aos meus pais, Daniela e Silvio, que sempre acreditaram na educação como instrumento de transformação e nunca deixaram faltar apoio, cuidado e exemplo.

Às minhas irmãs, Yngrid e Yasmim, pela presença, amizade e pelo vínculo familiar que sempre me fortaleceu.

Comecei este caminho como estudante. Concluo como engenheiro, marido, pai e servo de Cristo.

AGRADECIMENTO

Agradeço primeiramente a Deus, por sustentar cada etapa desta caminhada e por conceder sabedoria e perseverança para chegar até aqui.

Ao meu orientador, Prof. Gustavo Rabello dos Anjos, pela orientação cuidadosa, confiança e incentivo intelectual, fundamentais para o desenvolvimento deste trabalho e para o amadurecimento da proposta do MinervaMesh.

À Universidade Federal do Rio de Janeiro e ao corpo docente do curso de Engenharia Mecânica, pela formação técnica e científica que tornou possível a realização deste projeto.

Ao meu grande amigo e padrinho de casamento, Michel Silva Bonifácio, pelo apoio durante a graduação e pela presença constante em momentos decisivos da minha trajetória acadêmica.

Ao meu amigo Eliab Araújo, pelo incentivo para a conclusão do curso e pela motivação contínua no desenvolvimento deste trabalho.

Aos colegas, amigos e a todos que contribuíram direta ou indiretamente para minha formação como engenheiro.

RESUMO

A simulação numérica constitui ferramenta estratégica para análise e projeto em Engenharia Mecânica, porém seu uso em ambiente educacional ainda é limitado por custos de licenciamento, requisitos de infraestrutura e complexidade de configuração. Este trabalho apresenta o *MinervaMesh*, uma plataforma de simulação numérica executada integralmente em navegador, concebida para reduzir barreiras de acesso sem comprometer o rigor técnico. A solução adota arquitetura *client-side* baseada em *WebAssembly* e *PyScript*, permitindo a execução de rotinas em Python científico para solução de Equações Diferenciais Parciais por Método dos Elementos Finitos (MEF) e Método das Diferenças Finitas (MDF). A metodologia de validação prioriza casos com solução analítica conhecida para verificação de precisão, estabilidade e convergência, complementados por casos adicionais representativos de fluidodinâmica e transferência de calor. Os resultados indicam que a plataforma preserva consistência numérica e apresenta potencial de aplicação em ensino, prototipagem e demonstrações acadêmicas de mecânica computacional.

Palavras-Chave: Método dos Elementos Finitos, Método das Diferenças Finitas, Simulação Web, Mecânica Computacional, Ensino de Engenharia.

ABSTRACT

Numerical simulation is a core tool for analysis and design in Mechanical Engineering; however, its broader adoption in education is still constrained by software licensing costs, infrastructure requirements, and setup complexity. This work presents *MinervaMesh*, a browser-based numerical simulation platform designed to reduce access barriers while maintaining technical rigor. The proposed system follows a fully *client-side* architecture built on *WebAssembly* and *PyScript*, enabling scientific Python workflows for solving Partial Differential Equations with the Finite Element Method (FEM) and the Finite Difference Method (FDM). The validation strategy prioritizes benchmark cases with known analytical solutions to assess accuracy, stability, and convergence, followed by additional representative fluid-flow and heat-transfer cases. Results indicate that the platform preserves numerical consistency and is suitable for educational use, rapid prototyping, and academic demonstrations in computational mechanics.

Key-words: Finite Element Method, Finite Difference Method, Web Simulation, Computational Mechanics, Engineering Education.

SIGLAS

UFRJ - Universidade Federal do Rio de Janeiro

MEF - Método dos Elementos Finitos

MDF - Método das Diferenças Finitas

CFD - *Computational Fluid Dynamics* (Dinâmica dos Fluidos Computacional)

WASM - WebAssembly

SUPG - *Streamline Upwind/Petrov-Galerkin*

Sumário

1	Introdução	1
1.1	Introdução	1
1.2	Objetivos	2
1.3	Justificativa	3
1.4	Organização do Trabalho	3
2	Revisão Bibliográfica	4
2.1	Introdução ao Capítulo	4
2.2	Problemas Térmicos e Equações Governantes	4
2.2.1	Condução de Calor	4
2.2.2	Convecção e Acoplamento com Escoamento	4
2.2.3	Radiação Térmica	5
2.3	Métodos Numéricos para EDPs	5
2.3.1	Método das Diferenças Finitas (MDF)	5
2.3.2	Método dos Elementos Finitos (MEF)	5
2.4	Bases Históricas e Consolidação do MEF/CFD	6
2.5	Trabalhos Correlatos (Ênfase em TCCs Semelhantes)	6
2.6	Síntese da Revisão e Lacuna Atacada	7
3	Fundamentação Teórica	8
3.1	Introdução ao Capítulo	8
3.2	Equações Governantes	8
3.2.1	Condução e Convecção Térmica	9
3.2.2	Escoamento de Fluidos (Navier-Stokes)	9
3.3	Transição Discreta: O Método das Diferenças Finitas (MDF)	10

3.4	O Método dos Elementos Finitos (MEF)	10
3.4.1	Da Forma Forte à Forma Fraca	11
3.4.2	O Método de Galerkin	12
3.4.3	Elementos Triangulares Lineares (P1)	13
3.4.4	Matrizes Elementares	14
3.4.5	Montagem Global	15
3.5	Formulação Vorticidade-Corrente	15
3.5.1	Definições	15
3.5.2	Equação de Transporte da Vorticidade	16
3.5.3	Sistema Acoplado e Solução Iterativa	16
3.5.4	Matriz de Convecção e Estabilização SUPG	16
3.6	Integração Temporal	17
3.6.1	Método θ Generalizado	17
3.6.2	Esquema Semi-Implicito Adotado	18
3.6.3	Condição de Contorno de Vorticidade na Parede	18
3.7	Síntese do Capítulo	19
4	Desenvolvimento e Arquitetura	20
4.1	Visão Geral da Plataforma	20
4.2	Justificativa Tecnológica	22
4.2.1	PyScript e WebAssembly	22
4.2.2	Computação <i>Client-Side</i>	22
4.2.3	Comparação com Alternativas	23
4.3	Arquitetura de Software	24
4.3.1	Organização de Diretórios	24
4.3.2	Fluxo de Execução de uma Simulação	24
4.4	Módulos de Simulação MEF	26
4.4.1	Condução Permanente 1D	26
4.4.2	Condução Transiente 1D com $k(x)$ Variável	27
4.4.3	Convecção-Difusão 1D com Estabilização SUPG	28
4.4.4	Viga 1D — Análise de Flexão (Euler-Bernoulli)	29
4.4.5	Vibração 1D — Problema de Autovalor	30
4.4.6	Transferência de Calor 2D	31

4.4.7	Escoamento Potencial 2D	33
4.4.8	Navier-Stokes 2D (Vorticidade-Corrente)	33
4.5	Síntese do Capítulo	36
5	Resultados e Validação	38
5.1	Introdução	38
5.2	Condução Permanente 1D	39
5.2.1	Formulação do Problema	39
5.2.2	Solução Analítica	40
5.2.3	Solução MEF e Comparação	40
5.2.4	Análise de Erro	41
5.3	Condução Transiente 1D com $k(x)$ Variável	42
5.4	Convecção-Difusão 1D com Estabilização SUPG	42
5.5	Viga 1D — Flexão de Euler-Bernoulli	42
5.6	Vibração 1D — Problema de Autovalor	43
5.7	Transferência de Calor 2D	43
5.8	Escoamento Potencial 2D	43
5.9	Navier-Stokes 2D — Cavidade com Tampa Móvel	43
6	Conclusões	45
6.1	Conclusões	45
6.2	Trabalhos Futuros	45
	Bibliografia	46
A	Listagens dos Módulos de Simulação 1D	52
A.1	Condução Permanente 1D	52
A.2	Condução Transiente 1D com $k(x)$ Variável	55
A.3	Convecção-Difusão 1D com SUPG	64
A.4	Viga 1D — Euler-Bernoulli	70
A.5	Vibração 1D — Problema de Autovalor	83
B	Listagens dos Módulos de Simulação 2D	94
B.1	Módulo Compartilhado: Malha e Matrizes MEF	94
B.2	Módulo Compartilhado: Geometrias de Obstáculo	98

B.3	Transferência de Calor 2D	100
B.4	Escoamento Potencial 2D	104
B.5	Navier-Stokes 2D (Vorticidade-Corrente)	115

Lista de Figuras

4.1	Tela inicial do <i>MinervaMesh</i> com o menu de simulações disponíveis. .	21
4.2	Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.	25
4.3	Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.	27
4.4	Evolução temporal do campo de temperatura com condutividade variável $k(x)$	28
4.5	Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.	29
4.6	Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.	30
4.7	Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.	31
4.8	Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).	32
4.9	Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.	33
4.10	Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.	35
4.11	Navier-Stokes 2D — linhas de corrente para escoamento ao redor de um cilindro.	36
5.1	Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos P1) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes.	41

5.2	(Esqueleto) Evolução temporal do campo $T(x, t)$ e convergência ao regime permanente.	42
5.3	(Esqueleto) Comparação Galerkin vs. SUPG em alto número de Péclet.	42
5.4	(Esqueleto) Deflexão, momento fletor e esforço cortante para viga simplesmente apoiada com carga uniforme.	42
5.5	(Esqueleto) Cinco primeiros modos de vibração e respectivas frequências naturais.	43
5.6	(Esqueleto) Campo de temperatura em placa 2D durante a evolução transiente.	43
5.7	(Esqueleto) Linhas de corrente ao redor do cilindro e distribuição de C_p na superfície.	43
5.8	(Esqueleto) Campos de ψ , ω e linhas de corrente na cavidade a $Re = 100$	44
5.9	Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia, Ghia & Shin (1982) para a cavidade com tampa móvel a $Re = 100$. RMS = 0,036 em malha 41×41 com 2000 passos de tempo. Figura gerada pelo runner <code>.agents/review/thom_validation/run_cavity.py</code> com a fórmula de Thom na forma canônica.	44

Lista de Tabelas

4.1	Comparação entre plataformas de simulação numérica para ensino. . .	23
5.1	Parâmetros da simulação de condução permanente 1D.	40

Capítulo 1

Introdução

1.1 Introdução

A simulação numérica consolidou-se como instrumento central da engenharia contemporânea, complementando abordagens teóricas e experimentais na investigação de fenômenos físicos complexos. No contexto da Engenharia Mecânica, a Dinâmica dos Fluidos Computacional (*Computational Fluid Dynamics* – CFD) e a modelagem de transferência de calor viabilizam a análise de problemas governados por Equações Diferenciais Parciais (EDPs), muitas vezes inviáveis por solução analítica direta ou por experimentação em escala de laboratório [1, 2]. Essa capacidade preditiva tem papel relevante no dimensionamento, na otimização e na verificação de desempenho de sistemas térmicos e fluidodinâmicos.

Apesar dos avanços metodológicos, o acesso a ambientes de simulação permanece limitado por barreiras econômicas e operacionais. Soluções comerciais amplamente utilizadas exigem licenciamento oneroso, infraestrutura computacional específica e configuração de ambiente com elevado custo de entrada, sobretudo em cenários de ensino e pesquisa inicial. Como consequência, a formação em métodos numéricos frequentemente fica condicionada à disponibilidade de laboratório e suporte técnico especializado, o que reduz a reprodutibilidade e a acessibilidade das atividades acadêmicas [3].

Neste contexto, este trabalho propõe o *MinervaMesh*, uma plataforma de simulação numérica executada integralmente no navegador, com arquitetura *client-*

side. A solução combina Python científico com tecnologias web modernas, notadamente *WebAssembly* e *PyScript*, permitindo a execução local de rotinas de MEF e MDF sem instalação de dependências pelo usuário [4, 5]. A contribuição principal não se restringe à implementação computacional, mas inclui a organização de um ambiente didático de engenharia para formulação, solução e análise de problemas térmicos e fluidodinâmicos.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver e validar o **MinervaMesh**, uma plataforma computacional baseada na web para solução de EDPs de interesse em Engenharia Mecânica, operando integralmente no lado do cliente (*client-side*).

Os objetivos específicos são:

1. Implementar métodos numéricos clássicos, com ênfase no Método dos Elementos Finitos (MEF) e uso complementar do Método das Diferenças Finitas (MDF), empregando bibliotecas científicas Python no ambiente web [5].
2. Desenvolver uma interface interativa para pré-processamento, solução e pós-processamento, incluindo parametrização de malha e visualização de campos de interesse.
3. Validar a plataforma, prioritariamente, por meio de casos com solução analítica conhecida e, em seguida, demonstrar sua aplicação em casos adicionais de engenharia, com foco em mecânica dos fluidos e transferência de calor.
4. Consolidar um fluxo de trabalho reprodutível para problemas de:
 - Escoamentos potenciais;
 - Problemas de advecção-difusão (com estabilização SUPG) [6];
 - Equações de Navier-Stokes para escoamentos laminares incompressíveis.

1.3 Justificativa

A relevância do trabalho reside na articulação entre rigor numérico e acessibilidade computacional. Ao viabilizar simulações diretamente no navegador, reduz-se o atrito inicial de aprendizado e amplia-se o potencial de uso em disciplinas de graduação, monitorias e estudos independentes. Adicionalmente, a explicitação da formulação matemática e de sua implementação computacional fortalece o caráter acadêmico da proposta, permitindo que a plataforma seja utilizada tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos [7, 8].

1.4 Organização do Trabalho

Este trabalho está estruturado da seguinte forma:

- O **Capítulo 2** (Revisão Bibliográfica) apresenta o histórico e o estado da arte dos temas que sustentam o trabalho, incluindo problemas térmicos, MDF, MEF e trabalhos correlatos.
- O **Capítulo 3** (Fundamentação Teórica) apresenta o embasamento matemático necessário para a discretização de EDPs via MEF.
- O **Capítulo 4** (Desenvolvimento e Arquitetura) detalha a implementação do *MinervaMesh*, com ênfase na arquitetura *client-side* e PyScript.
- O **Capítulo 5** (Resultados e Validação) apresenta a verificação numérica em casos com solução analítica e casos de aplicação.
- O **Capítulo 6** (Conclusão) resume as contribuições e propõe trabalhos futuros.

Capítulo 2

Revisão Bibliográfica

2.1 Introdução ao Capítulo

Este capítulo apresenta a revisão bibliográfica que fundamenta o desenvolvimento do *MinervaMesh*, com foco em problemas térmicos e de mecânica dos fluidos resolvidos por Métodos Numéricos, em especial o Método das Diferenças Finitas (MDF) e o Método dos Elementos Finitos (MEF). A revisão está organizada em quatro blocos: fundamentos de transferência de calor, bases dos métodos numéricos, marcos do MEF/CFD e trabalhos correlatos.

2.2 Problemas Térmicos e Equações Governantes

2.2.1 Condução de Calor

A condução é descrita pela Lei de Fourier e pela equação da difusão térmica. Em regime estacionário, a formulação conduz às equações de Laplace e Poisson; em regime transiente, inclui-se o termo temporal associado ao armazenamento de energia [9, 10, 11, 12]. Esses modelos são a base de problemas clássicos de engenharia, como dissipação em componentes eletrônicos e análise térmica de discos de freio.

2.2.2 Convecção e Acoplamento com Escoamento

Quando há interação sólido-fluido, a convecção influencia a taxa de remoção de calor e exige o acoplamento entre equações de energia e de escoamento. A formulação

por volumes/elementos finitos para problemas convectivo-difusivos é amplamente discutida na literatura de CFD [1, 2, 13, 14, 15].

2.2.3 Radiação Térmica

Em aplicações de alta temperatura, os termos radiativos podem se tornar relevantes no balanço térmico global [9, 11]. Embora o escopo principal deste trabalho esteja em condução e convecção, a revisão considera radiação como extensão natural para trabalhos futuros.

2.3 Métodos Numéricos para EDPs

2.3.1 Método das Diferenças Finitas (MDF)

O MDF aproxima derivadas por expansões de Taylor, convertendo EDPs em sistemas algébricos. Sua implementação é direta em malhas estruturadas e, por isso, ele é frequentemente adotado em validações iniciais e em discretização temporal de problemas transientes [2, 16].

Esquemas explícitos e implícitos (Euler e Crank-Nicolson) apresentam diferentes compromissos entre custo computacional, estabilidade e precisão. Em problemas dominados por difusão, esquemas implícitos são preferidos por robustez numérica [2, 1].

2.3.2 Método dos Elementos Finitos (MEF)

O MEF oferece maior flexibilidade geométrica por meio de malhas não estruturadas e formulação variacional. Essa característica é decisiva em domínios complexos, comuns em aplicações reais de engenharia térmica e de fluidos [7, 17, 8, 18, 19, 20, 21].

A formulação de Galerkin transforma a EDP na forma fraca e permite a montagem de matrizes globais de rigidez, massa e convecção. Para elementos lineares triangulares, a implementação é eficiente e adequada a plataformas computacionais didáticas [8, 18, 7].

2.4 Bases Históricas e Consolidação do MEF/CFD

A literatura mostra uma evolução contínua desde os fundamentos variacionais até formulações estabilizadas para escoamentos com convecção dominante. Trabalhos de Galerkin, Hrennikoff e Courant consolidaram a base matemática para aproximações por partes [22, 23, 24]. Em seguida, contribuições de Turner e Clough estruturaram o MEF moderno aplicado à engenharia [25, 26].

No contexto de transporte convectivo e Navier-Stokes, formulações Petrov-Galerkin/SUPG reduziram oscilações numéricas e ampliaram a faixa de estabilidade dos métodos [6, 27, 28, 29]. Benchmarks clássicos de cavidade e degrau regressivo permanecem referências para validação de códigos [30, 31, 32, 33, 34].

Também se destaca a relevância de ferramentas de geração de malha na qualidade da solução, com destaque para o Gmsh em fluxos de trabalho acadêmicos e de pesquisa [35].

2.5 Trabalhos Correlatos (Ênfase em TCCs Semelhantes)

Em trabalhos de graduação recentes, observa-se forte convergência para aplicações de condução de calor e CFD com implementação numérica em Python. Na área térmica, destacam-se estudos sobre discos de freio, dissipadores e componentes eletrônicos heterogêneos [36, 37, 38, 39, 40, 41, 42, 43]. Esses trabalhos reforçam a importância de malhas adequadas, tratamento consistente de contornos e validação por casos de referência.

Na linha de validação metodológica, a comparação entre solução analítica, MDF e MEF foi explorada por Araujo [44], fornecendo evidências de convergência e comportamento de erro em diferentes discretizações. Em aplicações de fluidos e transferência de calor conjugada, também há contribuições relevantes com foco em escoamento para arrefecimento de eletrônicos e análise de campos de velocidade/vorticidade [45, 46, 47, 48, 49].

No contexto local mais recente, trabalhos como os de Gomes [50] e Oliveira [43] reforçam a continuidade da linha de pesquisa em transferência de calor com viés variacional/numérico e MEF tridimensional.

2.6 Síntese da Revisão e Lacuna Atacada

A revisão indica que: (i) há base teórica consolidada para problemas térmicos e de escoamento; (ii) MDF e MEF são métodos complementares, com o MEF mais adequado para geometrias complexas; e (iii) existe demanda por ferramentas didáticas acessíveis para implementação e validação desses métodos.

Nesse cenário, o *MinervaMesh* se posiciona ao integrar formulações clássicas de engenharia numérica em uma plataforma web, com foco em acessibilidade, reprodutibilidade e uso educacional, apoiando-se em bibliotecas científicas amplamente adotadas [5, 4].

Capítulo 3

Fundamentação Teórica

3.1 Introdução ao Capítulo

Este capítulo apresenta a formulação matemática e os métodos numéricos que fundamentam as análises realizadas no *MinervaMesh*. O objetivo central é fornecer uma base sólida, típica da Engenharia Mecânica, para a conversão dos modelos físicos contínuos em sistemas algébricos discretos resolvidos computacionalmente.

Inicialmente, revisam-se as equações diferenciais parciais (EDPs) governantes da condução de calor térmica e do escoamento de fluidos de forma conceitual. Em seguida, após uma breve fundamentação do Método das Diferenças Finitas (MDF), o capítulo se aprofunda extensamente no Método dos Elementos Finitos (MEF). O desenvolvimento abrange desde a formulação de Galerkin, passando pela discretização com elementos triangulares de primeira ordem (P1), até a consolidação da formulação em Vorticidade-Corrente adotada para a dinâmica de fluidos.

3.2 Equações Governantes

A modelagem térmica e fluidodinâmica sustenta-se em princípios de conservação: massa, quantidade de movimento e energia. Estas leis físicas, contínuas no espaço e no tempo, são expressas por EDPs que determinam o comportamento termofluidodinâmico do meio contínuo abordado.

3.2.1 Condução e Convecção Térmica

A transferência de calor em meios sólidos contínuos é regida pela equação da difusão de calor (Lei de Fourier acoplada à conservação da energia térmica). Para um meio estacionário, a distribuição de temperatura $T(\mathbf{x}, t)$ é ditada por [9]:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + q''' \quad (3.1)$$

onde ρ é a massa específica, c_p é o calor específico a pressão constante, k é a condutividade térmica do material, e q''' representa a taxa volumétrica de geração de energia térmica. Para regimes permanentes e propriedades constantes, a Equação 3.1 se simplifica na clássica equação de Poisson (ou Laplace, se $q''' = 0$).

Quando existe escoamento (*convecção*), adiciona-se à equação o transporte advectivo promovido pelo campo de velocidades $\mathbf{v}$:

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = \nabla \cdot (k \nabla T) + q''' \quad (3.2)$$

Esta é a equação de convecção-difusão, paradigma central para o estudo da transferência de calor conjugada ou escoamentos não-isotérmicos.

3.2.2 Escoamento de Fluidos (Navier-Stokes)

O escoamento de fluidos incompressíveis newtonianos é regido pela restrição de continuidade (conservação de massa) e pelas Equações de Navier-Stokes (conservação do momento linear) [14]:

$$\nabla \cdot \mathbf{v} = 0 \quad (3.3)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}_b \quad (3.4)$$

onde p representa o campo de pressão estática, μ a viscosidade dinâmica do fluido e $\mathbf{f}_b$ eventuais forças de campo (como as forças de empuxo gravitacional). Esta formulação em variáveis primitivas (velocidade-pressão) apresenta consideráveis desafios numéricos devido ao acoplamento embutido pela restrição de continuidade [1]. Consequentemente, abordagens numéricas como a Vorticidade-Corrente são frequentemente adotadas em 2D para contornar a incógnita da pressão diretamente.

3.3 Transição Discreta: O Método das Diferenças Finitas (MDF)

Antes do extensivo emprego de formulações variacionais associadas a malhas arbitrárias, a conversão de EDPs a um sistema matricial utilizou maciçamente as propriedades da expansão da série de Taylor. O Método das Diferenças Finitas (MDF) consiste na substituição direta dos operadores diferenciais exatos pelas respectivas aproximações algébricas discretizadas em nós de uma malha tipicamente ortogonal e estruturada [3].

Considere a representação da derivada parcial aproximada pelo método da diferença adiantada e atrasada para aproximações de derivadas espaciais e temporais:

$$\left. \frac{\partial u}{\partial x} \right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x} \quad (\text{Diferença Central}) \quad (3.5)$$

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (3.6)$$

Embora de inegável valor histórico e de extrema eficácia para discretizações explícitas no tempo e problemas de topologia regular simples, o MDF apresenta entraves práticos quando restrições de domínios geométricos altamente complexos exigem representações intrincadas de fronteiras, resultando em interpolações estérreo ou *stair-step* dos contornos físicos do domínio fluido ou de sólidos.

Para dirimir essa dependência geométrica e preservar maior rigor ao tratamento das condições de contorno de von Neumann ou mistas na fronteira, o presente trabalho avança, a seguir, na formulação integral provida pelo Método dos Elementos Finitos (MEF).

3.4 O Método dos Elementos Finitos (MEF)

O Método dos Elementos Finitos constitui a abordagem numérica central adotada neste trabalho. Diferentemente do MDF, o MEF parte de uma formulação integral (variacional ou de resíduos ponderados) da EDP, permitindo o uso de malhas não estruturadas que se adaptam naturalmente a geometrias complexas [18, 8].

Esta seção desenvolve a formulação desde a forma forte até a obtenção do sistema algébrico global.

3.4.1 Da Forma Forte à Forma Fraca

Considere, para fins de generalidade, uma EDP elíptica em um domínio $\Omega \subset \mathbb{R}^d$ com fronteira $\Gamma = \partial\Omega$. De maneira genérica, a *forma forte* do problema visa encontrar $u(\mathbf{x})$ tal que:

$$\mathcal{L}(u) = f \quad \text{em } \Omega \quad (3.7)$$

sujeita às condições de contorno:

$$u = \bar{u} \quad \text{em } \Gamma_D \quad (\text{Dirichlet}) \quad (3.8)$$

$$k \frac{\partial u}{\partial n} = \bar{q} \quad \text{em } \Gamma_N \quad (\text{Neumann}) \quad (3.9)$$

onde $\mathcal{L}$ é um operador diferencial (por exemplo, $-\nabla \cdot (k \nabla(\cdot))$ para condução), f é o termo fonte, $\bar{u}$ é o valor prescrito na fronteira de Dirichlet Γ_D , $\bar{q}$ é o fluxo prescrito na fronteira de Neumann Γ_N , e $\Gamma_D \cup \Gamma_N = \Gamma$.

A *forma fraca* (ou variacional) é obtida multiplicando-se a Equação 3.7 por uma *função-teste* (ou função de ponderação) $w \in \mathcal{V}_0$, pertencente a um espaço funcional adequado, e integrando-se sobre o domínio Ω :

$$\int_{\Omega} w \mathcal{L}(u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.10)$$

A vantagem fundamental desta operação reside na possibilidade de aplicar o Teorema de Green (integração por partes generalizada), transferindo uma ordem de derivação do operador diferencial $\mathcal{L}$ para a função-teste w . Considere o caso concreto da equação de Poisson estacionária ($-\nabla \cdot (k \nabla u) = f$). Multiplicando por w e integrando:

$$-\int_{\Omega} w \nabla \cdot (k \nabla u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.11)$$

Aplicando o Teorema de Green ao lado esquerdo:

$$\int_{\Omega} k \nabla w \cdot \nabla u d\Omega - \int_{\Gamma_N} w k \frac{\partial u}{\partial n} d\Gamma = \int_{\Omega} w f d\Omega \quad (3.12)$$

Substituindo a condição de Neumann (Eq. 3.9) no termo de contorno, obtém-se a *forma fraca*: encontrar $u \in \mathcal{V}$ tal que, para todo $w \in \mathcal{V}_0$:

$$\boxed{\int_{\Omega} k \nabla w \cdot \nabla u \, d\Omega = \int_{\Omega} w f \, d\Omega + \int_{\Gamma_N} w \bar{q} \, d\Gamma} \quad (3.13)$$

Esta expressão é de importância central: ela reduz os requisitos de regularidade sobre u (de C^2 para H^1), incorpora naturalmente as condições de Neumann no funcional, e constitui a base sobre a qual o MEF opera.

3.4.2 O Método de Galerkin

O método de Galerkin consiste em restringir o problema variacional contínuo (Eq. 3.13) a um subespaço de dimensão finita $\mathcal{V}^h \subset \mathcal{V}$, parametrizado por uma malha de elementos finitos. A solução aproximada u^h é expressa como uma combinação linear de N funções de base $\{N_j\}_{j=1}^N$:

$$u^h(\mathbf{x}) = \sum_{j=1}^N U_j N_j(\mathbf{x}) \quad (3.14)$$

onde U_j são os coeficientes nodais (incógnitas do sistema discreto) e $N_j(\mathbf{x})$ são as funções de forma associadas a cada nó j da malha.

A escolha fundamental do método de Galerkin é adotar as *mesmas* funções de base como funções-teste: $w^h = N_i$, $i = 1, \dots, N$. Substituindo na forma fraca (Eq. 3.13):

$$\sum_{j=1}^N U_j \int_{\Omega} k \nabla N_i \cdot \nabla N_j \, d\Omega = \int_{\Omega} N_i f \, d\Omega + \int_{\Gamma_N} N_i \bar{q} \, d\Gamma \quad \forall i = 1, \dots, N \quad (3.15)$$

Esta expressão define um sistema linear $\mathbf{KU} = \mathbf{F}$, onde:

$$K_{ij} = \int_{\Omega} k \nabla N_i \cdot \nabla N_j \, d\Omega \quad (\text{Matriz de Rigidez}) \quad (3.16)$$

$$F_i = \int_{\Omega} N_i f \, d\Omega + \int_{\Gamma_N} N_i \bar{q} \, d\Gamma \quad (\text{Vetor de Carga}) \quad (3.17)$$

A resolução deste sistema algébrico fornece os coeficientes nodais $\mathbf{U}$, a partir dos quais o campo aproximado u^h é reconstituído via Equação 3.14. A qualidade da

aproximação depende diretamente da riqueza do espaço $\mathcal{V}^h$, controlada pelo refinamento da malha e pela ordem polinomial das funções de base [7].

De maneira compacta, o procedimento de Galerkin converte o problema contínuo em:

$$\boxed{\mathbf{K}\mathbf{U} = \mathbf{F}} \quad (3.18)$$

A construção efetiva das matrizes $\mathbf{K}$ e do vetor $\mathbf{F}$ depende da escolha dos elementos e das funções de forma, desenvolvida na seção seguinte.

3.4.3 Elementos Triangulares Lineares (P1)

A discretização do domínio Ω é realizada por uma malha de N_e elementos triangulares, cujos vértices coincidem com os N nós globais da malha. Cada elemento e possui três nós locais $(1, 2, 3)$, com coordenadas (x_i, y_i) , $i = 1, 2, 3$.

As funções de forma lineares N_i^e para o elemento triangular P1 são definidas de modo que $N_i^e(\mathbf{x}_j) = \delta_{ij}$ (propriedade de partição da unidade nodal). Em coordenadas cartesianas, estas funções são expressas por [8]:

$$N_i^e(x, y) = \frac{1}{2A^e}(a_i + b_i x + c_i y), \quad i = 1, 2, 3 \quad (3.19)$$

onde A^e é a área do elemento triangular e os coeficientes são dados por permutação cíclica dos índices dos vértices:

$$a_i = x_j y_k - x_k y_j, \quad b_i = y_j - y_k, \quad c_i = x_k - x_j \quad (3.20)$$

com (i, j, k) em ordem cíclica $(1, 2, 3)$, $(2, 3, 1)$, $(3, 1, 2)$. A área do elemento é calculada por:

$$A^e = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)| \quad (3.21)$$

Os gradientes das funções de forma são constantes dentro de cada elemento P1:

$$\nabla N_i^e = \frac{1}{2A^e} \begin{pmatrix} b_i \\ c_i \end{pmatrix} \quad (3.22)$$

Esta propriedade simplifica consideravelmente as integrais elementares, uma vez que os integrandos envolvendo $\nabla N_i \cdot \nabla N_j$ tornam-se constantes no interior do elemento.

3.4.4 Matrizes Elementares

A partir das funções de forma P1, as integrais da formulação de Galerkin (Eqs. 3.16 e 3.17) são calculadas elemento a elemento e, em seguida, assembradas no sistema global.

3.4.4.1 Matriz de Rigidez Elementar

Para o problema de difusão com condutividade k constante por elemento, a matriz de rigidez elementar $\mathbf{K}^e$ de dimensão 3×3 é:

$$K_{ij}^e = \int_{\Omega^e} k \nabla N_i^e \cdot \nabla N_j^e d\Omega = \frac{k}{4A^e} (b_i b_j + c_i c_j) \quad (3.23)$$

3.4.4.2 Matriz de Massa Elementar

Em problemas transientes, a discretização temporal introduz um termo de acumulação que requer a *matriz de massa*. Para elementos P1, a matriz de massa consistente é [18]:

$$M_{ij}^e = \int_{\Omega^e} \rho c_p N_i^e N_j^e d\Omega = \frac{\rho c_p A^e}{12} \begin{cases} 2 & \text{se } i = j \\ 1 & \text{se } i \neq j \end{cases} \quad (3.24)$$

Alternativamente, a *matriz de massa concentrada* (*lumped*) distribui a massa uniformemente nos nós, resultando em uma matriz diagonal:

$$M_{ij}^{e,\text{lump}} = \frac{\rho c_p A^e}{3} \delta_{ij} \quad (3.25)$$

3.4.4.3 Vetor de Carga Elementar

Para um termo fonte f uniforme no elemento, o vetor de carga elementar resulta em:

$$F_i^e = \int_{\Omega^e} N_i^e f d\Omega = \frac{f A^e}{3} \quad (3.26)$$

3.4.5 Montagem Global

A construção do sistema global $\mathbf{KU} = \mathbf{F}$ é realizada pelo procedimento de *assembly*, que consiste em acumular as contribuições de cada elemento nas posições globais correspondentes. Para cada elemento e , a conectividade local-global é definida por um mapa de índices $\text{IEN}(a, e) \rightarrow I$, que associa o nó local a do elemento e ao nó global I .

A montagem é expressa como:

$$K_{IJ} = \sum_{e=1}^{N_e} K_{\text{IEN}(a,e), \text{IEN}(b,e)}^e, \quad F_I = \sum_{e=1}^{N_e} F_{\text{IEN}(a,e)}^e \quad (3.27)$$

Após a montagem, as condições de contorno de Dirichlet são impostas modificando-se as linhas correspondentes do sistema. A resolução do sistema linear resultante fornece os valores nodais $\mathbf{U}$.

3.5 Formulação Vorticidade-Corrente

A resolução direta das equações de Navier-Stokes em variáveis primitivas (velocidade-pressão) apresenta dificuldades numéricas associadas ao acoplamento pressão-velocidade e à satisfação da restrição de continuidade (Eq. 3.3). Em problemas bidimensionais, a formulação em *Vorticidade-Corrente* (ω - ψ) oferece uma alternativa eficaz, eliminando o campo de pressão das incógnitas e reduzindo o sistema a duas equações escalares [1, 2].

3.5.1 Definições

Para um escoamento bidimensional, define-se a *função corrente* $\psi(x, y, t)$ tal que o campo de velocidades satisfaça identicamente a equação da continuidade:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (3.28)$$

A *vorticidade*, componente escalar no plano 2D, é definida como o rotacional do campo de velocidades:

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (3.29)$$

Substituindo as definições da função corrente na expressão da vorticidade, obtém-se a *equação de Poisson para a função corrente*:

$$\nabla^2 \psi = -\omega \quad (3.30)$$

3.5.2 Equação de Transporte da Vorticidade

Aplicando o rotacional às equações de Navier-Stokes (Eq. 3.4), o gradiente de pressão é eliminado e obtém-se a equação de transporte da vorticidade:

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \nu \nabla^2 \omega \quad (3.31)$$

onde $\nu = \mu/\rho$ é a viscosidade cinemática. Esta equação possui a estrutura clássica de convecção-difusão, onde o campo de velocidades (u, v) transporta a vorticidade ω enquanto a difusão viscosa a dissipa.

3.5.3 Sistema Acoplado e Solução Iterativa

O sistema ω - ψ é resolvido iterativamente. A cada passo de tempo (ou iteração estacionária), o procedimento consiste em:

1. Resolver a equação de Poisson (Eq. 3.30) para ψ , dado ω .
2. Calcular o campo de velocidades (u, v) a partir de ψ (Eq. 3.28).
3. Resolver a equação de transporte (Eq. 3.31) para ω , dado (u, v) .
4. Verificar a convergência e repetir, se necessário.

3.5.4 Matriz de Convecção e Estabilização SUPG

A discretização via MEF do operador convectivo $\mathbf{v} \cdot \nabla \phi$ conduz à *matriz de convecção* elementar, de dimensão 3×3 para elementos P1:

$$C_{ij}^e = \int_{\Omega^e} N_i (\mathbf{v} \cdot \nabla N_j) d\Omega = \frac{A^e}{3} \left(\frac{u_e b_j + v_e c_j}{2A^e} \right) \quad (3.32)$$

onde u_e e v_e são as componentes de velocidade avaliadas no centroide do elemento.

Quando o campo de velocidades é intenso em relação à difusão (número de Péclet elevado), a formulação de Galerkin padrão produz oscilações espúrias. Para contornar essa limitação, emprega-se a técnica *Streamline Upwind Petrov-Galerkin* (SUPG), que adiciona uma difusão artificial orientada na direção do escoamento [28, 6].

Na formulação SUPG, a função-teste é modificada para $w_i = N_i + \tau_{\text{SUPG}} \mathbf{v} \cdot \nabla N_i$, com o parâmetro de estabilização:

$$\tau_{\text{SUPG}} = \frac{h_e}{2\|\mathbf{v}\|} \left(\coth \text{Pe}_e - \frac{1}{\text{Pe}_e} \right) \quad (3.33)$$

onde $\text{Pe}_e = \|\mathbf{v}\| h_e / (2\alpha)$ é o número de Péclet elementar.

No *MinervaMesh*, a estabilização SUPG é empregada no caso unidimensional de convecção-difusão, onde o efeito do número de Péclet é mais crítico. No solver bidimensional de Navier-Stokes, a convecção da vorticidade utiliza a formulação de Galerkin padrão (Eq. 3.32), sendo a estabilidade controlada pela escolha adequada do passo de tempo e do número de Reynolds.

3.6 Integração Temporal

Para problemas transientes, a discretização temporal converte a EDP semi-discreta (após a discretização espacial por MEF) em um sistema algébrico a cada intervalo de tempo Δt . No caso geral, o sistema semidiscreto assume a forma:

$$\mathbf{M} \frac{d\mathbf{U}}{dt} + \mathbf{K}\mathbf{U} = \mathbf{F}(t) \quad (3.34)$$

3.6.1 Método θ Generalizado

A família de métodos θ parametriza a ponderação temporal entre os instantes t^n e t^{n+1} :

$$\mathbf{M} \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t} + \mathbf{K} [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] = \theta \mathbf{F}^{n+1} + (1 - \theta) \mathbf{F}^n \quad (3.35)$$

Os casos particulares mais relevantes são:

- $\theta = 0$: Euler explícito (condicionalmente estável);
- $\theta = 1/2$: Crank-Nicolson (incondicionalmente estável, segunda ordem);
- $\theta = 1$: Euler implícito (incondicionalmente estável, primeira ordem).

3.6.2 Esquema Semi-Implícito Adotado

No solver de Navier-Stokes do *MinervaMesh*, a equação de transporte da vorticidade (Eq. 3.31) é discretizada com um esquema *semi-implícito*: o termo difusivo é tratado implicitamente ($\theta = 1$), enquanto o termo convectivo é avaliado explicitamente no instante t^n . Isso resulta no sistema:

$$\left(\mathbf{M} + \frac{\Delta t}{\text{Re}} \mathbf{K} \right) \boldsymbol{\omega}^{n+1} = \mathbf{M} \boldsymbol{\omega}^n - \Delta t \mathbf{C}^n \boldsymbol{\omega}^n \quad (3.36)$$

onde $\mathbf{C}^n$ é a matriz de convecção assembrada a partir do campo de velocidades no instante atual e $\text{Re} = \rho U L / \mu$ é o número de Reynolds. O tratamento implícito da difusão garante estabilidade incondicional para esse operador, enquanto a convecção explícita impor restrições sobre Δt associadas ao número de Courant.

Para problemas puramente difusivos (condução transiente), utiliza-se o método θ generalizado (Eq. 3.35), permitindo ao usuário selecionar entre Euler e Crank-Nicolson conforme o compromisso desejado entre precisão e estabilidade.

3.6.3 Condição de Contorno de Vorticidade na Parede

Na formulação ω - ψ , a vorticidade nas paredes sólidas não é conhecida *a priori* e deve ser calculada indiretamente a partir do campo de função corrente. Aplicando uma expansão de Taylor de ψ na direção normal à parede, obtém-se uma aproximação de primeira ordem para ω_w [51, 1]:

$$\omega_w = -\frac{2(\psi_{\text{nb}} - \psi_w)}{h^2} - \frac{2 u_{\text{tan}}}{h} \quad (3.37)$$

onde ψ_w e ψ_{nb} são os valores da função corrente na parede e no nó vizinho mais próximo na direção normal (apontando para o interior do domínio), respectivamente; h é a distância entre esses nós; e u_{tan} é a componente tangencial da velocidade da parede, consistente com a convenção $u = \partial\psi/\partial y$ adotada nesta formulação ($u_{tan} = 0$ para paredes estacionárias e $u_{tan} = U_{lid}$ para a tampa móvel da cavidade com velocidade U_{lid} na direção $+x$).

3.7 Síntese do Capítulo

Este capítulo estabeleceu o arcabouço matemático completo empregado pelo *MinervaMesh*. A formulação parte das EDPs governantes para fenômenos térmicos e fluidodinâmicos, transita pela introdução conceitual do MDF e se aprofunda no MEF via Galerkin. A discretização com elementos triangulares P1 permite a construção explícita das matrizes elementares de rigidez, massa e convecção, enquanto a formulação Vorticidade-Corrente viabiliza a solução de escoamentos 2D sem a necessidade de resolver diretamente o campo de pressão. A integração temporal semi-implícita e a aproximação de primeira ordem para a vorticidade na parede completam o ferramental necessário para a simulação dos casos apresentados no Capítulo 5.

Capítulo 4

Desenvolvimento e Arquitetura

Os módulos computacionais apresentados neste capítulo constituem implementações diretas das formulações matemáticas desenvolvidas no Capítulo 3. Cada simulador corresponde a uma discretização específica obtida via formulação variacional de Galerkin ou, nos casos pertinentes, por diferenças finitas, e a arquitetura do software reflete o fluxo numérico subjacente: discretização espacial (MEF/MDF), montagem do sistema matricial, integração temporal e imposição das condições de contorno. Dessa forma, a estrutura da plataforma reproduz, em nível computacional, a sequência de operações matemáticas que conduz das equações governantes contínuas aos sistemas algébricos discretos efetivamente resolvidos.

4.1 Visão Geral da Plataforma

O *MinervaMesh* é uma plataforma de simulação numérica projetada para operar integralmente no navegador web, sem dependência de servidores de processamento remoto nem instalação de software por parte do usuário. A plataforma reúne implementações dos métodos numéricos apresentados no Capítulo 3 em uma interface interativa que permite a parametrização do problema, a execução da simulação e a visualização dos resultados em uma única página.

A motivação central do projeto reside na eliminação das barreiras de acesso associadas às ferramentas de simulação tradicionais. Ambientes como ANSYS, COMSOL Multiphysics ou mesmo distribuições locais de Python científico exigem licencia-

mento, infraestrutura computacional dedicada e configuração de ambiente — requisitos que frequentemente inviabilizam o uso autônomo em disciplinas de graduação, monitorias e estudos independentes [3]. O *MinervaMesh* contorna essas limitações ao executar todo o processamento no dispositivo do próprio usuário, utilizando o navegador como ambiente de execução.

A plataforma abrange três domínios da Engenharia Mecânica:

- **Transferência de Calor:** condução estacionária e transiente, convecção-difusão com estabilização SUPG, transferência de calor 2D;
- **Mecânica dos Fluidos:** escoamento potencial, Navier-Stokes 2D via formulação Vorticidade-Corrente;
- **Mecânica Estrutural:** flexão de vigas (Euler-Bernoulli) e análise modal de vibração.

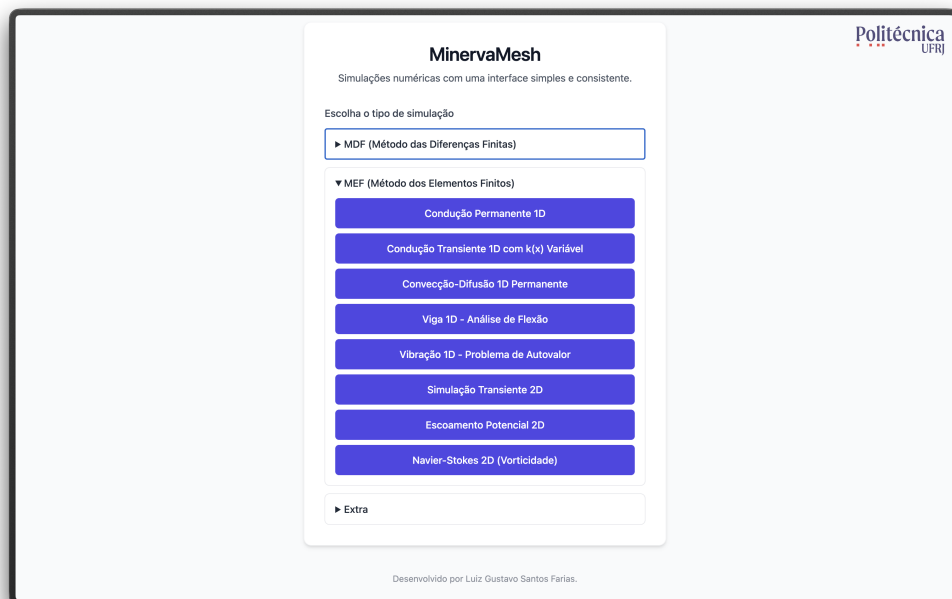


Figura 4.1: Tela inicial do *MinervaMesh* com o menu de simulações disponíveis.

4.2 Justificativa Tecnológica

4.2.1 PyScript e WebAssembly

A escolha do PyScript como runtime da plataforma fundamenta-se em três critérios: portabilidade, familiaridade da linguagem e manutenibilidade.

O PyScript é um *framework* de código aberto que permite a execução de programas Python diretamente no navegador, apoiando-se no Pyodide — uma compilação do interpretador CPython para WebAssembly (Wasm) [4]. O WebAssembly é um formato binário padronizado pelo W3C que executa código de baixo nível em velocidade próxima à nativa, dentro da *sandbox* de segurança do navegador. A cadeia de execução é:

1. O navegador carrega a página HTML contendo a tag `<script type="py">`.
2. O *runtime* do PyScript inicializa o Pyodide, que instancia uma máquina virtual CPython compilada para WebAssembly.
3. As dependências declaradas em `config.toml` (NumPy, SciPy, Matplotlib) são carregadas como pacotes pré-compilados para Wasm.
4. O código Python da simulação é interpretado pelo Pyodide, com acesso direto ao DOM do navegador para leitura de parâmetros e renderização de resultados.

A escolha de Python como linguagem de implementação é estratégica para o contexto educacional: a linguagem apresenta sintaxe próxima à notação matemática e dispõe de um ecossistema científico consolidado — NumPy para álgebra linear, SciPy para solvers esparsos, Matplotlib para visualização [5]. Ao manter a implementação em Python puro, o *MinervaMesh* permite que alunos inspecionem, compreendam e modifiquem o código das simulações — funcionalidade explicitamente disponível através do botão “Ver Código” presente em cada simulação.

4.2.2 Computação *Client-Side*

A arquitetura *client-side* implica que todo o processamento numérico ocorre no dispositivo do usuário. Esta decisão tem impactos diretos em quatro dimensões:

1. **Custo de infraestrutura:** o *MinervaMesh* é um site estático. Pode ser hospedado gratuitamente em plataformas como GitHub Pages ou Netlify, sem necessidade de servidor *backend*, banco de dados ou infraestrutura de *cloud computing*. O custo operacional é efetivamente zero.
2. **Escalabilidade:** não existe gargalo de servidor central. Cada usuário utiliza o poder computacional de sua própria máquina, eliminando filas de processamento e limites de usuários simultâneos.
3. **Privacidade:** nenhum dado de simulação transita pela rede. Parâmetros, malhas e resultados permanecem integralmente no dispositivo do usuário.
4. **Disponibilidade:** após o carregamento inicial dos *assets*, a plataforma pode operar sem conexão com a internet, viabilizando o uso em ambientes com conectividade limitada.

4.2.3 Comparação com Alternativas

A Tabela 4.1 posiciona o *MinervaMesh* frente às alternativas mais comuns no contexto de ensino de simulação numérica.

Tabela 4.1: Comparação entre plataformas de simulação numérica para ensino.

Critério	MinervaMesh	MATLAB	Jupyter	COMSOL
Custo de licença	Gratuito	Pago	Gratuito	Pago
Instalação necessária	Não	Sim	Sim	Sim
Execução no navegador	Sim	Não	Parcial	Não
Código-fonte acessível	Sim	Parcial	Sim	Não
Infraestrutura de servidor	Não	Não	Sim	Sim

A principal limitação da abordagem *client-side* reside na capacidade computacional: problemas com malhas muito refinadas (dezenas de milhares de nós) podem apresentar tempo de execução elevado, uma vez que o desempenho do WebAssembly, embora próximo ao nativo, ainda é inferior ao de implementações compiladas em C/Fortran. Para os casos de interesse deste trabalho, envolvendo malhas de centenas a poucos milhares de nós, essa limitação não se mostrou restritiva.

4.3 Arquitetura de Software

4.3.1 Organização de Diretórios

O repositório do *MinervaMesh* segue uma estrutura modular, onde cada simulação é autocontida em seu próprio diretório. A organização geral é apresentada a seguir:

```
minervamesh/  
|-- index.html           # Menu principal  
|-- assets/             # Logos e imagens globais  
|-- simulations/  
|   |-- mef/            # Elementos Finitos  
|   |   |-- <nome-do-modulo>/  
|   |   |   |-- index.html    # Interface do usuario  
|   |   |   |-- simulation.py  # Logica numerica  
|   |   |   |-- config.toml    # Dependencias Python  
|   |   |   |-- ...           # (8 modulos MEF)  
|   |   |-- escoamento-fluido-2d/  
|   |       |-- common.py      # Matrizes MEF 2D  
|   |       |-- geometry.py    # Geometrias de obstaculo  
|   |-- mdf/            # Diferencas Finitas  
|   |-- extra/          # Ferramentas auxiliares
```

Cada módulo de simulação segue um padrão uniforme de três arquivos:

- `index.html`: interface do usuário, contendo o formulário de parâmetros, o contêiner de resultados e a carga do PyScript;
- `simulation.py`: implementação da lógica numérica em Python, incluindo montagem de matrizes, solução do sistema e geração de gráficos;
- `config.toml`: declaração das dependências Python necessárias (por exemplo, `numpy`, `scipy`, `matplotlib`).

4.3.2 Fluxo de Execução de uma Simulação

O ciclo de vida de uma simulação no *MinervaMesh* pode ser descrito em cinco etapas:

1. **Carregamento:** o navegador requisita o `index.html` de um servidor HTTP simples. A tag `<script type="py">` instrui o PyScript a inicializar o Pyodide e carregar as dependências declaradas em `config.toml`.
2. **Parametrização:** o usuário configura os parâmetros físicos e numéricos através de campos de entrada na interface.
3. **Execução:** ao clicar em “Rodar”, o código Python é executado pelo Pyodide. As matrizes do MEF são montadas, o sistema linear é resolvido e os gráficos são gerados — tudo no navegador.
4. **Visualização:** os gráficos resultantes são convertidos para formato de imagem codificado em Base64 e inseridos dinamicamente no DOM da página.
5. **Inspeção:** o botão “Ver Código” exibe o código-fonte Python da simulação com *syntax highlighting*, permitindo ao aluno estudar a implementação.

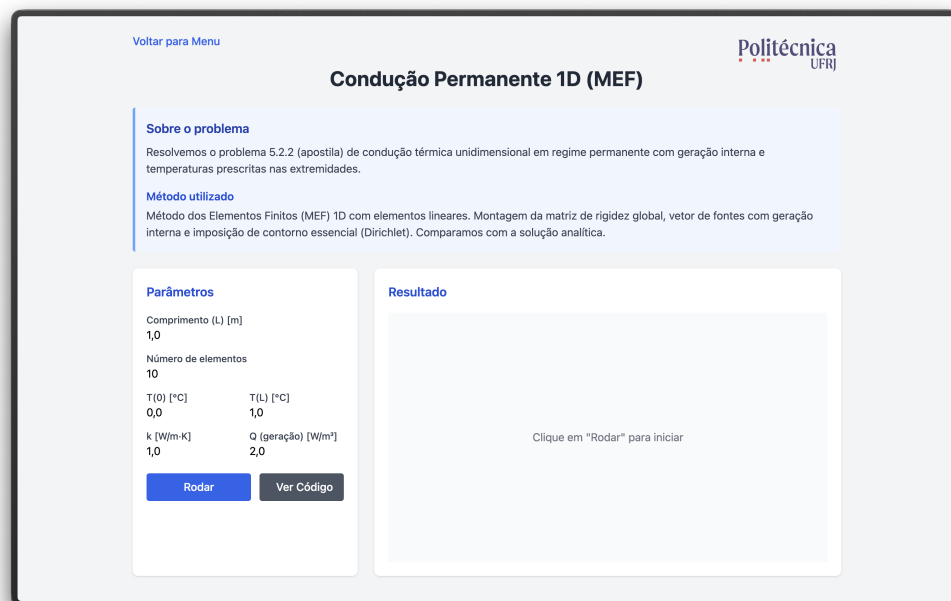


Figura 4.2: Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.

4.4 Módulos de Simulação MEF

Esta seção descreve cada módulo de simulação implementado no *Minerva-Mesh*, detalhando o problema físico resolvido, as decisões de implementação e os resultados obtidos. Os trechos de código apresentados referem-se às funções centrais de cada simulação; as listagens completas encontram-se nos Apêndices.

Todos os módulos implementam diretamente as formulações variacionais e discretizações desenvolvidas no Capítulo 3. As matrizes elementares de rigidez, massa e carga são construídas a partir das expressões derivadas nas Seções 3.4.4 e 3.5.4, e cada trecho de código apresentado nesta seção referencia explicitamente a equação teórica correspondente, mantendo correspondência direta entre operadores matemáticos e operadores numéricos.

4.4.1 Condução Permanente 1D

O primeiro módulo resolve o problema de condução térmica unidimensional em regime permanente com geração interna de calor. A equação governante é a forma 1D da equação de Poisson (Eq. 3.1 com $\partial T / \partial t = 0$):

$$-k \frac{d^2 T}{dx^2} = Q, \quad T(0) = T_0, \quad T(L) = T_L \quad (4.1)$$

A discretização por elementos finitos lineares resulta na matriz de rigidez e no vetor de carga elementares (Seção 3.4.4), implementados como:

```
1 # Elemento linear 1D: ke e be
2 ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
3 be = (Q * h / 2.0) * np.array([1.0, 1.0])
```

A montagem percorre os N_{el} elementos, acumulando as contribuições nas posições globais. Após a imposição das condições de Dirichlet (zerando linhas e colunas correspondentes), o sistema $\mathbf{KT} = \mathbf{b}$ é resolvido por `numpy.linalg.solve`. A Figura 4.3 mostra a comparação entre a solução MEF e a solução analítica.



Figura 4.3: Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.

4.4.2 Condução Transiente 1D com $k(x)$ Variável

Este módulo estende o problema anterior para o regime transiente, incorporando a matriz de massa e a integração temporal conforme o sistema semidiscreto estabelecido na Seção 3.6. A equação governante é:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k(x) \frac{\partial T}{\partial x} \right) \quad (4.2)$$

com condutividade térmica variável $k(x)$, definida como $k_1 = 5,0$ para $x < L/2$ e $k_2 = 1,0$ para $x \geq L/2$.

As matrizes elementares são calculadas por:

```
1 # Matrizes de elemento 1D transiente
2 ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])
```

A integração temporal utiliza o método θ generalizado (Eq. 3.35), seguindo rigorosamente o esquema de avanço temporal derivado na Seção 3.6, e permitindo ao usuário selecionar entre Euler explícito, Crank-Nicolson e Euler implícito. A Figura 4.4 ilustra a evolução temporal do campo de temperatura.

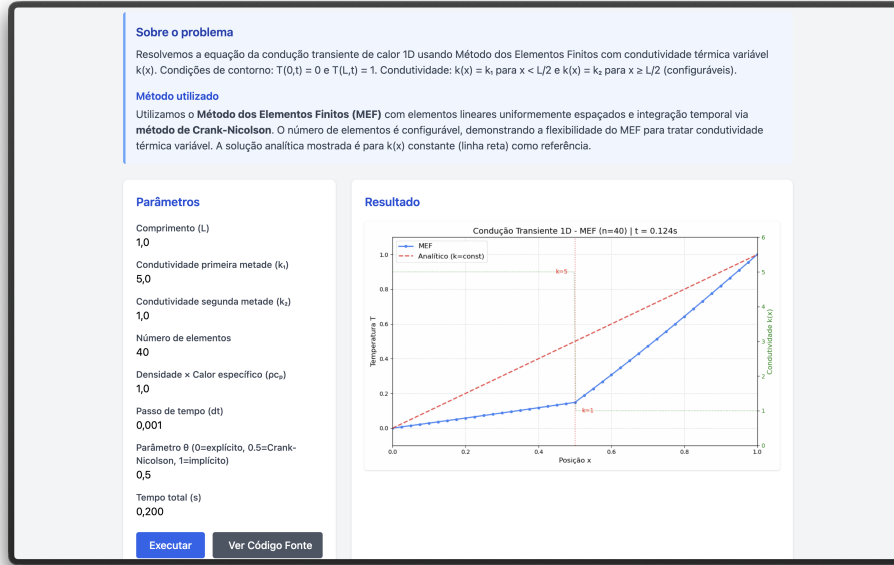


Figura 4.4: Evolução temporal do campo de temperatura com condutividade variável $k(x)$.

4.4.3 Convecção-Difusão 1D com Estabilização SUPG

O módulo de convecção-difusão 1D resolve a equação de advecção-difusão estacionária, aplicando a técnica de estabilização SUPG conforme a formulação desenvolvida na Seção 3.5.4:

$$-k \frac{d^2 T}{dx^2} + \rho c_p u \frac{dT}{dx} = Q \quad (4.3)$$

A discretização MEF gera três matrizes elementares: difusão, convecção e estabilização SUPG (Seção 3.5.4). O parâmetro τ_{SUPG} é calculado segundo a expressão analítica da Eq. 3.33, implementado como:

```
1 # Numero de Peclet local e tau SUPG
2 Pe_local = (rho_cp * u * h) / (2 * k)
3 tau = h / (2*abs(u)) * (1/np.tanh(abs(Pe_local))
4                               - 1/abs(Pe_local))
```

A Figura 4.5 demonstra o efeito da estabilização SUPG em um problema com número de Péclet elevado. Sem estabilização, a formulação de Galerkin padrão produz oscilações espúrias; com SUPG, a solução numérica converge suavemente

para a solução analítica.

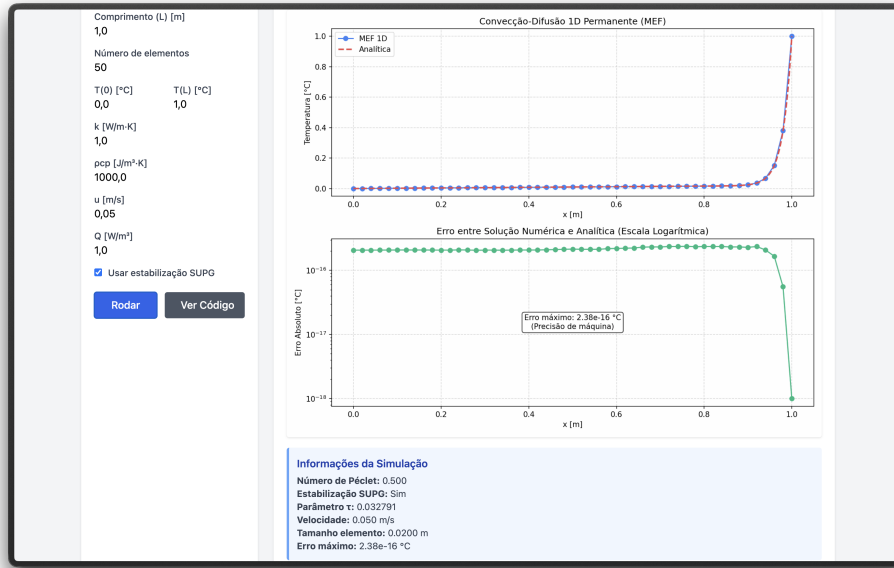


Figura 4.5: Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.

4.4.4 Viga 1D — Análise de Flexão (Euler-Bernoulli)

O módulo de viga 1D implementa elementos finitos de Euler-Bernoulli com dois graus de liberdade por nó: deslocamento vertical w e rotação θ , seguindo o procedimento de Galerkin apresentado na Seção 3.4.2 com funções de interpolação cúbicas de Hermite. A equação governante para a flexão estática é:

$$\frac{d^2}{dx^2} \left(EI \frac{d^2 w}{dx^2} \right) = q(x) \quad (4.4)$$

A matriz de rigidez elementar 4×4 é construída com as funções de interpolação cúbicas de Hermite:

```
1  # Matriz de rigidez do elemento de viga
2  ke = (EI / h**3) * np.array([
3      [12,    6*h,   -12,    6*h ],
4      [6*h,   4*h**2, -6*h,   2*h**2],
5      [-12,  -6*h,    12,   -6*h ],
6      [6*h,   2*h**2, -6*h,   4*h**2]
7  ])
```


O módulo suporta quatro condições de contorno (simplesmente apoiada, balanço, engastada-engastada e engastada-apoiada) e cinco tipos de carregamento (uniforme, linear, triangular, senoidal e pontual), além de propriedades materiais variáveis ao longo do comprimento. A Figura 4.6 apresenta a análise completa de uma viga, incluindo deflexão, diagramas de momento fletor e esforço cortante.

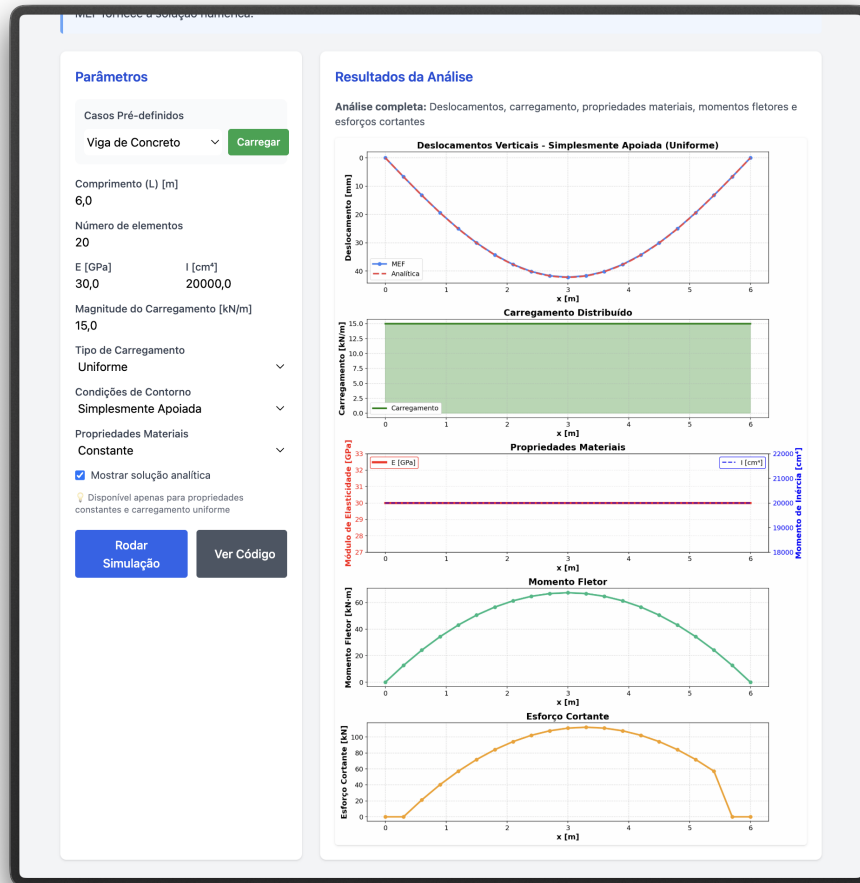


Figura 4.6: Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.

4.4.5 Vibração 1D — Problema de Autovalor

O módulo de vibração resolve o problema de autovalor generalizado $\mathbf{K}\phi = \lambda \mathbf{M}\phi$ para uma barra elástica 1D, onde as frequências naturais são $f_n = \sqrt{\lambda_n}/(2\pi)$ e os autovetores ϕ_n representam os modos de vibração. As matrizes de rigidez e massa são construídas conforme o procedimento de montagem global descrito na Seção 3.4.5, empregando massa concentrada (*lumped*, Eq. 3.25):

```

1 # Matrizes de rigidez e massa (barra 1D)
2 ke = (E*A / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho*A*h / 2) * np.array([[1, 0], [0, 1]])

```

O problema de autovalor é resolvido pela rotina `scipy.linalg.eigh`, que explora a simetria das matrizes para eficiência e estabilidade numérica. A solução fornece n frequências naturais e os correspondentes modos de vibração, comparáveis com as expressões analíticas clássicas — por exemplo, $f_n = nc/(2L)$ para barras engastadas em ambas as extremidades, onde $c = \sqrt{E/\rho}$ é a velocidade de propagação de ondas. A Figura 4.7 ilustra os cinco primeiros modos.



Figura 4.7: Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.

4.4.6 Transferência de Calor 2D

O módulo de transferência de calor bidimensional resolve a equação de condução transiente em uma placa quadrada discretizada com elementos triangulares P1 (Seção 3.4.3).

As matrizes elementares de rigidez e massa são construídas pela função compartilhada `element_matrices`, que implementa diretamente as expressões derivadas na Seção 3.4.4 (Eqs. 3.23 e 3.24):

```

1 def element_matrices(coords, k, rho, cv):
2     x0, x1, x2 = coords
3     area = 0.5 * abs((x1[0]-x0[0])*(x2[1]-x0[1])
4                     -(x2[0]-x0[0])*(x1[1]-x0[1]))
5     b = np.array([x1[1]-x2[1], x2[1]-x0[1], x0[1]-x1[1]])
6     c = np.array([x2[0]-x1[0], x0[0]-x2[0], x1[0]-x0[0]])
7     ke = (k/(4*area)) * (np.outer(b,b) + np.outer(c,c))
8     me = (rho*cv*area/12) * (np.ones((3,3)) + np.eye(3))
9     return ke, me

```

A integração temporal emprega o método de Crank-Nicolson ($\theta = 1/2$), conforme a formulação do método θ generalizado apresentada na Seção 3.6 (Eq. 3.35), formulado com matrizes esparsas (`scipy.sparse`) para viabilizar a execução no navegador com malhas de até 40×40 nós. A cada passo de tempo, um frame do campo de temperatura é gerado e acumulado para compor uma animação GIF. A Figura 4.8 ilustra um instante da evolução temporal.

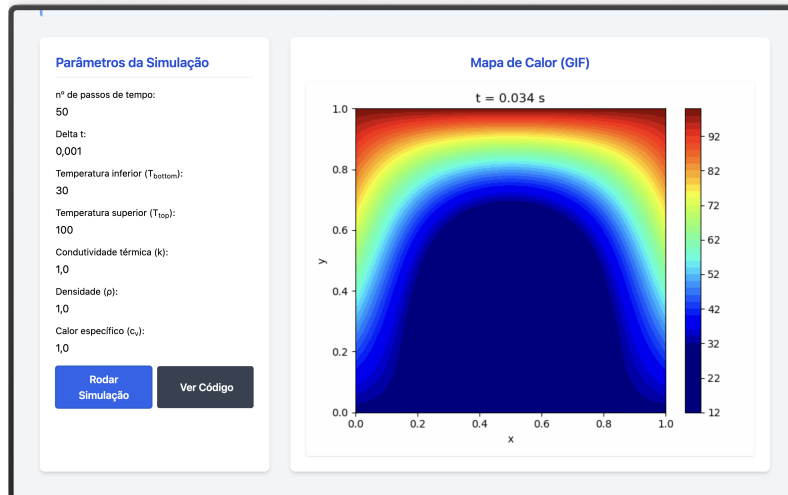


Figura 4.8: Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).

4.4.7 Escoamento Potencial 2D

O módulo de escoamento potencial resolve a equação de Laplace $\nabla^2\psi = 0$ em domínios 2D com obstáculos, caso particular da equação de Poisson para a função corrente (Eq. 3.30) com vorticidade nula. A malha triangular é gerada pelo módulo `common.py`, que também assembla as matrizes globais de rigidez e massa conforme o procedimento descrito na Seção 3.4.5.

O módulo `geometry.py` fornece diferentes geometrias de obstáculo (cilindro, retângulo, degrau), sendo os nós do contorno do obstáculo inseridos na malha com a condição $\psi = \text{constante}$. As velocidades são recuperadas por:

$$u = \frac{\partial\psi}{\partial y} \approx \frac{1}{2A^e} \sum_j \psi_j c_j, \quad v = -\frac{\partial\psi}{\partial x} \approx -\frac{1}{2A^e} \sum_j \psi_j b_j \quad (4.5)$$

A Figura 4.9 apresenta as linhas de corrente para escoamento ao redor de um obstáculo.

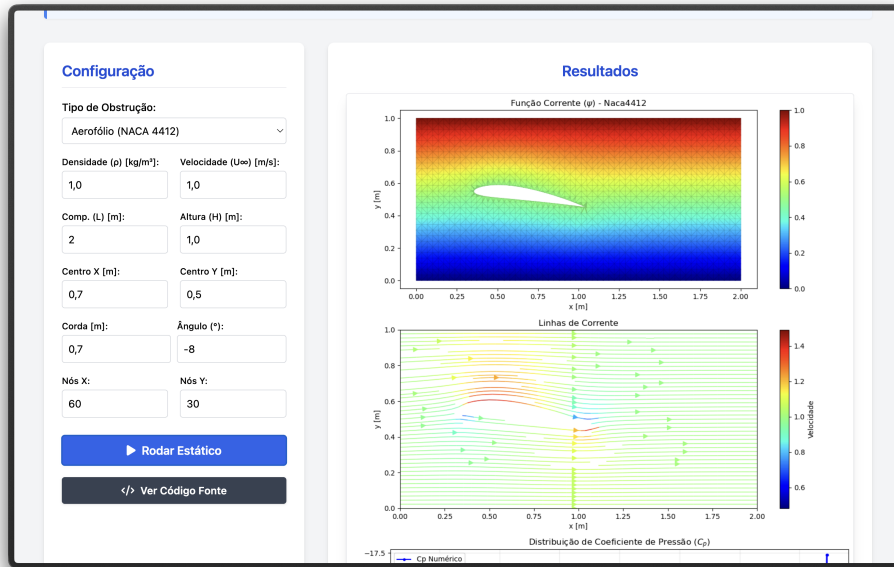


Figura 4.9: Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.

4.4.8 Navier-Stokes 2D (Vorticidade-Corrente)

O módulo mais complexo da plataforma implementa o solver de Navier-Stokes 2D na formulação Vorticidade-Corrente (ω - ψ), conforme descrito na Seção 3.5. A discretização temporal segue o esquema semi-implícito derivado na Seção 3.6.2

(Eq. 3.36), enquanto as condições de contorno de vorticidade nas paredes são impostas conforme a Eq. 3.37. O algoritmo segue o procedimento iterativo:

1. Resolver a equação de Poisson $\mathbf{K}\psi = \mathbf{M}\omega$ para ψ ;
2. Atualizar a vorticidade nas paredes sólidas conforme a Eq. 3.37;
3. Calcular (u, v) a partir de ψ (Eq. 4.5);
4. Assemblar a matriz de convecção $\mathbf{C}^n$ com o campo de velocidades atual;
5. Resolver o transporte da vorticidade pelo esquema semi-implícito (Eq. 3.36).

O solver suporta dois cenários: escoamento em canal com obstáculos (cilindro, retângulo, degrau) e cavidade com tampa móvel (*lid-driven cavity*). No caso da cavidade, todas as paredes possuem $\psi = 0$ e a parede superior impõe velocidade tangencial $u_{\text{lid}} = 1$, acrescentando o termo $-2u_{\text{tan}}/h$ à condição de contorno da vorticidade. A Figura 4.10 mostra os campos de ψ , ω e as linhas de corrente para a cavidade a $\text{Re} = 100$.

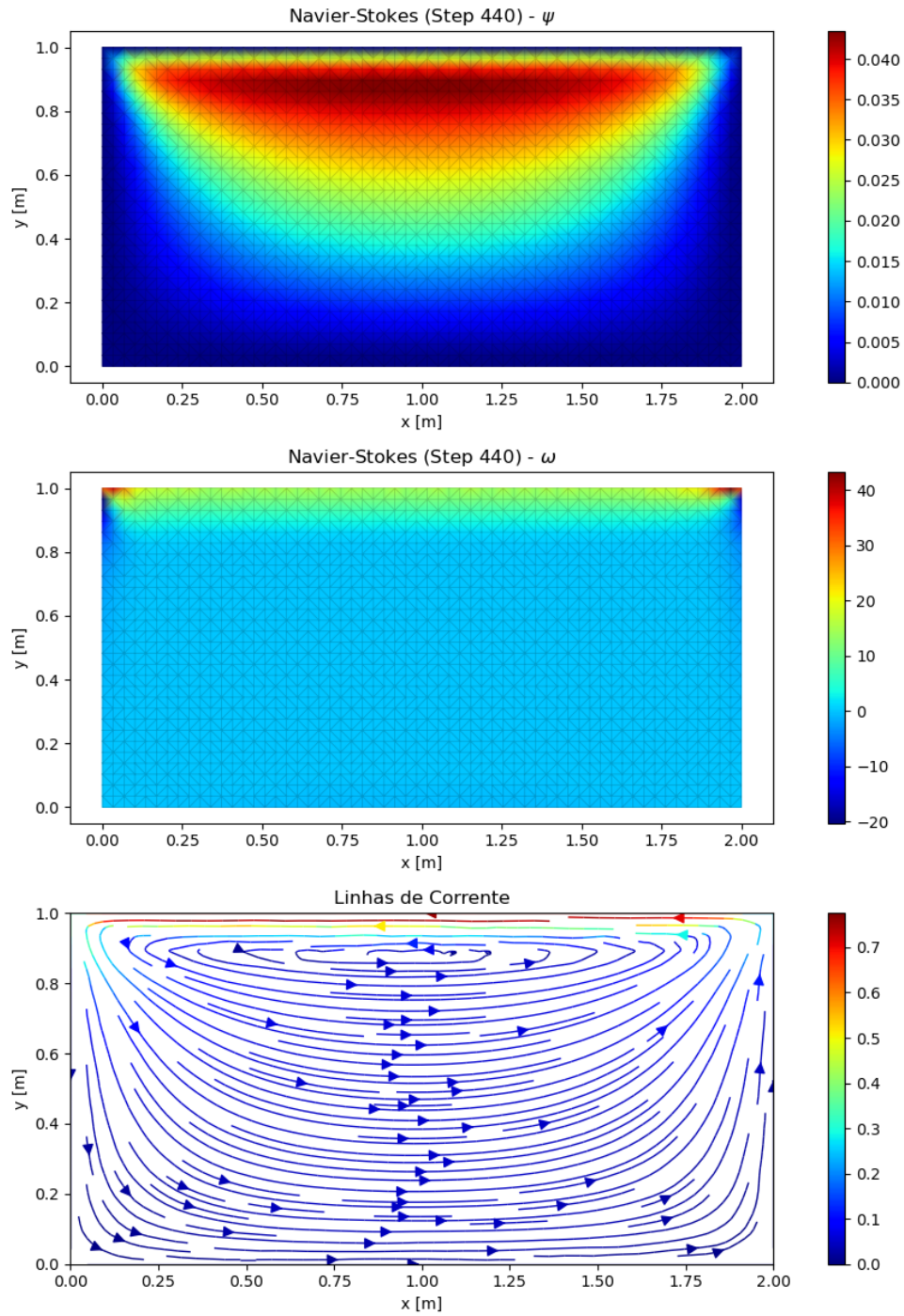


Figura 4.10: Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.

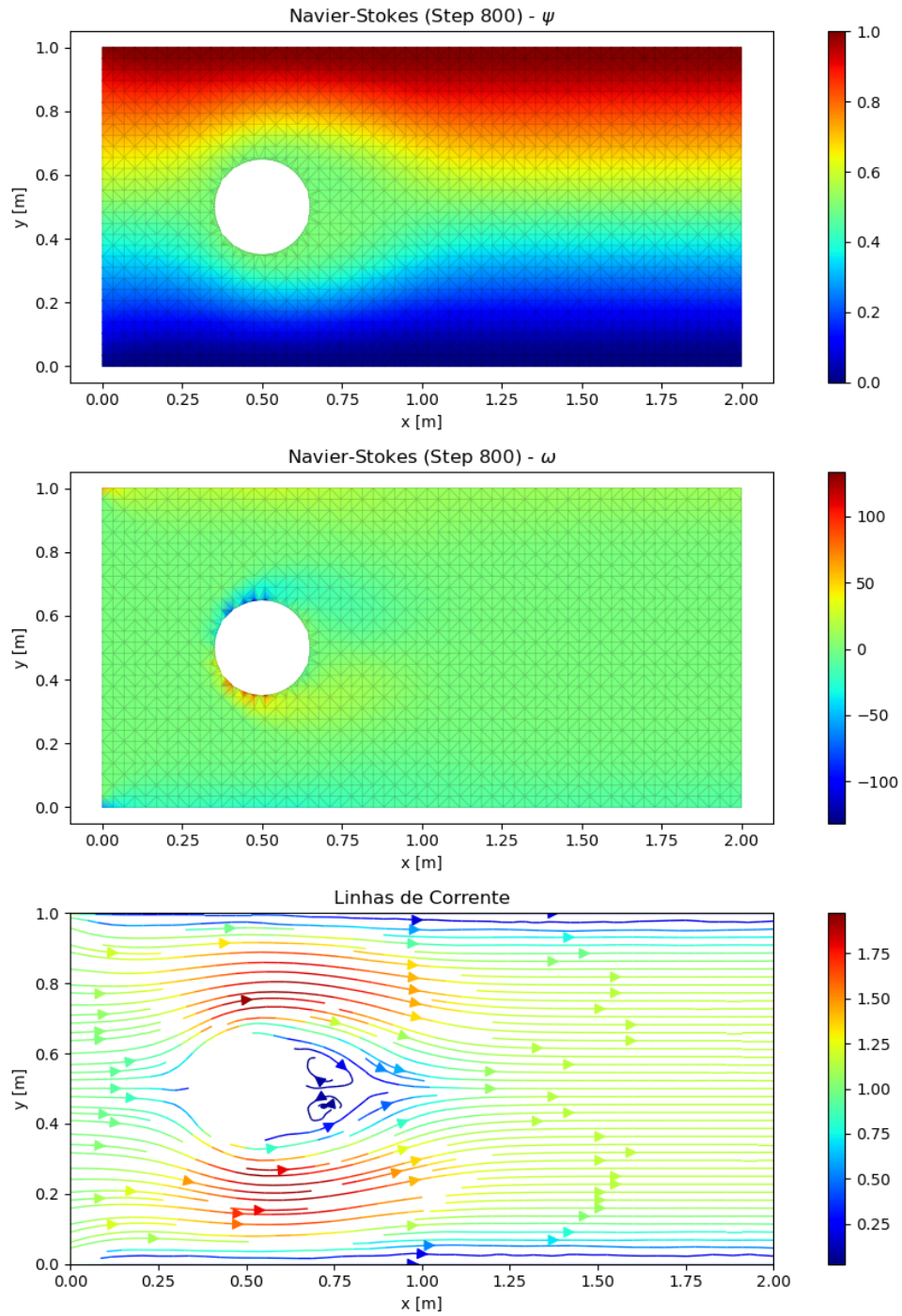


Figura 4.11: Navier-Stokes 2D — linhas de corrente para escoamento ao redor de um cilindro.

4.5 Síntese do Capítulo

Do ponto de vista da integração teórico-computacional, todos os módulos descritos neste capítulo compartilham o mesmo núcleo numérico: construção de ma-

trizes elementares, montagem global por conectividade nodal, solução de sistemas lineares e, nos casos transientes, avanço temporal via método θ ou esquema semi-implícito. Dessa forma, a plataforma *MinervaMesh* operacionaliza integralmente o arcabouço teórico estabelecido no Capítulo 3, mantendo correspondência direta entre as equações governantes contínuas, os sistemas discretizados obtidos pela formulação de Galerkin e os *solvers* efetivamente implementados. As listagens completas dos códigos, apresentadas nos Apêndices A e B, documentam essa correspondência em nível de implementação linha a linha.

Este capítulo apresentou a arquitetura e a implementação do *MinervaMesh*. A escolha do PyScript e da computação *client-side* viabiliza uma plataforma de simulação numérica com custo de infraestrutura zero, portabilidade total e código aberto para inspeção acadêmica. A organização modular do repositório — com cada simulação autocontida em três arquivos (HTML, Python, TOML) — permite a extensão da plataforma com novos casos sem impacto nos módulos existentes. Os oito módulos MEF descritos cobrem três domínios da Engenharia Mecânica (térmico, estrutural e fluidodinâmico), demonstrando a versatilidade da formulação de elementos finitos apresentada no Capítulo 3. O Capítulo 5 apresentará a validação quantitativa desses módulos através da comparação com soluções analíticas e benchmarks da literatura.

Capítulo 5

Resultados e Validação

5.1 Introdução

Este capítulo apresenta a validação numérica dos oito módulos de simulação por Elementos Finitos do *MinervaMesh*, descritos no Capítulo 4. Cada caso simulado é comparado com soluções analíticas conhecidas ou benchmarks consagrados da literatura, de modo a verificar a corretude da implementação e quantificar os erros numéricos associados.

A estratégia de validação segue um critério de complexidade crescente. Os primeiros casos — condução de calor em regime permanente e transiente — envolvem problemas unidimensionais com solução analítica fechada, permitindo o cálculo direto de métricas de erro como o erro máximo absoluto e o erro relativo na norma L_2 . Na sequência, os problemas de convecção-difusão testam a estabilização SUPG, comparando a solução estabilizada com a solução Galerkin padrão em regime de alto número de Péclet. Os módulos estruturais — flexão e vibração de barras — são validados contra soluções clássicas da Resistência dos Materiais e da Dinâmica Estrutural. Finalmente, os casos bidimensionais de transferência de calor, escoamento potencial e escoamento viscoso (Navier-Stokes) completam a validação com problemas de geometria e condições de contorno mais complexas.

Para cada caso, a metodologia de apresentação é a seguinte:

1. **Definição do problema:** equação governante, domínio, condições de contorno e parâmetros adotados.

2. **Referência de validação:** solução analítica ou benchmark utilizado para comparação.
3. **Resultados numéricos:** campos obtidos pelo simulador, apresentados graficamente.
4. **Análise de erro:** métricas quantitativas de comparação entre resultado numérico e referência.

Todos os resultados foram gerados diretamente no navegador, utilizando a plataforma *MinervaMesh* em execução local via PyScript/WebAssembly, conforme descrito no Capítulo 4. Os parâmetros de entrada de cada simulação são reproduzidos integralmente nas tabelas a seguir, garantindo a reprodutibilidade dos experimentos.

5.2 Condução Permanente 1D

O primeiro caso de validação consiste no problema de condução de calor em regime permanente, em uma barra unidimensional de comprimento L com geração interna de calor Q e condições de contorno de Dirichlet em ambas as extremidades. Este problema constitui o caso mais simples de validação do MEF, pois admite solução analítica exata e envolve elementos lineares unidimensionais (P1), correspondendo diretamente à formulação desenvolvida na Seção 3.4.1.

5.2.1 Formulação do Problema

O problema é governado pela equação de Poisson unidimensional, caso particular da equação geral de condução (Equação 3.1):

$$-k \frac{d^2 T}{dx^2} = Q, \quad x \in [0, L] \quad (5.1)$$

com as condições de contorno:

$$T(0) = T_0, \quad T(L) = T_L \quad (5.2)$$

A Tabela 5.1 apresenta os parâmetros utilizados na simulação.

Tabela 5.1: Parâmetros da simulação de condução permanente 1D.

Parâmetro	Símbolo	Valor
Comprimento da barra	L	1,0 m
Número de elementos	n_e	10
Temperatura em $x = 0$	T_0	0,0 °C
Temperatura em $x = L$	T_L	1,0 °C
Condutividade térmica	k	1,0 W/(m·°C)
Geração interna	Q	4,0 W/m ³

5.2.2 Solução Analítica

A integração direta da Equação (5.1) fornece a solução exata:

$$T(x) = -\frac{Q}{2k}x^2 + C_1x + C_2 \quad (5.3)$$

onde as constantes de integração são determinadas pelas condições de contorno:

$$C_2 = T_0, \quad C_1 = \frac{T_L - T_0 + \frac{QL^2}{2k}}{L} \quad (5.4)$$

Substituindo os valores da Tabela 5.1, obtém-se $C_1 = 3,0$ °C/m e $C_2 = 0,0$ °C. O perfil de temperatura resultante é uma parábola com valor máximo $T_{\max} \approx 1,125$ °C em $x^* = 0,75$ m, no interior do domínio — superior ao valor prescrito em qualquer das extremidades ($T_0 = 0$ °C, $T_L = 1,0$ °C) e decorrente da competição entre a geração interna de calor e a difusão para as fronteiras.

5.2.3 Solução MEF e Comparação

A discretização por elementos finitos utiliza $n_e = 10$ elementos lineares (P1) com espaçamento uniforme $h = L/n_e = 0,1$ m, resultando em 11 nós. A montagem do sistema global e a imposição das condições de contorno de Dirichlet seguem o procedimento descrito nas Seções 3.4.2 e 3.4.5. Para este problema de Poisson unidimensional com elementos P1, a solução MEF coincide exatamente com a solução analítica nos nós da malha (propriedade bem conhecida da interpolação linear por partes para polinômios de grau ≤ 2).

A Figura 5.1 apresenta os resultados obtidos. A curva MEF (pontos azuis) está perfeitamente sobreposta à solução analítica (linha tracejada vermelha), confirmando a corretude da implementação.

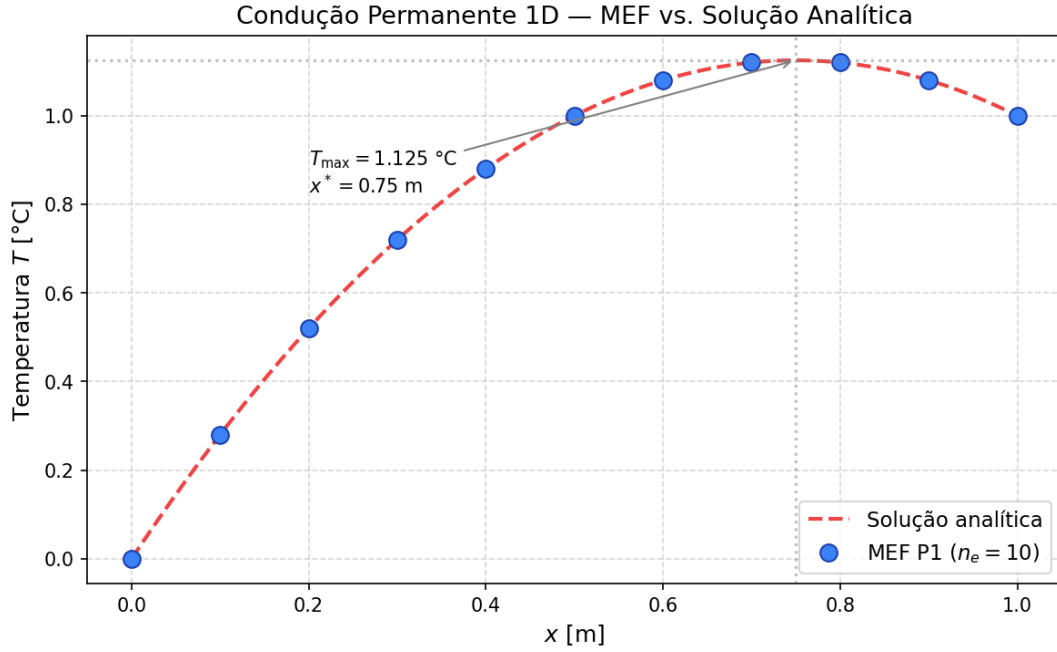


Figura 5.1: Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos P1) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes.

5.2.4 Análise de Erro

Para este caso particular — equação de Poisson 1D com fonte constante e elementos lineares — a solução analítica é um polinômio de grau 2. Como a interpolação P1 reproduz exatamente qualquer polinômio de grau ≤ 2 nos nós da malha, espera-se que o erro nodal seja da ordem do erro de arredondamento de máquina ($\sim 10^{-15}$). De fato, ao calcular o erro máximo absoluto e o erro relativo na norma L_2 :

$$e_{\max} = \max_i |T_i^{\text{MEF}} - T_i^{\text{ana}}|, \quad e_{L_2} = \frac{(\sum_i (T_i^{\text{MEF}} - T_i^{\text{ana}})^2)^{1/2}}{(\sum_i (T_i^{\text{ana}})^2)^{1/2}} \quad (5.5)$$

obtem-se $e_{\max} \approx 10^{-15} \text{ }^{\circ}\text{C}$ e $e_{L_2} \approx 10^{-15}$, confirmando erro essencialmente nulo nos nós da malha. Este resultado era esperado teoricamente e valida a montagem das matrizes elementares de rigidez e do vetor de carga, a montagem global e a imposição das condições de contorno de Dirichlet implementadas no módulo `conducao-permanente-barra1d-mef`.

Nota sobre a precisão nodal exata. A concordância perfeita nos nós não implica erro nulo entre os nós. O campo MEF é linear por partes, enquanto a solução exata é parabólica. O erro de interpolação entre nós é da ordem $O(h^2)$. Essa observação será relevante nos casos subseqüentes, que envolvem equações com soluções de maior grau ou não-polinomiais.

5.3 Condução Transiente 1D com $k(x)$ Variável

Figura 5.2: (Esqueleto) Evolução temporal do campo $T(x, t)$ e convergência ao regime permanente.

5.4 Convecção-Difusão 1D com Estabilização SUPG

Figura 5.3: (Esqueleto) Comparação Galerkin vs. SUPG em alto número de Péclet.

5.5 Viga 1D — Flexão de Euler-Bernoulli

Figura 5.4: (Esqueleto) Deflexão, momento fletor e esforço cortante para viga simplesmente apoiada com carga uniforme.

Figura 5.5: (Esqueleto) Cinco primeiros modos de vibração e respectivas frequências naturais.

Figura 5.6: (Esqueleto) Campo de temperatura em placa 2D durante a evolução transiente.

5.6 Vibração 1D — Problema de Autovalor

5.7 Transferência de Calor 2D

5.8 Escoamento Potencial 2D

Figura 5.7: (Esqueleto) Linhas de corrente ao redor do cilindro e distribuição de C_p na superfície.

5.9 Navier-Stokes 2D — Cavity com Tampa Móvel

Figura 5.8: (Esqueleto) Campos de ψ , ω e linhas de corrente na cavidade a $Re = 100$.

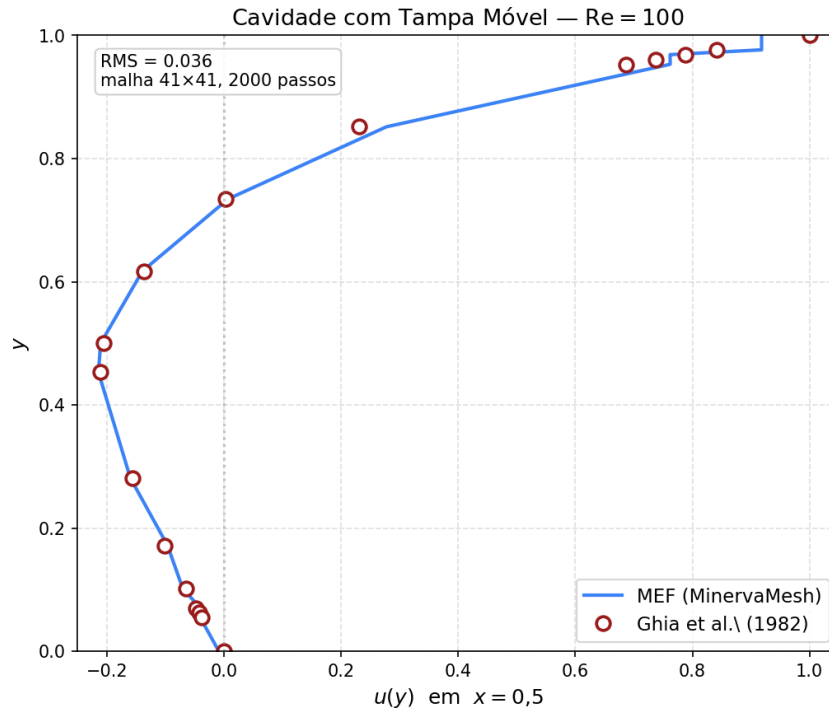


Figura 5.9: Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia, Ghia & Shin (1982) para a cavidade com tampa móvel a $Re = 100$. RMS = 0,036 em malha 41×41 com 2000 passos de tempo. Figura gerada pelo runner `.agents/review/thom_validation/run_cavity.py` com a fórmula de Thom na forma canônica.

Capítulo 6

Conclusões

6.1 Conclusões

Este trabalho apresentou o desenvolvimento e a validação do *MinervaMesh*, uma plataforma *client-side* para simulação numérica em Engenharia Mecânica. Os resultados obtidos demonstram a viabilidade do uso de tecnologias web modernas para a execução de métodos numéricos com rigor acadêmico.

6.2 Trabalhos Futuros

Como extensões deste trabalho, propõe-se a implementação de elementos de ordem superior, a inclusão de modelos de turbulência e a otimização do solver para problemas tridimensionais de maior escala.

Referências Bibliográficas

- [1] MALISKA, C. R., *Transferência de Calor e Mecânica dos Fluidos Computacional*. 2 ed. Rio de Janeiro, LTC, 2004.
- [2] VERSTEEG, H. K., MALALASEKERA, W., *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. 2 ed. Harlow, Pearson Education, 2007.
- [3] ANJOS, G. R., *Computação Científica para Engenheiros*. Rio de Janeiro, PEM/COPPE/UFRJ, 2024. Notas de Aula, versão de 23 de maio de 2024.
- [4] PyScript Development Team, “PyScript Documentation”, Disponível em: <https://pyscript.net>, 2023, Acesso em: 20 fev. 2026.
- [5] HARRIS, C. R., MILLMAN, K. J., WALT, S. J. V. D., *et al.*, “Array programming with NumPy”, *Nature*, v. 585, n. 7825, pp. 357–362, 2020.
- [6] BROOKS, A. N., HUGHES, T. J. R., “Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations”, *Computer Methods in Applied Mechanics and Engineering*, v. 32, n. 1-3, pp. 199–259, 1982.
- [7] FISH, J., BELYTSCHKO, T., *A First Course in Finite Elements*. West Sussex, John Wiley & Sons, 2007.
- [8] REDDY, J. N., *An Introduction to the Finite Element Method*. 3 ed. New York, McGraw-Hill, 2005.
- [9] INCROPERA, F. P., DEWITT, D. P., BERGMAN, T. L., *et al.*, *Fundamentals of Heat and Mass Transfer*. 6th ed. Hoboken, John Wiley & Sons, 2006.

- [10] OZISIK, M. N., *Heat Conduction*. 2 ed. New York, John Wiley & Sons, 1993.
- [11] CENGEL, Y. A., GHAJAR, A. J., *Heat and Mass Transfer: Fundamentals and Applications*. 5 ed. New York, McGraw-Hill Education, 2015.
- [12] BEJAN, A., *Heat Transfer*. New York, Wiley, 1992.
- [13] FOX, R. W., MCDONALD, A. T., PRITCHARD, P. J., *Introdução à Mecânica dos Fluidos*. 7th ed. Rio de Janeiro, LTC, 2010.
- [14] WHITE, F. M., *Viscous Fluid Flow*. 3 ed. New York, McGraw-Hill, 2006.
- [15] BIRD, R. B., STEWART, W. E., LIGHTFOOT, E. N., *Transport Phenomena*. 2 ed. New York, John Wiley & Sons, 2002.
- [16] PATANKAR, S. V., *Numerical Heat Transfer and Fluid Flow*. New York, Hemisphere Publishing Corporation, 1980.
- [17] LEWIS, R. W., NITHIARASU, P., SEETHARAMU, K. N., *Fundamentals of the Finite Element Method for Heat and Fluid Flow*. Chichester, John Wiley & Sons, 2004.
- [18] ZIENKIEWICZ, O. C., TAYLOR, R. L., ZHU, J. Z., *The Finite Element Method: Its Basis and Fundamentals*. 7th ed. Oxford, Butterworth-Heinemann, 2013.
- [19] BATHE, K.-J., *Finite Element Procedures*. Upper Saddle River, Prentice Hall, 1996.
- [20] LOGAN, D. L., *A First Course in the Finite Element Method*. 4 ed. Belmont, Thomson, 2007.
- [21] HUEBNER, K. H., DEWHIRST, D. L., SMITH, D. E., *et al.*, *The Finite Element Method for Engineers*. 4 ed. New York, John Wiley & Sons, 2001.
- [22] GALERKIN, B. G., “Series in some questions of elastic equilibrium of beams and plates”, *Vestnik Inzhenerov i Tekhnikov*, v. 19, pp. 897–908, 1915.
- [23] HRENNIKOFF, A., “Solution of problems of elasticity by the framework method”, *Journal of Applied Mechanics*, v. 8, n. 4, pp. A169–A175, 1941.

- [24] COURANT, R., “Variational methods for the solution of problems of equilibrium and vibrations”, *Bulletin of the American Mathematical Society*, v. 49, n. 1, pp. 1–23, 1943.
- [25] TURNER, M. J., CLOUGH, R. W., MARTIN, H. C., *et al.*, “Stiffness and deflection analysis of complex structures”, *Journal of the Aeronautical Sciences*, v. 23, n. 9, pp. 805–823, 1956.
- [26] CLOUGH, R. W., “The finite element method in plane stress analysis”. In: *Proceedings of the 2nd ASCE Conference on Electronic Computation*, Pittsburgh, 1960.
- [27] DONEA, J., “A Taylor-Galerkin method for convective transport problems”, *International Journal for Numerical Methods in Engineering*, v. 20, n. 1, pp. 101–119, 1984.
- [28] CHRISTIE, I., GRIFFITHS, D. F., MITCHELL, A. R., *et al.*, “Finite element methods for second order differential equations with significant first derivatives”, *International Journal for Numerical Methods in Engineering*, v. 10, pp. 1389–1396, 1976.
- [29] LÖHNER, R., MORGAN, K., ZIENKIEWICZ, O. C., “The solution of non-linear hyperbolic equation systems by the finite element method”, *International Journal for Numerical Methods in Fluids*, v. 4, pp. 1043–1063, 1984.
- [30] GHIA, U., GHIA, K. N., SHIN, C. T., “High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method”, *Journal of Computational Physics*, v. 48, pp. 387–411, 1982.
- [31] MARCHI, C. H., SUERO, R., ARAKI, L. K., “The lid-driven square cavity flow: numerical solution with a 1024 x 1024 grid”, *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, v. 31, n. 3, pp. 186–198, 2009.
- [32] THOMAS, C., MORGAN, K., TAYLOR, C., “A finite element analysis of flow over a backward facing step”, *Computers & Fluids*, v. 9, n. 3, pp. 265–278, 1981.

- [33] ARMALY, B. F., DURST, F., PEREIRA, J. C. F., *et al.*, “Experimental and theoretical investigation of backward-facing step flow”, *Journal of Fluid Mechanics*, v. 127, pp. 473–496, 1983.
- [34] PIRONNEAU, O., “On the transport-diffusion algorithm and its applications to the Navier-Stokes equations”, *Numerische Mathematik*, v. 38, pp. 309–332, 1982.
- [35] GEUZAIN, C., REMACLE, J.-F., “Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities”, *International Journal for Numerical Methods in Engineering*, v. 79, n. 11, pp. 1309–1331, 2009.
- [36] AGUIAR, G. D. S. O. N. D., *Simulação Numérica da Transferência de Calor em um Disco de Freio Durante Frenagem usando o Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [37] ALVES, F. R. D. M., *Solução da equação tridimensional transiente do calor através do método de elementos finitos aplicado a discos de freio*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [38] FERREIRA, G. P., *Criação de um código em Python para simular a transferência de calor em um processador computacional*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [39] PINHEIRO, V. G., *Análise de condução de calor em componentes eletrônicos de material heterogêneo utilizando método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [40] CAPITANIO, T. A., *Implementação do método de elementos finitos em Python para o estudo paramétrico de dissipadores de calor de processadores computacionais*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [41] MIRANDA, L. M., *Estudo de geometrias de canais em placa fria para arrefecimento de eletrônicos através do método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.

- [42] TEIXEIRA, M. M., *Solução da equação de Laplace tridimensional para a temperatura aplicada a um bloco de motor através do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [43] OLIVEIRA, M. D. D., *Estudo da transferência de calor em dissipador de calor de aletas retas por meio do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [44] ARAUJO, R. S. D. A., *Comparação e validação de métodos numéricos aplicados à Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [45] FLORENTINO, Y. D. S., *Estudo de escoamentos para arrefecimento de eletrônicos através de Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [46] SANTOS, J. D. O. D., *Investigação dos campos de velocidade, vorticidade e função corrente em escoamento livre utilizando o Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [47] SILVA, J. A., *Método de Elementos Finitos em Python com a abordagem lagrangiana-euleriana para as oscilações de um cilindro*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [48] AMARAL, T. A. C. D., *Análise CFD de difusão de espécie química para diferentes geometrias de stent farmacológico*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [49] GOMES, M. F., *Análise CFD de escoamento em artérias coronárias para diferentes configurações de restrição de fluxo*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [50] GOMES, Y. P., *Análise Variacional e Numérica da Equação de Poisson em Problemas de Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.

- [51] THOM, A., “The Flow Past Circular Cylinders at Low Speeds”, *Proceedings of the Royal Society A*, v. 141, n. 845, pp. 651–669, 1933.

Apêndice A

Listagens dos Módulos de Simulação 1D

Este apêndice apresenta as listagens completas dos módulos de simulação unidimensionais implementados no *MinervaMesh*. Os códigos são incluídos com o propósito de garantir a reprodutibilidade científica dos resultados apresentados neste trabalho: cada listagem corresponde diretamente às formulações matemáticas derivadas no Capítulo 3 e às implementações descritas no Capítulo 4. Cabe ressaltar que este apêndice não constitui um repositório de software, mas sim documentação técnica destinada à verificação e validação das soluções numéricas obtidas. Os códigos Python a seguir correspondem aos arquivos `simulation.py` de cada caso, executados integralmente no navegador via PyScript/WebAssembly.

A.1 Condução Permanente 1D

Listagem A.1: Condução permanente 1D — `simulation.py`

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Condução Permanente 1D (MEF) --- Problema 5.2.2
5 Barra 1D com geração interna Q, T(0)=T0 e T(L)=TL.
6 Elementos lineares, solução analítica para comparação.
7 """
8
```

```

9  import numpy as np
10 import matplotlib.pyplot as plt
11 from js import document
12 from pyscript import when
13 import asyncio
14 import io
15 import imageio.v2 as imageio
16 import base64
17
18
19 # =====
20 # 1. Solução analítica
21 # =====
22 def analytical_solution(x, Q, k, L, T0, TL):
23     # Solução da EDO:  $-k T'' = Q$ ,  $T(0)=T0$ ,  $T(L)=TL$ 
24     C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
25     C2 = T0
26     return - (Q / (2 * k)) * x * x + C1 * x + C2
27
28
29 # =====
30 # 2. Executar simulação MEF 1D
31 # =====
32 @when("click", "#runMEF1D")
33 async def run_mef_1d(event=None):
34     # Abrir modal de execução
35     sim_loading = document.getElementById("sim-loading")
36     try:
37         sim_loading.showModal()
38     except Exception:
39         pass
40     await asyncio.sleep(0.05)
41
42     # Parâmetros
43     L = float(document.getElementById("L").value)
44     nel = int(document.getElementById("nel").value)
45     T0 = float(document.getElementById("T0").value)
46     TL = float(document.getElementById("TL").value)
47     k = float(document.getElementById("k").value)

```



```

48     Q = float(document.getElementById("Q").value)
49
50     # Malha 1D
51     nn = nel + 1
52     x = np.linspace(0.0, L, nn)
53     h = L / nel
54
55     # Matrizes/vetores globais
56     K = np.zeros((nn, nn))          # matriz de rigidez (apostila:
        K)
57     b = np.zeros(nn)                # vetor de carregamento (
        apostila: b)
58
59     # Elemento linear: ke = k/h * [[1,-1],[-1,1]]; be = Q*h/2 *
        [1,1]
60     ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
61     be = (Q * h / 2.0) * np.array([1.0, 1.0])
62
63     # Montagem
64     for e in range(nel):
65         n1, n2 = e, e + 1
66         K[n1:n2+1, n1:n2+1] += ke
67         b[n1:n2+1] += be
68
69     # Contornos essenciais (Dirichlet)
70     # T(0)=T0
71     K[0, :] = 0.0
72     K[0, 0] = 1.0
73     b[0] = T0
74     # T(L)=TL
75     K[-1, :] = 0.0
76     K[-1, -1] = 1.0
77     b[-1] = TL
78
79     # Resolver
80     T = np.linalg.solve(K, b)
81
82     # Analítico (grade densa para curva lisa)
83     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))

```

```

84     T_ana_dense = analytical_solution(x_dense, Q, k, L, T0, TL)
85
86     # Plot
87     fig, ax = plt.subplots(figsize=(8, 4))
88     ax.plot(x, T, 'o-', label='MEF 1D', color='#3B82F6')
89     ax.plot(x_dense, T_ana_dense, '--', label='Analítica', color
90             = '#EF4444')
91     ax.set_xlabel('x [m]')
92     ax.set_ylabel('Temperatura [°C]')
93     ax.set_title('Condução Permanente 1D (MEF)')
94     ax.grid(True, linestyle='--', alpha=0.5)
95     ax.legend()
96     plt.tight_layout()
97
98     buf = io.BytesIO()
99     plt.savefig(buf, format='png')
100    plt.close(fig)
101    buf.seek(0)
102    img = imageio.imread(buf)
103    gif_buffer = io.BytesIO()
104    imageio.mimsave(gif_buffer, [img], format='GIF', duration
105                    =1.0)
106    gif_buffer.seek(0)
107    gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
108                        -8')
109
110    container = document.getElementById("mef1d-output")
111    container.innerHTML = f"<img src='data:image/gif;base64,{
112                        gif_base64}' class='rounded shadow w-full' />"
113
114    try:
115        sim_loading.close()
116    except Exception:
117        pass

```

A.2 Condução Transiente 1D com $k(x)$ Variável

Listagem A.2: Condução transiente 1D — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Problema Térmico Transiente 1D (MEF)
5  -----
6
7  Modelo:  $\rho c \frac{\partial T}{\partial t} = \frac{\partial}{\partial x}(k(x) \frac{\partial T}{\partial x})$ ,  $0 < x < L$ ,  $t > 0$ 
8  Condições de contorno:  $T(0,t) = 0$ ,  $T(L,t) = 1$  (fixas)
9  Condutividade térmica descontínua em  $x=L/2$ , com valores  $k_1$  (
    primeira metade)
10 e  $k_2$  (segunda metade) configuráveis via interface.
11
12 Este script simula a condução transiente de calor 1D usando Mé
    todo dos Elementos Finitos
13 com condutividade térmica variável e número de elementos
    configurável.
14 Gera um GIF com os frames e embute no DOM via PyScript.
15 """
16
17 import numpy as np
18 import matplotlib.pyplot as plt
19 from js import document
20 from pyscript import when
21 import asyncio
22 import imageio.v2 as imageio
23 import io
24 import base64
25
26 # =====
27 # 1. Função de condutividade térmica k(x)
28 # =====
29 def thermal_conductivity(x, L, k1=5.0, k2=1.0):
30     """
31     Retorna a condutividade térmica  $k(x)$ .
32     Condutividade abrupta (não suavizada).
33     """
34     return np.where(x < L/2, k1, k2)

```

```

35
36
37 # =====
38 # 2. Solução analítica de regime permanente para k(x) descontí-
    nuo
39 # =====
40 def analytical_steady_state(x, L, k1, k2):
41     """
42     Solução analítica de regime permanente para k(x) descontínuo
        em x=L/2
43     com T(0)=0 e T(L)=1.
44
45     Em permanente, o fluxo q = -k·dT/dx é constante. Integrando
        em cada região
46     e impondo continuidade de T em L/2 e T(L)=1:
47         C      = 2·k1·k2 / (L·(k1+k2))
48         T(x)    = (C/k1)·x                                para x < L
        /2
49         T(x)    = (C·L/(2·k1)) + (C/k2)·(x - L/2)        para x >=
        L/2
50     """
51     C = 2.0 * k1 * k2 / (L * (k1 + k2))
52     T = np.zeros_like(x, dtype=float)
53     mask_left = x < L / 2
54     T[mask_left] = (C / k1) * x[mask_left]
55     T_half = (C / k1) * (L / 2)
56     T[~mask_left] = T_half + (C / k2) * (x[~mask_left] - L / 2)
57     return T
58
59
60 # =====
61 # 2. Matrizes de elemento para MEF 1D
62 # =====
63 def element_matrices_1d(x1, x2, k_avg, rho_cp):
64     """Calcula as matrizes de elemento para condução 1D."""
65     h = x2 - x1 # comprimento do elemento
66
67     # Matriz de rigidez do elemento
68     ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])

```

```

69
70     # Matriz de massa do elemento (capacidade térmica)
71     me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])
72
73     return ke, me
74
75
76     # =====
77     # 3. Montar sistema global
78     # =====
79     def assemble_system(n_elements, L, rho_cp=1.0, k1=5.0, k2=1.0):
80         """Monta as matrizes globais K e M do sistema MEF."""
81         n_nodes = n_elements + 1
82
83         x_nodes = np.zeros(n_nodes)
84
85         # Primeira metade (x < L/2) - pontos uniformes
86         n_left = n_elements // 2
87         x_nodes[:n_left+1] = np.linspace(0, L/2, n_left+1)
88
89         # Segunda metade (x >= L/2) - pontos uniformes
90         n_right = n_elements - n_left
91         x_nodes[n_left:] = np.linspace(L/2, L, n_right+1)
92
93         # Inicializar matrizes globais
94         K = np.zeros((n_nodes, n_nodes))
95         M = np.zeros((n_nodes, n_nodes))
96
97         # Montar sistema elemento por elemento
98         for i in range(n_elements):
99             x1, x2 = x_nodes[i], x_nodes[i+1]
100
101             # Calcular condutividade média do elemento
102             # Para elementos que cruzam a fronteira x=L/2, usar mé
103             dia ponderada
104             if x1 < L/2 and x2 > L/2:
105                 # Elemento cruza a fronteira - calcular média
106                 ponderada

```

```

105         frac_left = (L/2 - x1) / (x2 - x1)  # fração à
            esquerda da fronteira
106         frac_right = (x2 - L/2) / (x2 - x1)  # fração à
            direita da fronteira
107         k_avg = frac_left * k1 + frac_right * k2
108     else:
109         # Elemento inteiramente em uma região
110         x_center = (x1 + x2) / 2
111         k_avg = thermal_conductivity(x_center, L, k1, k2)
112
113     ke, me = element_matrizes_1d(x1, x2, k_avg, rho_cp)
114
115     # Montar matrizes globais
116     K[i:i+2, i:i+2] += ke
117     M[i:i+2, i:i+2] += me
118
119     return K, M, x_nodes
120
121
122 # =====
123 # 4. Renderizar frame
124 # =====
125 def render_frame(T, x_nodes, t, n_elements, k1=1.0, k2=5.0):
126     """Desenha um frame e retorna a imagem como array para o GIF
127     ."""
128
129     fig, ax = plt.subplots(figsize=(10, 6))
130
131     # Plotar temperatura numérica
132     ax.plot(x_nodes, T, 'o-', label="MEF", color="#3B82F6",
133            linewidth=2, markersize=4)
134
135     # Plotar solução analítica de regime permanente (referência)
136     x_dense = np.linspace(0, x_nodes[-1], 200)
137     T_analytical = analytical_steady_state(x_dense, x_nodes[-1],
138            k1, k2)
139     ax.plot(x_dense, T_analytical, '--', label="Regime
140            permanente (analítico)", color="#EF4444", linewidth=2)
141
142     # Plotar condutividade térmica como fundo

```

```

138     k_values = thermal_conductivity(x_dense, x_nodes[-1], k1, k2
139                                     )
140
141     # Criar segundo eixo para k(x)
142     ax2 = ax.twinx()
143     ax2.plot(x_dense, k_values, ':', color='green', alpha=0.7,
144             linewidth=1)
145     ax2.set_ylabel('Condutividade k(x)', color='green', fontsize
146                   =12)
147     ax2.tick_params(axis='y', labelcolor='green')
148     ax2.set_ylim(0, max(k1, k2) + 1)
149
150     ax.set_ylim(-0.1, 1.1)
151     ax.set_xlim(0, x_nodes[-1])
152     ax.set_xlabel("Posição x", fontsize=12)
153     ax.set_ylabel("Temperatura T", fontsize=12)
154     ax.set_title(f"Condução Transiente 1D - MEF (n={n_elements})
155                 | t = {t:.3f}s", fontsize=14)
156     ax.grid(True, linestyle='--', alpha=0.5)
157     ax.legend(fontsize=12)
158
159     # Adicionar linha vertical no meio para mostrar mudança de k
160     # Linha vertical no meio
161     ax.axvline(x=x_nodes[-1]/2, color='red', linestyle=':',
162              alpha=0.5)
163
164     # Rótulos em y = k1 e y = k2 no eixo de k(x)
165     eps = 0.02 * x_nodes[-1] # pequeno deslocamento horizontal
166     para não colidir com a linha
167     ax2.text(x_nodes[-1]/2 - eps, k1, f'k={k1:g}', ha='right',
168            va='center',
169            color='red', fontsize=10)
170     ax2.text(x_nodes[-1]/2 + eps, k2, f'k={k2:g}', ha='left', va
171            ='center',
172            color='red', fontsize=10)
173
174     plt.tight_layout()
175     buf = io.BytesIO()
176     plt.savefig(buf, format='png', dpi=100)

```

```

169     plt.close(fig)
170     buf.seek(0)
171     return imageio.imread(buf)
172
173
174     # =====
175     # 5. Executar simulação
176     # =====
177     @when("click", "#runHeatBtn")
178     async def run_simulation(event=None):
179         """Executa a simulação MEF, monta frames e publica um GIF no
180             contêiner HTML."""
181         # parâmetros
182         L = float(document.getElementById("L").value)
183         n_elements = int(document.getElementById("n_elements").value
184             )
185         k1 = float(document.getElementById("k1").value)
186         k2 = float(document.getElementById("k2").value)
187         rho_cp = float(document.getElementById("rho_cp").value)
188         dt = float(document.getElementById("dt").value)
189         theta = float(document.getElementById("theta").value)
190         tmax = float(document.getElementById("tmax").value)
191
192         # Montar sistema MEF
193         K, M, x_nodes = assemble_system(n_elements, L, rho_cp, k1,
194             k2)
195         n_nodes = len(x_nodes)
196
197         # Condições iniciais (temperatura zero em todo domínio,
198             exceto bordas)
199         T = np.zeros(n_nodes)
200
201         # Aplicar condições de contorno
202         T[0] = 0.0      #  $T(0, t) = 0$ 
203         T[-1] = 1.0     #  $T(L, t) = 1$ 
204
205         # Identificar nós de contorno
206         boundary_nodes = [0, n_nodes-1]
207         interior_nodes = list(range(1, n_nodes-1))

```



```

204
205     # Preparar sistema para integração temporal conforme equação
        da apostila
206     #  $((M/\Delta t) + \theta K) T^{(n+1)} = [(M/\Delta t) - (1-\theta)K] T^n$ 
207     A = M/dt + theta * K
208     B = M/dt - (1 - theta) * K
209
210     # Aplicar condições de contorno no sistema
211     for i in boundary_nodes:
212         A[i, :] = 0
213         A[i, i] = 1
214         B[i, :] = 0
215         B[i, i] = 1
216
217     # abrir modal de execução durante a simulação
218     container = document.getElementById("heat-output")
219     sim_loading = document.getElementById("sim-loading")
220     if sim_loading is not None:
221         try:
222             sim_loading.showModal()
223         except Exception:
224             pass
225     await asyncio.sleep(0.05)
226
227     frames = []
228     nt = max(1, int(tmax / dt))
229
230     # Frame inicial
231     frames.append(render_frame(T, x_nodes, 0.0, n_elements, k1,
        k2))
232
233     # =====
234     # 6. Loop temporal (Crank-Nicolson com suavização suave)
235     # =====
236     T_prev = T.copy() # Para suavização temporal
237
238     for n in range(1, nt + 1):
239         # Resolver sistema:  $A * T^{(n+1)} = B * T^n + f$ 
240         b = B @ T

```

```

241
242     # Aplicar condições de contorno no vetor b
243     b[0] = 0.0      #  $T(0,t) = 0$ 
244     b[-1] = 1.0     #  $T(L,t) = 1$ 
245
246     # Resolver sistema linear
247     T_new = np.linalg.solve(A, b)
248
249     # Atualizar temperatura
250     T = T_new
251
252     # Garantir condições de contorno exatas
253     T[0] = 0.0
254     T[-1] = 1.0
255
256     if n % max(1, nt // 50) == 0 or n == nt:
257         frames.append(render_frame(T, x_nodes, n*dt,
258                                   n_elements, k1, k2))
259
260     # =====
261     # 7. Montar GIF e publicar no DOM
262     # =====
263     gif_buffer = io.BytesIO()
264     imageio.mimsave(gif_buffer, frames, format='GIF', duration
265                     =0.1, loop=0)
266     gif_buffer.seek(0)
267     gif_base64 = base64.b64encode(gif_buffer.read()).decode("utf
268                               -8")
269     container.innerHTML = f"<img src='data:image/gif;base64,{
270                               gif_base64}' class='rounded shadow w-full' />"
271
272     if sim_loading is not None:
273         try:
274             sim_loading.close()
275         except Exception:
276             pass

```

A.3 Convecção-Difusão 1D com SUPG

Listagem A.3: Convecção-difusão 1D com SUPG — simulation.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Convecção-Difusão 1D Permanente (MEF) --- Problema 5.2.4
5  Equação:  $-k \frac{d^2 T}{dx^2} + \rho_{cp} u \frac{dT}{dx} = Q$ 
6  Condições:  $T(0)=T_0$ ,  $T(L)=T_L$ 
7  Elementos lineares com estabilização SUPG.
8  """
9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from js import document
13 from pyscript import when
14 import asyncio
15 import io
16 import imageio.v2 as imageio
17 import base64
18
19
20 # =====
21 # 1. Solução analítica
22 # =====
23 def analytical_solution(x, Q, k, rho_cp, u, L, T0, TL):
24     """
25     Solução analítica da equação de convecção-difusão 1D
26     permanente.
27     Equação:  $-k \frac{d^2 T}{dx^2} + \rho_{cp} u \frac{dT}{dx} = Q$ 
28     Condições:  $T(0)=T_0$ ,  $T(L)=T_L$ 
29
30     Forma estável numericamente mesmo para grandes números de Pé
31     clet: a razão
32      $(\exp(\alpha x) - 1) / (\exp(\alpha L) - 1)$  é reescrita fatorando  $\exp(\alpha$ 
33      $L)$  quando  $\alpha > 0$ ,
34     evitando overflow de exp para  $\alpha \cdot L$  grande.
35     """
```

```

33     # Caso puramente difusivo:  $-k T'' = Q$ 
34     if abs(rho_cp * u) < 1e-14:
35         C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
36         return -(Q / (2 * k)) * x * x + C1 * x + T0
37
38     alpha = rho_cp * u / k
39     Tp    = Q * x / (rho_cp * u)
40     Tp_L  = Q * L / (rho_cp * u)
41
42     #  $g(x) := (\exp(\alpha x) - 1) / (\exp(\alpha L) - 1)$ 
43     if alpha > 0:
44         # fatora  $\exp(\alpha L)$  para evitar overflow
45         g = np.exp(alpha * (x - L)) * (1.0 - np.exp(-alpha * x))
46             \
47             / (1.0 - np.exp(-alpha * L))
48     else:
49         # para  $\alpha \leq 0$ ,  $\expm1$  já é estável
50         g = np.expm1(alpha * x) / np.expm1(alpha * L)
51
52     return T0 + Tp + (TL - T0 - Tp_L) * g
53
54     # =====
55     # 2. Executar simulação MEF 1D com convecção-difusão
56     # =====
57     @when("click", "#runConveccaoDifusao1D")
58     async def run_conveccao_difusao_1d(event=None):
59         # Abrir modal de execução
60         sim_loading = document.getElementById("sim-loading")
61         try:
62             sim_loading.showModal()
63         except Exception:
64             pass
65         await asyncio.sleep(0.05)
66
67         # Parâmetros
68         L = float(document.getElementById("L").value)
69         nel = int(document.getElementById("nel").value)
70         T0 = float(document.getElementById("T0").value)

```

```

71 TL = float(document.getElementById("TL").value)
72 k = float(document.getElementById("k").value)
73 rho_cp = float(document.getElementById("rho_cp").value)
74 u = float(document.getElementById("u").value)
75 Q = float(document.getElementById("Q").value)
76 use_supg = document.getElementById("use_supg").checked
77
78 # Malha 1D uniforme
79 nn = nel + 1
80 x = np.linspace(0.0, L, nn)
81 h = L / nel
82
83 # Número de Péclet local
84 Pe_local = (rho_cp * u * h) / (2 * k)
85
86 # Parâmetro de estabilização SUPG
87 if use_supg and abs(Pe_local) > 1e-10:
88     tau = h / (2 * abs(u)) * (1 / np.tanh(abs(Pe_local)) - 1
89         / abs(Pe_local))
89 else:
90     tau = 0.0
91
92 # Matrizes/vetores globais
93 K = np.zeros((nn, nn)) # matriz de rigidez
94 b = np.zeros(nn) # vetor de carregamento
95
96 # Elemento linear com convecção-difusão
97 # Matriz de difusão: ke_diff = k/h * [[1,-1],[-1,1]]
98 ke_diff = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
99
100 # Matriz de convecção: ke_conv = rho_cp*u/2 * [[-1,1],[-1,1]]
101 ke_conv = (rho_cp * u / 2.0) * np.array([[-1.0, 1.0], [-1.0,
102     1.0]])
102
103 # Vetor de fontes: be = Q*h/2 * [1,1]
104 be = (Q * h / 2.0) * np.array([1.0, 1.0])
105
106 # Estabilização SUPG (se habilitada)
107 if use_supg and tau > 0:

```

```

108         # Matriz de estabilização SUPG
109         ke_supg = tau * rho_cp * u * u / h * np.array([[1.0,
110             -1.0], [-1.0, 1.0]])
111         # Vetor de estabilização SUPG
112         be_supg = tau * Q * u * np.array([-1.0, 1.0])
113
114         # Combinar todas as contribuições
115         ke_total = ke_diff + ke_conv + ke_supg
116         be_total = be + be_supg
117     else:
118         ke_total = ke_diff + ke_conv
119         be_total = be
120
121     # Montagem
122     for e in range(nel):
123         n1, n2 = e, e + 1
124         K[n1:n2+1, n1:n2+1] += ke_total
125         b[n1:n2+1] += be_total
126
127     # Contornos essenciais (Dirichlet)
128     #  $T(0)=T_0$ 
129     K[0, :] = 0.0
130     K[0, 0] = 1.0
131     b[0] = T0
132     #  $T(L)=T_L$ 
133     K[-1, :] = 0.0
134     K[-1, -1] = 1.0
135     b[-1] = TL
136
137     # Resolver
138     T = np.linalg.solve(K, b)
139
140     # Analítico (grade densa para curva lisa)
141     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))
142     T_ana_dense = analytical_solution(x_dense, Q, k, rho_cp, u,
143         L, T0, TL)
144
145     # Solução analítica nos pontos da malha numérica para cá
146     lculo do erro

```

```

144     T_ana_mesh = analytical_solution(x, Q, k, rho_cp, u, L, T0,
145                                     TL)
146
147     # Plot
148
149     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
150
151     # Gráfico principal
152     ax1.plot(x, T, 'o-', label='MEF 1D', color='#3B82F6',
153              markersize=6)
154     ax1.plot(x_dense, T_ana_dense, '--', label='Analítica',
155              color='#EF4444', linewidth=2)
156     ax1.set_xlabel('x [m]')
157     ax1.set_ylabel('Temperatura [°C]')
158     ax1.set_title('Convecção-Difusão 1D Permanente (MEF)')
159     ax1.grid(True, linestyle='--', alpha=0.5)
160     ax1.legend()
161
162     # Gráfico de erro
163     erro = np.abs(T - T_ana_mesh)
164     # Evitar erro zero para escala logarítmica
165     erro = np.maximum(erro, 1e-18)
166     ax2.semilogy(x, erro, 'o-', color='#10B981', markersize=6)
167     ax2.set_xlabel('x [m]')
168     ax2.set_ylabel('Erro Absoluto [°C]')
169     ax2.set_title('Erro entre Solução Numérica e Analítica (
170                    Escala Logarítmica)')
171     ax2.grid(True, linestyle='--', alpha=0.5)
172     # Adicionar texto explicativo
173     ax2.text(0.5, 0.5, f'Erro máximo: {np.max(erro):.2e} °C\n(
174                    Precisão de máquina)',
175              transform=ax2.transAxes, ha='center', va='center',
176              bbox=dict(boxstyle='round', facecolor='white',
177                        alpha=0.8),
178              fontsize=10)
179
180     plt.tight_layout()
181
182     buf = io.BytesIO()
183     plt.savefig(buf, format='png', dpi=150)

```

```

177     plt.close(fig)
178     buf.seek(0)
179     img = imageio.imread(buf)
180     gif_buffer = io.BytesIO()
181     imageio.mimsave(gif_buffer, [img], format='GIF', duration
182                     =1.0)
183     gif_buffer.seek(0)
184     gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
185                             -8')
186
187     container = document.getElementById("conveccao-difusao-
188                             output")
189     container.innerHTML = f"<img src='data:image/gif;base64,{
190                             gif_base64}' class='rounded shadow w-full' />"
191
192     # Atualizar informações sobre Péclet
193     pe_info = document.getElementById("pe-info")
194     pe_info.innerHTML = f"""
195     <div class="bg-blue-50 border-l-4 border-blue-400 p-3
196         rounded">
197         <h4 class="font-semibold text-blue-800 mb-1">Informações
198             da Simulação</h4>
199         <p class="text-sm text-gray-700"><strong>Número de Pé
200             clet:</strong> {Pe_local:.3f}</p>
201         <p class="text-sm text-gray-700"><strong>Estabilização
202             SUPG:</strong> {'Sim' if use_supg else 'Não'}</p>
203         <p class="text-sm text-gray-700"><strong>Parâmetro  $\tau$ :</
204             strong> {tau:.6f}</p>
205         <p class="text-sm text-gray-700"><strong>Velocidade:</
206             strong> {u:.3f} m/s</p>
207         <p class="text-sm text-gray-700"><strong>Tamanho
208             elemento:</strong> {h:.4f} m</p>
209         <p class="text-sm text-gray-700"><strong>Erro máximo:</
210             strong> {np.max(erro):.2e} °C</p>
211     </div>
212     """
213
214     try:
215         sim_loading.close()

```



```
204     except Exception:
205         pass
```

A.4 Viga 1D — Euler-Bernoulli

Listagem A.4: Viga 1D (Euler-Bernoulli) — `simulation.py`

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Viga 1D Avançada (MEF) --- Problema 5.2.5 Expandido
5  Análise completa de flexão de vigas usando MEF com elementos de
6  viga de Euler-Bernoulli.
7  Inclui múltiplos tipos de carregamento, condições de contorno,
8  propriedades variáveis e análises completas.
9  """
10
11 8
12
13 9
14 import numpy as np
15 import matplotlib.pyplot as plt
16 from js import document
17 from pyscript import when
18 import asyncio
19 import io
20 import imageio.v2 as imageio
21 import base64
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```

30 def load_triangular(x, q_max, L):
31     """Carregamento triangular (máximo no centro)"""
32     return q_max * (1 - 2 * np.abs(x - L/2) / L)
33
34 def load_sinusoidal(x, q0, L):
35     """Carregamento senoidal"""
36     return q0 * np.sin(np.pi * x / L)
37
38 def load_point(x, P, x_pos, L):
39     """Carregamento pontual (aproximado como distribuição
40         localizada)"""
41     # Aproximação: distribuição gaussiana muito concentrada
42     sigma = L / 100 # Largura da distribuição
43     return P * np.exp(-0.5 * ((x - x_pos) / sigma)**2) / (sigma
44         * np.sqrt(2 * np.pi))
45
46 # =====
47 # 2. Soluções analíticas
48 # =====
49 def analytical_solution_simply_supported(x, q_func, E, I, L):
50     """Solução analítica para viga simplesmente apoiada"""
51     if q_func == "uniform":
52         q = 1.0 # Valor normalizado
53         return (q / (24 * E * I)) * (L**3 * x - 2 * L * x**3 + x
54             **4)
55     elif q_func == "triangular":
56         q_max = 1.0
57         return (q_max / (120 * E * I)) * (L**4 * x - 2 * L**2 *
58             x**3 + x**5)
59     return np.zeros_like(x)
60
61 def analytical_solution_cantilever(x, q_func, E, I, L):
62     """Solução analítica para viga em balanço"""
63     if q_func == "uniform":
64         q = 1.0
65         return (q / (24 * E * I)) * (6 * L**2 * x**2 - 4 * L * x
66             **3 + x**4)
67     elif q_func == "triangular":

```

```

64         q_max = 1.0
65         return (q_max / (120 * E * I)) * (10 * L**3 * x**2 - 10
        * L**2 * x**3 + 5 * L * x**4 - x**5)
66     return np.zeros_like(x)
67
68 def analytical_solution_fixed_fixed(x, q_func, E, I, L):
69     """Solução analítica para viga engastada-engastada"""
70     if q_func == "uniform":
71         q = 1.0
72         return (q / (24 * E * I)) * (L**2 * x**2 - 2 * L * x**3
        + x**4)
73     return np.zeros_like(x)
74
75
76 # =====
77 # 3. Funções auxiliares para propriedades variáveis
78 # =====
79 def get_variable_properties(x, prop_type, base_value, L):
80     """Calcula propriedades variáveis ao longo da viga"""
81     if prop_type == "constant":
82         return np.full_like(x, base_value)
83     elif prop_type == "linear":
84         # Propriedade varia linearmente de base_value a 0.5*
85         base_value
86         return base_value * (1 - 0.5 * x / L)
87     elif prop_type == "parabolic":
88         # Propriedade varia parabolicamente (máximo no centro)
89         return base_value * (1 - 0.3 * (x - L/2)**2 / (L/2)**2)
90     return np.full_like(x, base_value)
91
92 # =====
93 # 4. Executar simulação MEF avançada para viga 1D
94 # =====
95 @when("click", "#runViga1D")
96 async def run_viga_1d(event=None):
97     # Abrir modal de execução
98     sim_loading = document.getElementById("sim-loading")
99     try:

```

```

100         sim_loading.showModal()
101     except Exception:
102         pass
103     await asyncio.sleep(0.05)
104
105     # Parâmetros básicos
106     L = float(document.getElementById("L").value)
107     nel = int(document.getElementById("nel").value)
108     E_base = float(document.getElementById("E").value) * 1e9 #
109         Converter para Pa
110     I_base = float(document.getElementById("I").value) * 1e-8 #
111         Converter para m4
112     q_magnitude = float(document.getElementById("q").value) *
113         1000 # Converter para N/m
114     bc_type = document.getElementById("bc_type").value
115     load_type = document.getElementById("load_type").value
116     prop_type = document.getElementById("prop_type").value
117     show_analytical = document.getElementById("show_analytical")
118         .checked
119
120     # Malha 1D
121     nn = nel + 1
122     x = np.linspace(0.0, L, nn)
123     h = L / nel
124
125     # Propriedades variáveis
126     E = get_variable_properties(x, prop_type, E_base, L)
127     I = get_variable_properties(x, prop_type, I_base, L)
128
129     # Carregamento
130     if load_type == "uniform":
131         q = load_uniform(x, q_magnitude, L)
132     elif load_type == "linear":
133         q0 = q_magnitude
134         qL = 0.5 * q_magnitude
135         q = load_linear(x, q0, qL, L)
136     elif load_type == "triangular":
137         q = load_triangular(x, q_magnitude, L)
138     elif load_type == "sinusoidal":

```

```

135     q = load_sinusoidal(x, q_magnitude, L)
136 elif load_type == "point":
137     P = q_magnitude * L # Força pontual equivalente
138     x_pos = L / 2 # Posição da força
139     q = load_point(x, P, x_pos, L)
140 else:
141     q = load_uniform(x, q_magnitude, L)
142
143 # Para elementos de viga, cada nó tem 2 DOFs: deslocamento
    vertical (w) e rotação (θ)
144 ndof = 2 * nn
145 K = np.zeros((ndof, ndof))
146 f = np.zeros(ndof)
147
148 # Montagem da matriz global com propriedades variáveis
149 for e in range(nel):
150     # Propriedades do elemento (média dos nós)
151     E_elem = (E[e] + E[e+1]) / 2
152     I_elem = (I[e] + I[e+1]) / 2
153     EI = E_elem * I_elem
154
155     # Carregamento do elemento (média dos nós)
156     q_elem = (q[e] + q[e+1]) / 2
157
158     # Matriz de rigidez do elemento
159     ke = (EI / h**3) * np.array([
160         [12,      6*h,    -12,      6*h],
161         [6*h,     4*h**2, -6*h,     2*h**2],
162         [-12,    -6*h,     12,    -6*h],
163         [6*h,     2*h**2, -6*h,     4*h**2]
164     ])
165
166     # Vetor de forças do elemento
167     fe = (q_elem * h / 12) * np.array([6, h, 6, -h])
168
169     # DOFs do elemento
170     dofs = [2*e, 2*e+1, 2*(e+1), 2*(e+1)+1]
171
172     # Montar matriz de rigidez

```

```

173         for i in range(4):
174             for j in range(4):
175                 K[dofs[i], dofs[j]] += ke[i, j]
176
177         # Montar vetor de forças
178         for i in range(4):
179             f[dofs[i]] += fe[i]
180
181     # Aplicar condições de contorno
182     if bc_type == "simply_supported":
183         # Simplesmente apoiada:  $w(0) = 0$ ,  $w(L) = 0$ 
184         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
185         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
186
187     elif bc_type == "cantilever":
188         # Viga em balanço:  $w(0) = 0$ ,  $\theta(0) = 0$ 
189         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
190         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
191
192     elif bc_type == "fixed_fixed":
193         # Engastada-engastada:  $w(0) = 0$ ,  $\theta(0) = 0$ ,  $w(L) = 0$ ,  $\theta(L)$ 
194         #  $= 0$ 
195         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
196         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
197         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
198         K[2*nn-1, :] = 0; K[2*nn-1, 2*nn-1] = 1; f[2*nn-1] = 0
199
200     elif bc_type == "fixed_supported":
201         # Engastada-apoiada:  $w(0) = 0$ ,  $\theta(0) = 0$ ,  $w(L) = 0$ 
202         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
203         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
204         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
205
206     # Resolver sistema
207     u = np.linalg.solve(K, f)
208
209     # Extrair deslocamentos verticais e rotações
210     w = u[::2] # deslocamentos verticais
211     theta = u[1::2] # rotações

```

```

211
212     # Pos-processamento de M (momento) e V (cortante) via
        derivadas canonicas
213     # das funcoes de Hermite cubica. Para cada elemento de
        comprimento h com
214     # DOFs [w1, theta1, w2, theta2]:
215     #   M(x) = -EI * w''(x) e linear em xi (coordenada local em
        [0, 1])
216     #   V(x) = -EI * w'''(x) e constante no elemento (3a
        derivada de cubico)
217     # Avaliamos M nos dois nos locais e V uma vez por elemento,
        somando nos
218     # nos globais e depois dividindo pelo numero de elementos
        adjacentes.
219     # Isso elimina o salto espurio que um pos-processamento por
        diferencas
220     # finitas introduzia nos 2 nos de cada extremidade.
221     M = np.zeros(nn)
222     V = np.zeros(nn)
223     count = np.zeros(nn)
224     for e in range(nel):
225         E_elem = (E[e] + E[e+1]) / 2
226         I_elem = (I[e] + I[e+1]) / 2
227         EI = E_elem * I_elem
228         w1, th1 = w[e], theta[e]
229         w2, th2 = w[e+1], theta[e+1]
230         # d2N/dxi2 em xi=0: [-6, -4h, 6, -2h]    -> M no no
            esquerdo
231         M_left = -EI / h**2 * (-6*w1 - 4*h*th1 + 6*w2 - 2*h*th2
            )
232         # d2N/dxi2 em xi=1: [ 6,  2h, -6,  4h]    -> M no no
            direito
233         M_right = -EI / h**2 * ( 6*w1 + 2*h*th1 - 6*w2 + 4*h*th2
            )
234         # d3N/dxi3 constante: [12, 6h, -12, 6h]    -> V no
            elemento
235         V_elem = -EI / h**3 * (12*w1 + 6*h*th1 - 12*w2 + 6*h*
            th2)
236         M[e] += M_left

```

```

237         M[e+1] += M_right
238         V[e]    += V_elem
239         V[e+1] += V_elem
240         count[e] += 1
241         count[e+1] += 1
242     M /= count
243     V /= count
244
245     # Solução analítica para comparação (se disponível)
246     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))
247     w_ana_dense = np.zeros_like(x_dense)
248
249     if show_analytical and prop_type == "constant":
250         if bc_type == "simply_supported" and load_type == "
            uniform":
251             # Solução analítica para viga simplesmente apoiada
                com carregamento uniforme
252             q_ana = q_magnitude # Usar a magnitude real do
                carregamento
253             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (L
                **3 * x_dense - 2 * L * x_dense**3 + x_dense**4)
254         elif bc_type == "cantilever" and load_type == "uniform":
255             # Solução analítica para viga em balanço com
                carregamento uniforme
256             q_ana = q_magnitude
257             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (6
                * L**2 * x_dense**2 - 4 * L * x_dense**3 +
                x_dense**4)
258         elif bc_type == "fixed_fixed" and load_type == "uniform"
            :
259             # Solução analítica para viga engastada-engastada
                com carregamento uniforme
260             q_ana = q_magnitude
261             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (L
                **2 * x_dense**2 - 2 * L * x_dense**3 + x_dense
                **4)
262
263     # Determinar título da condição de contorno
264     bc_titles = {

```



```

265         "simply_supported": "Simplesmente Apoiada",
266         "cantilever": "Em Balanço",
267         "fixed_fixed": "Engastada-Engastada",
268         "fixed_supported": "Engastada-Apoiada"
269     }
270     bc_title = bc_titles.get(bc_type, "Desconhecida")
271
272     # Determinar título do carregamento
273     load_titles = {
274         "uniform": "Uniforme",
275         "linear": "Linear",
276         "triangular": "Triangular",
277         "sinusoidal": "Senoidal",
278         "point": "Pontual"
279     }
280     load_title = load_titles.get(load_type, "Desconhecido")
281
282     # Plot avançado com múltiplos gráficos
283     fig = plt.figure(figsize=(12, 18))
284
285     # Layout: 5x1 grid
286     gs = fig.add_gridspec(5, 1, height_ratios=[2, 1.5, 1.5, 1.5,
287         1.5])
288
289     # 1. Deslocamentos
290     ax1 = fig.add_subplot(gs[0, :])
291     ax1.plot(x, w*1000, 'o-', label='MEF', color='#3B82F6',
292         markersize=6, linewidth=3)
293     if show_analytical and np.any(w_ana_dense):
294         ax1.plot(x_dense, w_ana_dense*1000, '--', label='Analí
295             tica', color='#EF4444', linewidth=3)
296     ax1.set_xlabel('x [m]', fontsize=14, fontweight='bold')
297     ax1.set_ylabel('Deslocamento [mm]', fontsize=14, fontweight=
298         'bold')
299     ax1.set_title(f'Deslocamentos Verticais - {bc_title} ({
300         load_title})', fontsize=16, fontweight='bold')
301     ax1.grid(True, linestyle='--', alpha=0.5)
302     ax1.legend(fontsize=12)
303     ax1.invert_yaxis()

```

```

299     ax1.tick_params(labelsize=12)
300
301     # 2. Carregamento
302     ax2 = fig.add_subplot(gs[1, :])
303     ax2.plot(x, q/1000, 'g-', linewidth=3, label='Carregamento')
304     ax2.fill_between(x, 0, q/1000, alpha=0.3, color='green')
305     ax2.set_xlabel('x [m]', fontsize=14, fontweight='bold')
306     ax2.set_ylabel('Carregamento [kN/m]', fontsize=14,
307                    fontweight='bold')
308     ax2.set_title('Carregamento Distribuído', fontsize=16,
309                  fontweight='bold')
310     ax2.grid(True, linestyle='--', alpha=0.5)
311     ax2.legend(fontsize=12)
312     ax2.tick_params(labelsize=12)
313
314     # 3. Propriedades
315     ax3 = fig.add_subplot(gs[2, :])
316
317     # Plotar linha vermelha primeiro (E) com estilo mais visível
318     line1 = ax3.plot(x, E/1e9, 'r-', linewidth=4, label='E [GPa]
319                    ', alpha=0.9, zorder=3)
320
321     # Criar eixo secundário para linha azul (I) - tracejada e
322     mais fina
323     ax3_twin = ax3.twinx()
324     line2 = ax3_twin.plot(x, I*1e8, 'b--', linewidth=2, label='I
325                    [cm4]', alpha=0.8, zorder=2)
326
327     # Configurar eixos e labels
328     ax3.set_xlabel('x [m]', fontsize=14, fontweight='bold')
329     ax3.set_ylabel('Módulo de Elasticidade [GPa]', color='r',
330                   fontweight='bold', fontsize=14)
331     ax3_twin.set_ylabel('Momento de Inércia [cm4]', color='b',
332                       fontweight='bold', fontsize=14)
333     ax3.set_title('Propriedades Materiais', fontweight='bold',
334                  fontsize=16)
335
336     # Configurar cores dos ticks
337     ax3.tick_params(axis='y', labelcolor='r', labelsize=12)

```

```

330     ax3_twin.tick_params(axis='y', labelcolor='b', labelsiz=12)
331     ax3.tick_params(axis='x', labelsiz=12)
332
333     # Grid mais sutil
334     ax3.grid(True, linestyle='--', alpha=0.3)
335
336     # Ajustar limites dos eixos para melhor visualização
337     E_min, E_max = np.min(E/1e9), np.max(E/1e9)
338     I_min, I_max = np.min(I*1e8), np.max(I*1e8)
339
340     ax3.set_ylim(0.9 * E_min, 1.1 * E_max)
341     ax3_twin.set_ylim(0.9 * I_min, 1.1 * I_max)
342
343     # Adicionar legendas com cores corretas e mais visíveis
344     ax3.legend(loc='upper left', fontsize=12, framealpha=0.9,
345               edgecolor='r')
346     ax3_twin.legend(loc='upper right', fontsize=12, framealpha
347                    =0.9, edgecolor='b')
348
349     # 4. Momentos fletores
350     ax4 = fig.add_subplot(gs[3, :])
351     ax4.plot(x, M/1000, 'o-', color='#10B981', markersize=6,
352             linewidth=3)
353     ax4.set_xlabel('x [m]', fontsize=14, fontweight='bold')
354     ax4.set_ylabel('Momento Fletor [kN·m]', fontsize=14,
355                   fontweight='bold')
356     ax4.set_title('Momento Fletor', fontsize=16, fontweight='
357                   bold')
358     ax4.grid(True, linestyle='--', alpha=0.5)
359     ax4.tick_params(labelsiz=12)
360
361     # 5. Esforços cortantes
362     ax5 = fig.add_subplot(gs[4, :])
363     ax5.plot(x, V/1000, 'o-', color='#F59E0B', markersize=6,
364             linewidth=3)
365     ax5.set_xlabel('x [m]', fontsize=14, fontweight='bold')
366     ax5.set_ylabel('Esforço Cortante [kN]', fontsize=14,
367                   fontweight='bold')

```

```

361     ax5.set_title('Esforço Cortante', fontsize=16, fontweight='
        bold')
362     ax5.grid(True, linestyle='--', alpha=0.5)
363     ax5.tick_params(labelsize=12)
364
365     plt.tight_layout()
366
367     # Converter para GIF
368     buf = io.BytesIO()
369     plt.savefig(buf, format='png', dpi=150, bbox_inches='tight')
370     plt.close(fig)
371     buf.seek(0)
372     img = imageio.imread(buf)
373     gif_buffer = io.BytesIO()
374     imageio.mimsave(gif_buffer, [img], format='GIF', duration
        =1.0)
375     gif_buffer.seek(0)
376     gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
        -8')
377
378     container = document.getElementById("vigaId-output")
379     container.innerHTML = f"<img src='data:image/gif;base64,{
        gif_base64}' class='rounded shadow w-full' />"
380
381     try:
382         sim_loading.close()
383     except Exception:
384         pass
385
386
387     # =====
388     # 5. Função para casos pré-definidos
389     # =====
390     @when("click", "#loadCase")
391     async def load_preset_case(event=None):
392         case = document.getElementById("preset_cases").value
393
394         if case == "concrete_beam":
395             document.getElementById("L").value = "6.0"

```

```

396     document.getElementById("E").value = "30.0"
397     document.getElementById("I").value = "20000.0"
398     document.getElementById("q").value = "15.0"
399     document.getElementById("bc_type").value = "
        simply_supported"
400     document.getElementById("load_type").value = "uniform"
401     document.getElementById("prop_type").value = "constant"
402
403     elif case == "steel_beam":
404         document.getElementById("L").value = "8.0"
405         document.getElementById("E").value = "200.0"
406         document.getElementById("I").value = "50000.0"
407         document.getElementById("q").value = "25.0"
408         document.getElementById("bc_type").value = "
            simply_supported"
409         document.getElementById("load_type").value = "uniform"
410         document.getElementById("prop_type").value = "constant"
411
412     elif case == "cantilever_roof":
413         document.getElementById("L").value = "4.0"
414         document.getElementById("E").value = "200.0"
415         document.getElementById("I").value = "15000.0"
416         document.getElementById("q").value = "8.0"
417         document.getElementById("bc_type").value = "cantilever"
418         document.getElementById("load_type").value = "triangular
            "
419         document.getElementById("prop_type").value = "linear"
420
421     elif case == "bridge_beam":
422         document.getElementById("L").value = "20.0"
423         document.getElementById("E").value = "200.0"
424         document.getElementById("I").value = "100000.0"
425         document.getElementById("q").value = "50.0"
426         document.getElementById("bc_type").value = "
            simply_supported"
427         document.getElementById("load_type").value = "point"
428         document.getElementById("prop_type").value = "constant"

```

A.5 Vibração 1D — Problema de Autovalor

Listagem A.5: Vibração 1D — simulation.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Problema de Autovalor 1D - Vibração em Corpo Sólido (MEF)
5  Problema 5.2.6: Solução de problema de autovalor 1D - vibração
   em um corpo sólido
6
7  Este código resolve o problema de vibração livre de uma barra 1D
   usando MEF,
8  calculando frequências naturais e modos de vibração através da
   solução do problema de autovalor.
9  """
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13 from js import document
14 from pyscript import when
15 import asyncio
16 import io
17 import imageio.v2 as imageio
18 import base64
19 from scipy.linalg import eig
20
21
22 # =====
23 # 1. Funções auxiliares para propriedades variáveis
24 # =====
25 def get_variable_properties(x, prop_type, base_value, L):
26     """Calcula propriedades variáveis ao longo da barra"""
27     if prop_type == "constant":
28         return np.full_like(x, base_value)
29     elif prop_type == "linear":
30         # Propriedade varia linearmente de base_value a 0.5*
           base_value
31         return base_value * (1 - 0.5 * x / L)
```

```

32     elif prop_type == "parabolic":
33         # Propriedade varia parabolicamente (máximo no centro)
34         return base_value * (1 - 0.3 * (x - L/2)**2 / (L/2)**2)
35     elif prop_type == "exponential":
36         # Propriedade varia exponencialmente
37         return base_value * np.exp(-0.5 * x / L)
38     return np.full_like(x, base_value)
39
40
41 # =====
42 # 2. Soluções analíticas para frequências naturais
43 # =====
44 def analytical_frequencies_free_free(L, E, rho, n_modes=5):
45     """Frequências naturais analíticas para barra livre-livre"""
46     c = np.sqrt(E / rho) # Velocidade de onda
47     frequencies = []
48     for n in range(1, n_modes + 1):
49         # Para barra livre-livre:  $f_n = (n-1) * c / (2*L)$  para  $n > 1$ 
50         # Primeiro modo é movimento de corpo rígido ( $f = 0$ )
51         if n == 1:
52             frequencies.append(0.0)
53         else:
54             frequencies.append((n-1) * c / (2*L))
55     return np.array(frequencies)
56
57
58 def analytical_frequencies_fixed_fixed(L, E, rho, n_modes=5):
59     """Frequências naturais analíticas para barra engastada-
60         engastada"""
61     c = np.sqrt(E / rho) # Velocidade de onda
62     frequencies = []
63     for n in range(1, n_modes + 1):
64         frequencies.append(n * c / (2*L))
65     return np.array(frequencies)
66
67 def analytical_frequencies_fixed_free(L, E, rho, n_modes=5):

```

```

68     """Frequências naturais analíticas para barra engastada-
        livre"""
69     c = np.sqrt(E / rho) # Velocidade de onda
70     frequencies = []
71     for n in range(1, n_modes + 1):
72         frequencies.append((2*n-1) * c / (4*L))
73     return np.array(frequencies)
74
75
76 # =====
77 # 3. Executar simulação MEF para vibração 1D
78 # =====
79 @when("click", "#runVibração1D")
80 async def run_vibração_1d(event=None):
81     # Abrir modal de execução
82     sim_loading = document.getElementById("sim-loading")
83     try:
84         sim_loading.showModal()
85     except Exception:
86         pass
87     await asyncio.sleep(0.05)
88
89     # Parâmetros básicos
90     L = float(document.getElementById("L").value)
91     nel = int(document.getElementById("nel").value)
92     E_base = float(document.getElementById("E").value) * 1e9 #
        Converter para Pa
93     rho_base = float(document.getElementById("rho").value) # kg
        /m³
94     A_base = float(document.getElementById("A").value) * 1e-4 #
        Converter para m²
95     bc_type = document.getElementById("bc_type").value
96     prop_type = document.getElementById("prop_type").value
97     n_modes = 5 # Fixo em 5 modos
98     show_analytical = document.getElementById("show_analytical")
        .checked
99
100
101     nn = nel + 1

```



```

102     x = np.linspace(0.0, L, nn)
103     h = L / nel
104
105     # Propriedades variáveis
106     E = get_variable_properties(x, prop_type, E_base, L)
107     rho = get_variable_properties(x, prop_type, rho_base, L)
108     A = get_variable_properties(x, prop_type, A_base, L)
109
110     # Para elementos de barra, cada nó tem 1 DOF: deslocamento
axial (u)
111     ndof = nn
112     K = np.zeros((ndof, ndof)) # Matriz de rigidez
113     M = np.zeros((ndof, ndof)) # Matriz de massa
114
115     # Montagem das matrizes globais
116     for e in range(nel):
117         # Propriedades do elemento (média dos nós)
118         E_elem = (E[e] + E[e+1]) / 2
119         rho_elem = (rho[e] + rho[e+1]) / 2
120         A_elem = (A[e] + A[e+1]) / 2
121
122         # Matriz de rigidez do elemento de barra
123         ke = (E_elem * A_elem / h) * np.array([
124             [1, -1],
125             [-1, 1]
126         ])
127
128         # Matriz de massa do elemento de barra (massa
concentrada)
129         me = (rho_elem * A_elem * h / 2) * np.array([
130             [1, 0],
131             [0, 1]
132         ])
133
134         # DOFs do elemento
135         dofs = [e, e+1]
136
137         # Montar matriz de rigidez
138         for i in range(2):

```

```

139         for j in range(2):
140             K[dofs[i], dofs[j]] += ke[i, j]
141
142         # Montar matriz de massa
143         for i in range(2):
144             for j in range(2):
145                 M[dofs[i], dofs[j]] += me[i, j]
146
147         # Aplicar condições de contorno
148         if bc_type == "free_free":
149             # Barra livre-livre: sem restrições
150             pass
151
152         elif bc_type == "fixed_fixed":
153             # Barra engastada-engastada:  $u(0) = 0$ ,  $u(L) = 0$ 
154             # Aplicar penalidade nas diagonais principais
155             penalty = 1e12
156             K[0, 0] += penalty
157             K[nn-1, nn-1] += penalty
158
159         elif bc_type == "fixed_free":
160             # Barra engastada-livre:  $u(0) = 0$ 
161             penalty = 1e12
162             K[0, 0] += penalty
163
164         elif bc_type == "fixed_supported":
165             # Barra engastada-apoiada:  $u(0) = 0$ ,  $u(L) = 0$ 
166             penalty = 1e12
167             K[0, 0] += penalty
168             K[nn-1, nn-1] += penalty
169
170         # Resolver problema de autovalor:  $K * \phi = \lambda * M * \phi$ 
171         # Usar scipy.linalg.eigh para matrizes simétricas
172         eigenvalues, eigenvectors = eigh(K, M)
173
174         # Calcular frequências naturais (Hz)
175         frequencies = np.sqrt(eigenvalues) / (2 * np.pi)
176
177         # Ordenar por frequência crescente

```

```

178     idx = np.argsort(frequencies)
179     frequencies = frequencies[idx]
180     eigenvectors = eigenvectors[:, idx]
181
182     # Pegar apenas os primeiros n_modes
183     frequencies = frequencies[:n_modes]
184     eigenvectors = eigenvectors[:, :n_modes]
185
186     # Normalizar modos de vibração
187     for i in range(n_modes):
188         eigenvectors[:, i] = eigenvectors[:, i] / np.max(np.abs(
189             eigenvectors[:, i]))
190
191     # Solução analítica para comparação (se disponível)
192     freq_analytical = np.zeros(n_modes)
193
194     if show_analytical and prop_type == "constant":
195         if bc_type == "free_free":
196             freq_analytical = analytical_frequencies_free_free(L,
197                 E_base, rho_base, n_modes)
198         elif bc_type == "fixed_fixed":
199             freq_analytical = analytical_frequencies_fixed_fixed(
200                 L, E_base, rho_base, n_modes)
201         elif bc_type == "fixed_free":
202             freq_analytical = analytical_frequencies_fixed_free(
203                 L, E_base, rho_base, n_modes)
204
205     # Determinar título da condição de contorno
206     bc_titles = {
207         "free_free": "Livre-Livre",
208         "fixed_fixed": "Engastada-Engastada",
209         "fixed_free": "Engastada-Livre",
210         "fixed_supported": "Engastada-Apoiada"
211     }
212     bc_title = bc_titles.get(bc_type, "Desconhecida")
213
214     # Plot avançado com múltiplos gráficos
215     fig = plt.figure(figsize=(14, 22))

```

```

213     # Layout: 7x1 grid (1 tabela + 5 modos + 1 propriedades)
214     gs = fig.add_gridspec(7, 1, height_ratios=[1.2, 2.2, 2.2,
215         2.2, 2.2, 2.2, 1.5], hspace=0.5)
216
217     # 1. Tabela de frequências
218     ax1 = fig.add_subplot(gs[0, :])
219     ax1.axis('off')
220
221     # Criar tabela de frequências
222     table_data = []
223     show_analytical_table = show_analytical and prop_type == "
224         constant"
225
226     for i in range(min(5, n_modos)):
227         row = [f'Modo {i+1}', f'{frequencies[i]:.2f} Hz']
228         if show_analytical_table and i < len(freq_analytical)
229             and freq_analytical[i] > 0:
230             error = abs(frequencies[i] - freq_analytical[i]) /
231                 freq_analytical[i] * 100
232             row.append(f'{freq_analytical[i]:.2f} Hz')
233             row.append(f'{error:.1f}%')
234         elif show_analytical_table:
235             row.extend(['-', '-'])
236         table_data.append(row)
237
238     # Definir cabeçalhos e larguras baseado nas condições
239     if show_analytical_table:
240         headers = ['Modo', 'MEF [Hz]', 'Analítica [Hz]', 'Erro
241             [%]']
242         col_widths = [0.2, 0.2, 0.2, 0.2]
243     else:
244         headers = ['Modo', 'MEF [Hz]']
245         col_widths = [0.3, 0.3]
246
247     table = ax1.table(cellText=table_data, colLabels=headers,
248         cellLoc='center', loc='center',
249         colWidths=col_widths)
250     table.auto_set_font_size(False)
251     table.set_fontsize(12)

```

```

247     table.scale(1, 2)
248
249     # Colorir cabecalho
250     for i in range(len(headers)):
251         table[(0, i)].set_facecolor('#4F46E5')
252         table[(0, i)].set_text_props(weight='bold', color='white
253                                     ')
254
255     ax1.set_title(f'Frequências Naturais - {bc_title}', fontsize
256                 =16, fontweight='bold', pad=50)
257
258     # 2-6. Modos de vibração (5 modos fixos)
259     colors = ['#3B82F6', '#EF4444', '#10B981', '#F59E0B', '#8
260             B5CF6']
261     for i in range(min(5, n_modes)):
262         ax = fig.add_subplot(gs[i+1, :])
263
264         # Plotar modo de vibração
265         ax.plot(x, eigenvectors[:, i], 'o-', color=colors[i],
266               markersize=6, linewidth=3,
267               label=f'Modo {i+1}: {frequencies[i]:.2f} Hz')
268
269         # Adicionar linha de referência em zero
270         ax.axhline(y=0, color='black', linestyle='--', alpha
271                   =0.3)
272
273         ax.set_xlabel('x [m]', fontsize=14, fontweight='bold')
274         ax.set_ylabel('Amplitude Normalizada', fontsize=14,
275                       fontweight='bold')
276         ax.set_title(f'Modo de Vibração {i+1}', fontsize=16,
277                       fontweight='bold')
278         ax.grid(True, linestyle='--', alpha=0.5)
279         ax.legend(fontsize=12)
280         ax.tick_params(labelsize=12)
281
282         # Adicionar nós da malha
283         ax.scatter(x, eigenvectors[:, i], color=colors[i], s=50,
284                   zorder=5)
285
286

```

```

278     # 7. Propriedades materiais
279     ax_props = fig.add_subplot(gs[6, :])
280
281     # Plotar linha vermelha primeiro (E) com estilo mais visível
282     line1 = ax_props.plot(x, E/1e9, 'r-', linewidth=4, label='E
        [GPa]', alpha=0.9, zorder=3)
283
284     # Criar eixo secundário para linha azul (rho) - tracejada e
        mais fina
285     ax_props_twin = ax_props.twinx()
286     line2 = ax_props_twin.plot(x, rho, 'b--', linewidth=2, label
        = 'ρ [kg/m³]', alpha=0.8, zorder=2)
287
288     # Configurar eixos e labels
289     ax_props.set_xlabel('x [m]', fontsize=14, fontweight='bold')
290     ax_props.set_ylabel('Módulo de Elasticidade [GPa]', color='r
        ', fontweight='bold', fontsize=14)
291     ax_props_twin.set_ylabel('Densidade [kg/m³]', color='b',
        fontweight='bold', fontsize=14)
292     ax_props.set_title('Propriedades Materiais', fontweight='
        bold', fontsize=16)
293
294     # Configurar cores dos ticks
295     ax_props.tick_params(axis='y', labelcolor='r', labelsz=12)
296     ax_props_twin.tick_params(axis='y', labelcolor='b',
        labelsz=12)
297     ax_props.tick_params(axis='x', labelsz=12)
298
299     # Grid mais sutil
300     ax_props.grid(True, linestyle='--', alpha=0.3)
301
302     # Ajustar limites dos eixos para melhor visualização
303     E_min, E_max = np.min(E/1e9), np.max(E/1e9)
304     rho_min, rho_max = np.min(rho), np.max(rho)
305
306     ax_props.set_ylim(0.9 * E_min, 1.1 * E_max)
307     ax_props_twin.set_ylim(0.9 * rho_min, 1.1 * rho_max)
308
309     # Adicionar legendas com cores corretas e mais visíveis

```

```

310     ax_props.legend(loc='upper left', fontsize=12, framealpha
        =0.9, edgecolor='r')
311     ax_props_twin.legend(loc='upper right', fontsize=12,
        framealpha=0.9, edgecolor='b')
312
313     # Converter para GIF
314     buf = io.BytesIO()
315     plt.savefig(buf, format='png', dpi=150, bbox_inches='tight')
316     plt.close(fig)
317     buf.seek(0)
318     img = imageio.imread(buf)
319     gif_buffer = io.BytesIO()
320     imageio.mimsave(gif_buffer, [img], format='GIF', duration
        =1.0)
321     gif_buffer.seek(0)
322     gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
        -8')
323
324     container = document.getElementById("vibracao1d-output")
325     container.innerHTML = f"<img src='data:image/gif;base64,{
        gif_base64}' class='rounded shadow w-full'/"
326
327     try:
328         sim_loading.close()
329     except Exception:
330         pass
331
332
333     # =====
334     # 4. Função para casos pré-definidos
335     # =====
336     @when("click", "#loadCase")
337     async def load_preset_case(event=None):
338         case = document.getElementById("preset_cases").value
339
340         if case == "steel_bar":
341             document.getElementById("L").value = "2.0"
342             document.getElementById("E").value = "200.0"
343             document.getElementById("rho").value = "7850.0"

```

```

344     document.getElementById("A").value = "100.0"
345     document.getElementById("bc_type").value = "fixed_fixed"
346     document.getElementById("prop_type").value = "constant"
347
348     elif case == "aluminum_bar":
349         document.getElementById("L").value = "1.5"
350         document.getElementById("E").value = "70.0"
351         document.getElementById("rho").value = "2700.0"
352         document.getElementById("A").value = "50.0"
353         document.getElementById("bc_type").value = "fixed_free"
354         document.getElementById("prop_type").value = "constant"
355
356     elif case == "concrete_column":
357         document.getElementById("L").value = "3.0"
358         document.getElementById("E").value = "30.0"
359         document.getElementById("rho").value = "2400.0"
360         document.getElementById("A").value = "400.0"
361         document.getElementById("bc_type").value = "fixed_fixed"
362         document.getElementById("prop_type").value = "linear"
363
364     elif case == "composite_bar":
365         document.getElementById("L").value = "1.0"
366         document.getElementById("E").value = "100.0"
367         document.getElementById("rho").value = "2000.0"
368         document.getElementById("A").value = "25.0"
369         document.getElementById("bc_type").value = "free_free"
370         document.getElementById("prop_type").value = "parabolic"

```


Apêndice B

Listagens dos Módulos de Simulação 2D

Este apêndice apresenta as listagens completas dos módulos de simulação bidimensionais implementados no *MinervaMesh*, incluindo os módulos compartilhados de geração de malha e geometria. Os códigos são incluídos com o propósito de garantir a reprodutibilidade científica dos resultados apresentados neste trabalho: cada listagem corresponde diretamente às formulações matemáticas derivadas no Capítulo 3 e às implementações descritas no Capítulo 4. Cabe ressaltar que este apêndice não constitui um repositório de software, mas sim documentação técnica destinada à verificação e validação das soluções numéricas obtidas. Os códigos Python a seguir são executados integralmente no navegador via PyScript/WebAssembly.

B.1 Módulo Compartilhado: Malha e Matrizes MEF

Listagem B.1: Módulo compartilhado de malha e matrizes MEF — `common.py`

```
1 import numpy as np
2 import matplotlib.tri as tri
3 from scipy.sparse import lil_matrix, csr_matrix
4 import matplotlib.pyplot as plt
5 import io
6 import base64
```

```

7
8 # =====
9 # 1. Geração da Malha (Genérica)
10 # =====
11 def generate_mesh_generic(L, H, boundary_points, mask_function,
12                             cx, cy, nx, ny):
13     """
14     Gera malha triangular com um buraco definido por
15     boundary_points.
16     boundary_points: array (N, 2) com pontos da fronteira do
17     obstáculo.
18     mask_function: função f(x, y) -> bool que retorna True se o
19     ponto deve ser MANTIDO (fora do obstáculo).
20     """
21     # 1. Grid de fundo
22     x = np.linspace(0, L, nx)
23     y = np.linspace(0, H, ny)
24     dx = x[1] - x[0]
25     dy = y[1] - y[0]
26     h_mesh = min(dx, dy)
27
28     Xg, Yg = np.meshgrid(x, y)
29     X_flat = Xg.flatten()
30     Y_flat = Yg.flatten()
31
32     # 2. Nós da fronteira do obstáculo (passados como argumento)
33     x_bound = boundary_points[:, 0]
34     y_bound = boundary_points[:, 1]
35
36     # 3. Filtrar nós do grid usando a função de máscara
37     # Buffer para evitar triângulos muito finos (slivers)
38     # A função de máscara deve ser conservadora (remover apenas
39     # o que está garantidamente dentro)
40     # Mas aqui vamos aplicar diretamente
41     mask_keep = mask_function(X_flat, Y_flat)
42
43     X_grid_valid = X_flat[mask_keep]
44     Y_grid_valid = Y_flat[mask_keep]

```

```

41     # 4. Combinar nós
42     X = np.concatenate([X_grid_valid, x_bound])
43     Y = np.concatenate([Y_grid_valid, y_bound])
44
45     # 5. Triangulação
46     triang = tri.Triangulation(X, Y)
47
48     # 6. Mascaram triângulos espúrios (dentro do obstáculo)
49     x_tri = X[triang.triangles].mean(axis=1)
50     y_tri = Y[triang.triangles].mean(axis=1)
51
52     # Se o centróide não passa na máscara de "manter", então
53     removemos
54     # Mas cuidado: a máscara de manter pode ser "dist > r +
55     buffer".
56     # Para triângulos, queremos remover se estiver DENTRO do
57     obstáculo exato.
58     # Vamos assumir que mask_function(x, y, strict=True) remove
59     o interior.
60     # Por simplicidade, vamos passar uma segunda função ou usar
61     a mesma com lógica interna.
62     # Vamos usar a mesma mask_function. Se retornar False (
63     remover), mascaramos.
64
65     triang.set_mask(~mask_function(x_tri, y_tri))
66
67     return X, Y, triang
68
69     # =====
70     # 2. Matrizes MEF (Triângulo Linear)
71     # =====
72
73     def element_matrices(coords):
74         x0, x1, x2 = coords
75         detJ = (x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0] - x0[0]) *
76             (x1[1] - x0[1])
77         area = 0.5 * abs(detJ)
78
79         b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
80         c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])

```

```

73
74     ke = (1.0 / (4 * area)) * (np.outer(b, b) + np.outer(c, c))
75     me = (area / 12.0) * (np.ones((3, 3)) + np.eye(3))
76
77     return ke, me, area, b, c
78
79 def assemble_matrices(X, Y, triang):
80     n_nodes = len(X)
81     K = lil_matrix((n_nodes, n_nodes))
82     M = lil_matrix((n_nodes, n_nodes))
83
84     elements = triang.triangles
85     mask = triang.mask if triang.mask is not None else np.zeros(
86         len(elements), dtype=bool)
87
88     for i, el in enumerate(elements):
89         if mask[i]: continue
90
91         coords = np.column_stack((X[el], Y[el]))
92         ke, me, area, b, c = element_matrices(coords)
93
94         for row in range(3):
95             for col in range(3):
96                 K[el[row], el[col]] += ke[row, col]
97                 M[el[row], el[col]] += me[row, col]
98
99     return K.tocsr(), M.tocsr()
100
101 # =====
102 # 3. Helpers de Plotagem
103 # =====
104 def plot_to_base64(fig):
105     buf = io.BytesIO()
106     plt.savefig(buf, format='png', bbox_inches='tight')
107     plt.close(fig)
108     buf.seek(0)
109     return base64.b64encode(buf.read()).decode("utf-8")

```

B.2 Módulo Compartilhado: Geometrias de Obstáculo

Listagem B.2: Geometrias de obstáculo — geometry.py

```
1 import numpy as np
2
3 def get_cylinder(params):
4     cx, cy, r = params["cx"], params["cy"], params["r"]
5     n_pts = 100
6     theta = np.linspace(0, 2*np.pi, n_pts, endpoint=False)
7     x_bound = cx + r * np.cos(theta)
8     y_bound = cy + r * np.sin(theta)
9     boundary_pts = np.column_stack([x_bound, y_bound])
10
11     def mask_func(x, y):
12         return (x - cx)**2 + (y - cy)**2 > (r * 1.01)**2
13
14     return boundary_pts, mask_func
15
16 def get_rectangle(params):
17     cx, cy = params["cx"], params["cy"]
18     w, h = params["w"], params["h"]
19
20     perim = 2 * (w + h)
21     n_w = int(100 * (w / perim))
22     n_h = int(100 * (h / perim))
23     n_w = max(5, n_w)
24     n_h = max(5, n_h)
25
26     hw = w / 2
27     hh = h / 2
28
29     x_b = np.linspace(cx - hw, cx + hw, n_w, endpoint=False)
30     y_b = np.full_like(x_b, cy - hh)
31
32     y_r = np.linspace(cy - hh, cy + hh, n_h, endpoint=False)
33     x_r = np.full_like(y_r, cx + hw)
34
35     x_t = np.linspace(cx + hw, cx - hw, n_w, endpoint=False)
```

```

36     y_t = np.full_like(x_t, cy + hh)
37
38     y_l = np.linspace(cy + hh, cy - hh, n_h, endpoint=False)
39     x_l = np.full_like(y_l, cx - hw)
40
41     x_bound = np.concatenate([x_b, x_r, x_t, x_l])
42     y_bound = np.concatenate([y_b, y_r, y_t, y_l])
43     boundary_pts = np.column_stack([x_bound, y_bound])
44
45     def mask_func(x, y):
46         buffer = 1.01
47         return (np.abs(x - cx) > hw * buffer) | (np.abs(y - cy)
48             > hh * buffer)
49
50     return boundary_pts, mask_func
51
52 def get_step(params):
53     # Backward facing step
54     # Step is at bottom left corner.
55     # Dimensions: step_h, step_l
56     step_h = params["step_h"]
57     step_l = params["step_l"]
58
59     # Boundary points for the step (L-shape)
60     # 1. Top of step (0, step_h) -> (step_l, step_h)
61     # 2. Back of step (step_l, step_h) -> (step_l, 0)
62
63     n_pts = 50
64     x_top = np.linspace(0, step_l, n_pts)
65     y_top = np.full_like(x_top, step_h)
66
67     y_back = np.linspace(step_h, 0, n_pts)
68     x_back = np.full_like(y_back, step_l)
69
70     # Concatenate
71     x_bound = np.concatenate([x_top, x_back])
72     y_bound = np.concatenate([y_top, y_back])
73     boundary_pts = np.column_stack([x_bound, y_bound])

```

```

74     def mask_func(x, y):
75         # Remove if x < step_l AND y < step_h
76         # Keep if x >= step_l OR y >= step_h
77         # Add small buffer to avoid removing boundary nodes
78         # We want to remove the block (0,0) to (step_l, step_h)
79         # So keep if NOT (x < step_l-eps AND y < step_h-eps)
80         eps = 1e-3
81         return ~((x < step_l - eps) & (y < step_h - eps))
82
83     return boundary_pts, mask_func
84
85
86
87     def get_none(params):
88         # Empty (for Cavity or pure channel)
89         return np.empty((0, 2), lambda x, y: np.ones_like(x, dtype=
90             bool))
91
92     def get_geometry(geo_type, params):
93         if geo_type == "cylinder":
94             return get_cylinder(params)
95         elif geo_type == "rectangle":
96             return get_rectangle(params)
97         elif geo_type == "step":
98             return get_step(params)
99         else:
100             return get_none(params)

```

B.3 Transferência de Calor 2D

Listagem B.3: Transferência de calor 2D — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Simulação Transiente de Condução de Calor em Placa 2D
5  @author: lgsfarias
6  """

```

```

7
8 # =====
9 # 0. Importações
10 # =====
11 import numpy as np
12 import matplotlib.pyplot as plt
13 import matplotlib.tri as tri
14 from scipy.sparse import lil_matrix, csr_matrix
15 from scipy.sparse.linalg import spsolve
16 from pyscript import when, document
17 import imageio.v2 as imageio
18 import io
19 import base64
20 import asyncio
21
22 # =====
23 # 1. Geração da Malha
24 # =====
25 nx, ny = 40, 40
26 x = np.linspace(0, 1, nx)
27 y = np.linspace(0, 1, ny)
28 Xg, Yg = np.meshgrid(x, y)
29 X = Xg.flatten()
30 Y = Yg.flatten()
31 points = np.column_stack((X, Y))
32 triang = tri.Triangulation(X, Y)
33 elements = triang.triangles
34 n_nodes = len(points)
35
36 # =====
37 # 2. Matrizes Locais
38 # =====
39 def element_matrices(coords, k, rho, cv):
40     x0, x1, x2 = coords
41     area = 0.5 * abs((x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0]
42         - x0[0]) * (x1[1] - x0[1]))
43     b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
44     c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
45     ke = (k / (4 * area)) * (np.outer(b, b) + np.outer(c, c))

```



```

45     me = (rho * cv * area / 12.0) * (np.ones((3, 3)) + np.eye(3)
46         )
47
48     return ke, me
49
50 # =====
51 # 3. Gerar imagem do frame
52 # =====
53 def generate_frame_image(u, step, dt, Tbottom, Ttop):
54     fig, ax = plt.subplots(figsize=(6, 5))
55     tpc = ax.tricontourf(triang, u, levels=50, cmap='jet', vmin=
56         Tbottom, vmax=Ttop)
57     ax.set_title(f"t = {step * dt:.3f} s")
58     ax.set_xlabel("x")
59     ax.set_ylabel("y")
60     fig.colorbar(tpc, ax=ax)
61     buf = io.BytesIO()
62     plt.tight_layout()
63     plt.savefig(buf, format='png')
64     plt.close(fig)
65     buf.seek(0)
66     return imageio.imread(buf)
67
68 # =====
69 # 4. Rodar Simulação
70 # =====
71 @when("click", "#run-btn")
72 async def run_simulation(event=None):
73     # Mostrar spinner de loading dentro do plot-output
74     document.getElementById("plot-output").innerHTML = """
75     <div class="flex flex-col items-center justify-center py-6
76         ">
77
78         <div class="animate-spin h-10 w-10 border-4 border-blue
79             -500 border-t-transparent rounded-full"></div>
80
81         <p class="mt-4 text-blue-700 font-medium">Calculando
82             simulação...</p>
83
84     </div>
85     """

```

```

78     await asyncio.sleep(0.05) # permite que o DOM atualize antes
    de continuar
79
80     # Parâmetros
81     dt = float(document.getElementById("dt").value)
82     n_steps = int(document.getElementById("n_steps").value)
83     Tbottom = float(document.getElementById("Tbottom").value)
84     Ttop = float(document.getElementById("Ttop").value)
85     k = float(document.getElementById("k").value)
86     rho = float(document.getElementById("rho").value)
87     cv = float(document.getElementById("cv").value)
88
89     # Matrizes globais
90     M = lil_matrix((n_nodes, n_nodes))
91     K = lil_matrix((n_nodes, n_nodes))
92     for el in elements:
93         coords = points[el]
94         ke, me = element_matrices(coords, k, rho, cv)
95         for i in range(3):
96             for j in range(3):
97                 K[el[i], el[j]] += ke[i, j]
98                 M[el[i], el[j]] += me[i, j]
99     M = M.tocsr()
100    K = K.tocsr()
101
102    # Condições iniciais
103    u = np.zeros(n_nodes)
104    for i in range(n_nodes):
105        xi, yi = X[i], Y[i]
106        if np.isclose(xi, 0) or np.isclose(xi, 1):
107            u[i] = Tbottom + (Ttop - Tbottom) * yi
108        elif np.isclose(yi, 0):
109            u[i] = Tbottom
110        elif np.isclose(yi, 1):
111            u[i] = Ttop
112
113    u_init = u.copy()
114    boundary_nodes = [i for i in range(n_nodes) if

```

```

115         np.isclose(X[i], 0) or np.isclose(X[i], 1)
116         or
117         np.isclose(Y[i], 0) or np.isclose(Y[i], 1)
118     ]
119
120     A = M + dt / 2 * K
121     B = M - dt / 2 * K
122     A = A.tolil()
123     for i in boundary_nodes:
124         A[i, :] = 0
125         A[i, i] = 1
126     A = A.tocsr()
127
128     # Loop de tempo
129     frames = []
130     for step in range(n_steps):
131         b = B @ u
132         b[boundary_nodes] = u_init[boundary_nodes]
133         u = spsolve(A, b)
134         frames.append(generate_frame_image(u, step, dt, Tbottom,
135                                           Ttop))
136
137     # Salvar GIF
138     gif_buffer = io.BytesIO()
139     imageio.mimsave(gif_buffer, frames, format='GIF', duration
140                     =0.1, loop=0)
141     gif_buffer.seek(0)
142     gif_base64 = base64.b64encode(gif_buffer.read()).decode("utf
143                             -8")
144
145     # Mostrar resultado final
146     document.getElementById("plot-output").innerHTML = (
147         f"<img src='data:image/gif;base64,{gif_base64}' class='
148             rounded shadow w-full' />"
149     )

```

B.4 Escoamento Potencial 2D

Listagem B.4: Escoamento potencial 2D — simulation.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.tri as tri
4  from scipy.sparse.linalg import spsolve
5  from pyscript import when, document
6  import asyncio
7  from common import generate_mesh_generic, assemble_matrices,
    plot_to_base64
8
9  # =====
10 # 1. Geometrias
11 # =====
12 def get_geometry(geo_type, params):
13     cx, cy = params["cx"], params["cy"]
14
15     if geo_type == "cylinder":
16         r = params["r"]
17         n_pts = 100
18         theta = np.linspace(0, 2*np.pi, n_pts, endpoint=False)
19         x_bound = cx + r * np.cos(theta)
20         y_bound = cy + r * np.sin(theta)
21
22         def mask_func(x, y):
23             return (x - cx)**2 + (y - cy)**2 > (r * 1.01)**2
24
25         return np.column_stack([x_bound, y_bound]), mask_func
26
27     elif geo_type == "airfoil" or geo_type == "naca4412":
28         # General NACA 4-Digit Series Generator
29         # M: Max camber (1st digit)
30         # P: Max camber pos (2nd digit)
31         # T: Max thickness (3rd, 4th digits)
32
33         if geo_type == "naca4412":
34             M = 0.04
35             P = 0.40
36             T = 0.12
37         else: # Default 0012

```

```

38         M = 0.00
39         P = 0.00
40         T = 0.12
41
42         chord = params["chord"]
43         angle = np.radians(params["angle"])
44
45         n_pts = 100
46         beta = np.linspace(0, np.pi, n_pts)
47         xc = 0.5 * (1 - np.cos(beta)) # Cosine spacing [0, 1]
48
49         # Thickness distribution (yt)
50         yt = 5 * T * (0.2969*np.sqrt(xc) - 0.1260*xc - 0.3516*xc
51             **2 + 0.2843*xc**3 - 0.1015*xc**4)
52
53         # Camber line (yc) and gradient (dyc_dx)
54         yc = np.zeros_like(xc)
55         dyc_dx = np.zeros_like(xc)
56
57         if M > 0:
58             # Forward of max camber (0 <= x <= P)
59             idx1 = xc <= P
60             yc[idx1] = (M / P**2) * (2*P*xc[idx1] - xc[idx1]**2)
61             dyc_dx[idx1] = (2*M / P**2) * (P - xc[idx1])
62
63             # Aft of max camber (P < x <= 1)
64             idx2 = xc > P
65             yc[idx2] = (M / (1-P)**2) * ((1-2*P) + 2*P*xc[idx2]
66                 - xc[idx2]**2)
67             dyc_dx[idx2] = (2*M / (1-P)**2) * (P - xc[idx2])
68
69         theta_c = np.arctan(dyc_dx)
70
71         # Upper and Lower surface coordinates
72         x_upper = xc - yt * np.sin(theta_c)
73         y_upper = yc + yt * np.cos(theta_c)
74         x_lower = xc + yt * np.sin(theta_c)
75         y_lower = yc - yt * np.cos(theta_c)

```

```

75         # Scale by chord
76         x_upper *= chord
77         y_upper *= chord
78         x_lower *= chord
79         y_lower *= chord
80
81         # Combine (trailing edge to leading edge to trailing
            edge)
82         x_local = np.concatenate([x_upper[::-1], x_lower[1:]])
83         y_local = np.concatenate([y_upper[::-1], y_lower[1:]])
84
85         # Center at cx, cy (approximate center of chord)
86         x_local -= chord / 2
87
88         # Rotate and Translate
89         c, s = np.cos(angle), np.sin(angle)
90         x_rot = c * x_local - s * y_local + cx
91         y_rot = s * x_local + c * y_local + cy
92
93         from matplotlib.path import Path
94         poly_path = Path(np.column_stack([x_rot, y_rot]))
95
96         def mask_func(x, y):
97             pts = np.column_stack([x, y])
98             return ~poly_path.contains_points(pts, radius=-1e-3)
99
100        return np.column_stack([x_rot, y_rot]), mask_func
101
102    return None, None
103
104    # =====
105    # 2. Solver Potencial
106    # =====
107    async def solve_potential(params, plot_div):
108        # 1. Geometria e Malha
109        boundary_pts, mask_func = get_geometry(params["geo_type"],
110                                                params)
111
112        X, Y, triang = generate_mesh_generic(

```

```

112         params["L"], params["H"], boundary_pts, mask_func,
113         params["cx"], params["cy"], params["nx"], params["ny"]
114     )
115     n_nodes = len(X)
116
117     # 2. Matrizes
118     K, M = assemble_matrices(X, Y, triang)
119
120     # 3. Fronteiras
121     eps = 1e-5
122     inlet_nodes = np.where(X < eps)[0]
123     outlet_nodes = np.where(X > params["L"] - eps)[0]
124     wall_bottom = np.where(Y < eps)[0]
125     wall_top = np.where(Y > params["H"] - eps)[0]
126
127     # Obstacle nodes: são os últimos N pontos em X, Y (pois
128     concatenamos)
129     n_bound = len(boundary_pts)
130     obstacle_nodes = np.arange(n_nodes - n_bound, n_nodes)
131
132     # 4. Condições de Contorno
133     psi = np.zeros(n_nodes)
134
135     # Normalizado para  $\Psi(H) = v_{inf} * H$ 
136     def get_psi_inlet(y_val):
137         return params["v_inf"] * y_val
138
139     psi[inlet_nodes] = get_psi_inlet(Y[inlet_nodes])
140     psi[wall_bottom] = 0.0
141     psi[wall_top] = params["v_inf"] * params["H"]
142
143     # Obstacle: Psi constante + Gamma
144     # Qual valor base? Psi no centro do obstáculo (aproximado)
145     psi_base = get_psi_inlet(params["cy"])
146     # Hardcode Gamma = 0.0 (Pure Potential Flow)
147     params["Gamma"] = 0.0
148     psi[obstacle_nodes] = psi_base + params["Gamma"]
149
150     # Dirichlet

```

```

150     dirichlet_psi = np.concatenate([inlet_nodes, wall_bottom,
                                     wall_top, obstacle_nodes])
151     dirichlet_psi = np.unique(dirichlet_psi)
152     free_psi = np.setdiff1d(np.arange(n_nodes), dirichlet_psi)
153
154     # Solver
155     K_free = K[free_psi, :][:, free_psi]
156     b_free = -K[free_psi, :][:, dirichlet_psi] @ psi[
        dirichlet_psi]
157     psi[free_psi] = spsolve(K_free, b_free)
158
159     # --- CÁLCULO AERODINÂMICO ---
160     # 1. Velocidade nos nós
161     # Interpolador linear para gradientes (velocidade) nos
        elementos
162     tci = tri.LinearTriInterpolator(triang, psi)
163     (dpsi_dx, dpsi_dy) = tci.gradient(X, Y)
164     u_nodes = dpsi_dy
165     v_nodes = -dpsi_dx
166
167     # Velocidade de referência (Inlet)
168     U_inf = params["v_inf"]
169     V_sq = u_nodes**2 + v_nodes**2
170     Cp = 1.0 - V_sq / (U_inf**2)
171
172     # 2. Extrair dados na superfície do obstáculo
173     # obstacle_nodes já temos. Precisamos ordenar para plotar e
        integrar.
174     # Ordenar pelo ângulo em relação ao centro (cx, cy) funciona
        para todos (convexos)
175     obs_idx = obstacle_nodes
176     x_obs = X[obs_idx]
177     y_obs = Y[obs_idx]
178     cp_obs = Cp[obs_idx]
179
180     angles = np.arctan2(y_obs - params["cy"], x_obs - params["cx
        "])
181     sorted_indices = np.argsort(angles)
182

```



```

183     x_sorted = x_obs[sorted_indices]
184     y_sorted = y_obs[sorted_indices]
185     cp_sorted = cp_obs[sorted_indices]
186     angles_sorted = angles[sorted_indices]
187
188     # 3. Integração Numérica para CL e CD
189     # F = Integral(-Cp * n * ds)
190     # Aproximação por segmentos lineares entre pontos ordenados
191     CL_num = 0.0
192     CD_num = 0.0
193
194     # Fechar o loop (último conecta ao primeiro)
195     x_loop = np.append(x_sorted, x_sorted[0])
196     y_loop = np.append(y_sorted, y_sorted[0])
197     cp_loop = np.append(cp_sorted, cp_sorted[0])
198
199     for i in range(len(x_sorted)):
200         dx = x_loop[i+1] - x_loop[i]
201         dy = y_loop[i+1] - y_loop[i]
202         ds = np.sqrt(dx*dx + dy*dy)
203
204         # Normal (rotacionar tangente 90 graus para fora)
205         # Tangente: (dx, dy). Normal: (dy, -dx) (Verificar
206         sentido horário/anti-horário)
207         # Pontos ordenados por ângulo (-pi a pi) -> Sentido Anti
208         -Horário
209         # Tangente aponta no sentido anti-horário.
210         # Normal para fora deve ser (dy, -dx).
211         nx = dy / ds
212         ny = -dx / ds
213
214         cp_avg = 0.5 * (cp_loop[i] + cp_loop[i+1])
215
216         # Força = -Cp * n * ds
217         CD_num += -cp_avg * nx * ds
218         CL_num += -cp_avg * ny * ds
219
220     # Normalizar pela corda/diâmetro?
221     # Cp já é adimensional. A integral dá Força/ (0.5 rho U^2).

```

```

220     # Para obter CL, dividimos pela corda (ou diâmetro).
221     if params["geo_type"] == "cylinder":
222         ref_len = 2 * params["r"]
223     elif params["geo_type"] == "square":
224         ref_len = params["side"] # Aproximado
225     elif params["geo_type"] == "airfoil" or params["geo_type"]
226         == "naca4412":
227         ref_len = params["chord"]
228     else:
229         ref_len = 1.0
230
231     CL_num /= ref_len
232     CD_num /= ref_len
233
234     # --- CÁLCULO DIMENSIONAL ---
235     # Lift (L) = 0.5 * rho * V^2 * CL * chord
236     # Circulation (Gamma) = L / (rho * V)
237
238     rho = params["rho"]
239     v_inf = params["v_inf"]
240     q_inf = 0.5 * rho * v_inf**2
241
242     # Dimensional Force (N) per unit span
243     Lift_dim = q_inf * CL_num * ref_len
244     Drag_dim = q_inf * CD_num * ref_len
245
246     # Dimensional Circulation (m^2/s)
247     # L = rho * V * Gamma => Gamma = L / (rho * V)
248     if abs(v_inf) > 1e-6:
249         Gamma_dim = Lift_dim / (rho * v_inf)
250     else:
251         Gamma_dim = 0.0
252
253     # --- VISUALIZAÇÃO ---
254     fig = plt.figure(figsize=(10, 12))
255     gs = fig.add_gridspec(3, 1, height_ratios=[1.5, 1.5, 1])
256
257     ax1 = fig.add_subplot(gs[0])
258     ax2 = fig.add_subplot(gs[1])

```

```

258     ax3 = fig.add_subplot(gs[2])
259
260     # 1. Função Corrente + Malha
261     # Smooth gradient with Gouraud shading and Jet colormap
262     contour = ax1.tripcolor(triang, psi, shading='gouraud', cmap
        = 'jet')
263     # Stronger mesh visibility
264     ax1.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
265     ax1.set_title(f"Função Corrente ( $\psi$ ) - {params['
        geo_type'].capitalize()}")
266     ax1.set_aspect('equal')
267     ax1.set_xlabel("x [m]")
268     ax1.set_ylabel("y [m]")
269     fig.colorbar(contour, ax=ax1)
270
271     # 2. Linhas de Corrente
272     xi = np.linspace(0, params["L"], 100)
273     yi = np.linspace(0, params["H"], 100)
274     Xi, Yi = np.meshgrid(xi, yi)
275     (dpsi_dx_grid, dpsi_dy_grid) = tci.gradient(Xi, Yi)
276     u_grid = dpsi_dy_grid
277     v_grid = -dpsi_dx_grid
278     vel_mag_grid = np.sqrt(u_grid**2 + v_grid**2)
279     strm = ax2.streamplot(Xi, Yi, u_grid, v_grid, color=
        vel_mag_grid, cmap='jet', density=1.5, linewidth=1,
        arrowsize=1.5)
280     ax2.set_title("Linhas de Corrente")
281     ax2.set_aspect('equal')
282     ax2.set_xlim(0, params["L"])
283     ax2.set_ylim(0, params["H"])
284     ax2.set_xlabel("x [m]")
285     ax2.set_ylabel("y [m]")
286     fig.colorbar(strm.lines, ax=ax2, label='Velocidade')
287
288     # 3. Distribuição de Cp
289     if params["geo_type"] == "cylinder":
290         # Plotar vs Theta (graus)
291         theta_deg = np.degrees(angles_sorted)

```

```

292         ax3.plot(theta_deg, cp_sorted, 'b-', label='Numérico',
293                  linewidth=2)
294         ax3.set_xlabel(r'$\theta$ (graus)')
295
296         # Analítico (Potencial sem circulação):  $C_p = 1 - 4\sin^2(\theta)$ 
297         theta_ana = np.linspace(-np.pi, np.pi, 200)
298         cp_ana = 1 - 4 * np.sin(theta_ana)**2
299         ax3.plot(np.degrees(theta_ana), cp_ana, 'r--', label='
300             Analítico (Teórico)', alpha=0.7)
301
302     elif params["geo_type"] == "airfoil" or params["geo_type"]
303         == "naca4412":
304         # Plotar vs x/c
305         # Separar extradorso (upper) e intradorso (lower)
306         # Upper:  $y > c_y$  (aprox, ou usar ângulo)
307         # Rotacionar de volta para achar x ao longo da corda?
308         # Simplificação: usar x_sorted
309         ax3.plot(x_sorted, cp_sorted, 'b.-', label='Cp Numérico'
310                )
311         ax3.set_xlabel('x [m]')
312
313     else:
314         ax3.plot(angles_sorted, cp_sorted, 'b.-')
315         ax3.set_xlabel('Ângulo (rad)')
316
317     ax3.set_ylabel(r'$C_p$ (adimensional)')
318     ax3.set_title(r'Distribuição de Coeficiente de Pressão (
319         $C_p$)')
320     ax3.grid(True, alpha=0.3)
321     ax3.invert_yaxis() # Convenção aerodinâmica: Cp negativo
322                        para cima
323     ax3.legend()
324
325     plt.tight_layout()
326
327     img_base64 = plot_to_base64(fig)

```

```

323     plot_div.innerHTML = f''
324
325     # Exibir Resultados Numéricos (Apenas Numérico)
326     res_html = f"""
327     <div class="grid grid-cols-1 gap-4 text-sm">
328         <div class="bg-blue-50 p-3 rounded text-center shadow-sm
            ">
329             <p class="font-bold text-indigo-700 mb-2">Resultados
                Aerodinâmicos</p>
330             <div class="grid grid-cols-1 gap-1">
331                 <p><span class="font-semibold">Sustentação (L)
                    :</span> {Lift_dim:.4f} N/m</p>
332                 <p><span class="font-semibold">Arrasto (D):</
                    span> {Drag_dim:.4f} N/m</p>
333                 <p><span class="font-semibold">Circulação (&
                    Gamma;):</span> {Gamma_dim:.4f} m2/s</p>
334             </div>
335         </div>
336     </div>
337     """
338     document.getElementById("lift-results").innerHTML = res_html
339
340     # =====
341     # 3. Interface Handler
342     # =====
343     @when("click", "#run-btn")
344     async def run_handler(event=None):
345         plot_div = document.getElementById("plot-output")
346         plot_div.innerHTML = "Calculando..."
347         await asyncio.sleep(0.1)
348
349         try:
350             params = {
351                 "L": float(document.getElementById("L").value),
352                 "H": float(document.getElementById("H").value),
353                 "cx": float(document.getElementById("cx").value),
354                 "cy": float(document.getElementById("cy").value),

```

```

355         "nx": int(document.getElementById("nx").value),
356         "ny": int(document.getElementById("ny").value),
357         # "Gamma": float(document.getElementById("Gamma").
           value), # Removed
358         "geo_type": document.getElementById("geo_type").
           value,
359         "rho": float(document.getElementById("rho").value),
360         "v_inf": float(document.getElementById("v_inf").
           value)
361     }
362
363     if params["geo_type"] == "cylinder":
364         params["r"] = float(document.getElementById("r").
           value)
365     elif params["geo_type"] == "airfoil" or params["geo_type"] == "naca4412":
366         params["chord"] = float(document.getElementById("
           chord").value)
367         params["angle"] = float(document.getElementById("
           angle_airfoil").value)
368
369     await solve_potential(params, plot_div)
370
371 except Exception as e:
372     plot_div.innerHTML = f"Erro: {e}"
373     print(e)

```

B.5 Navier-Stokes 2D (Vorticidade-Corrente)

Listagem B.5: Navier-Stokes 2D — simulation.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.tri as tri
4  from scipy.sparse import lil_matrix
5  from scipy.sparse.linalg import spsolve
6  from pyscript import when, document
7  import asyncio

```

```

8  from common import generate_mesh_generic, assemble_matrices,
    plot_to_base64
9  import geometry
10
11  # Global control flag
12  STOP_SIMULATION = False
13
14  @when("click", "#stop-btn")
15  def stop_simulation_handler(event=None):
16      global STOP_SIMULATION
17      STOP_SIMULATION = True
18      document.getElementById("stop-btn").classList.add("hidden")
19      document.getElementById("stop-btn").classList.remove("flex")
20      document.getElementById("run-btn").classList.remove("hidden"
        )
21      document.getElementById("run-btn").classList.add("flex")
22
23  # =====
24  # NS Helpers
25  # =====
26  def assemble_convection(X, Y, triang, u_el, v_el):
27      n_nodes = len(X)
28      C = lil_matrix((n_nodes, n_nodes))
29      elements = triang.triangles
30      mask = triang.mask if triang.mask is not None else np.zeros(
        len(elements), dtype=bool)
31
32      for i, el in enumerate(elements):
33          if mask[i]: continue
34          x = X[el]
35          y = Y[el]
36          area = 0.5 * np.abs(x[0]*(y[1]-y[2]) + x[1]*(y[2]-y[0])
            + x[2]*(y[0]-y[1]))
37          b = np.array([y[1] - y[2], y[2] - y[0], y[0] - y[1]])
38          c = np.array([x[2] - x[1], x[0] - x[2], x[1] - x[0]])
39          ue = u_el[i]
40          ve = v_el[i]
41          K_dot_grad = (ue * b + ve * c) / (2 * area)
42          ce = np.outer(np.ones(3) * (area / 3.0), K_dot_grad)

```

```

43         for row in range(3):
44             for col in range(3):
45                 C[el[row], el[col]] += ce[row, col]
46     return C.tocsr()
47
48 def get_boundary_normals(X, Y, boundary_nodes, triang):
49     adj = [set() for _ in range(len(X))]
50     mask = triang.mask if triang.mask is not None else np.zeros(
51         len(triang.triangles), dtype=bool)
52     for i, el in enumerate(triang.triangles):
53         if mask[i]: continue
54         for j in range(3):
55             n1, n2 = el[j], el[(j+1)%3]
56             adj[n1].add(n2)
57             adj[n2].add(n1)
58
59     boundary_set = set(boundary_nodes)
60     valid_neighbors = np.zeros(len(boundary_nodes), dtype=int)
61     h_sq = np.zeros(len(boundary_nodes))
62
63     for i, node in enumerate(boundary_nodes):
64         node_neighbors = list(adj[node])
65         best_nb = -1
66         min_dist = 1e9
67         for nb in node_neighbors:
68             if nb not in boundary_set:
69                 d = (X[node]-X[nb])**2 + (Y[node]-Y[nb])**2
70                 if d < min_dist:
71                     min_dist = d
72                     best_nb = nb
73         if best_nb == -1:
74             for nb in node_neighbors:
75                 d = (X[node]-X[nb])**2 + (Y[node]-Y[nb])**2
76                 if d < min_dist:
77                     min_dist = d
78                     best_nb = nb
79         valid_neighbors[i] = best_nb
80         h_sq[i] = min_dist
81     return valid_neighbors, h_sq

```



```

81
82 # =====
83 # Solver NS
84 # =====
85 async def solve_navier_stokes(params, plot_div):
86     # 1. Malha
87     cx, cy = params["cx"], params["cy"]
88
89     # Helper for inlet profile
90     def get_psi_inlet(y_val):
91         # Default: u=1, psi=y
92         return y_val
93
94     scenario = params.get("scenario", "channel")
95
96     if scenario == "cavity":
97         # Force no geometry/obstruction for cavity
98         boundary_pts = np.empty((0, 2))
99         def mask_func(x, y): return np.ones_like(x, dtype=bool)
100     else:
101         boundary_pts, mask_func = geometry.get_geometry(params["
            geo_type"], params)
102
103     X, Y, triang = generate_mesh_generic(
104         params["L"], params["H"], boundary_pts, mask_func,
105         cx, cy, params["nx"], params["ny"]
106     )
107     n_nodes = len(X)
108
109     # 2. Matrices
110     K, M = assemble_matrices(X, Y, triang)
111
112     # 3. Fronteiras
113     eps = 1e-5
114     inlet_nodes = np.where(X < eps)[0]
115     wall_bottom = np.where(Y < eps)[0]
116     wall_top = np.where(Y > params["H"] - eps)[0]
117
118     # Obstacle nodes (last N points)

```

```

119     n_bound = len(boundary_pts)
120     obstacle_nodes = np.arange(n_nodes - n_bound, n_nodes)
121
122     solid_walls = np.concatenate([wall_bottom, wall_top,
123                                   obstacle_nodes])
124
125     solid_walls = np.unique(solid_walls)
126
127     wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
128                                                     solid_walls, triang)
129
130     # 4. Inicialização
131     # 4. Inicialização e Condições de Contorno
132     psi = np.zeros(n_nodes)
133     omega = np.zeros(n_nodes)
134
135     scenario = params.get("scenario", "channel")
136
137     if scenario == "cavity":
138         # Lid-Driven Cavity
139         # Walls: Bottom, Left, Right, Top
140         # Psi = 0 on all walls (closed box)
141         # Omega BC drives the flow.
142
143         # Identify walls
144         eps = 1e-5
145         wall_bottom = np.where(Y < eps)[0]
146         wall_top = np.where(Y > params["H"] - eps)[0]
147         wall_left = np.where(X < eps)[0]
148         wall_right = np.where(X > params["L"] - eps)[0]
149
150         # All boundary nodes
151         all_walls = np.concatenate([wall_bottom, wall_top,
152                                     wall_left, wall_right])
153         all_walls = np.unique(all_walls)
154
155         # Psi BC: 0 everywhere on boundary
156         psi[all_walls] = 0.0
157
158         dirichlet_psi = all_walls

```

```

155
156     # Omega BC nodes (same as Psi for now, updated in loop)
157     dirichlet_omega = all_walls
158
159     # For Thom's formula, we need neighbors for ALL walls
160     wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
161                                                     all_walls, triang)
162
163     # Store wall types for specific BC application
164     # Top wall moves:  $u = 1 \rightarrow d\Psi/dy = 1$ 
165     # Thom's formula with moving wall:
166     #  $\omega_w = -2(\psi_{nb} - \psi_w - h*u_{wall}) / h^2$ 
167     # Here  $u_{wall} = 1$  for top, 0 for others.
168
169     # We need to map each wall node to its velocity ( $u_{tan}$ )
170     wall_u_tan = np.zeros(len(all_walls))
171
172     # Find indices of top wall nodes within all_walls
173     # This is  $O(N^2)$  worst case but  $N_{bound}$  is small.
174     # Better: Create a map.
175
176     # Let's just iterate
177     for i, node_idx in enumerate(all_walls):
178         if Y[node_idx] > params["H"] - eps:
179             wall_u_tan[i] = 1.0 # Moving lid
180         else:
181             wall_u_tan[i] = 0.0
182
183     solid_walls = all_walls # For consistency with loop
184
185 else:
186     # Channel Flow (Original Logic)
187     psi[inlet_nodes] = get_psi_inlet(Y[inlet_nodes])
188     psi[wall_bottom] = 0.0
189
190     # Top Wall BC
191     psi[wall_top] = params["H"]
192
193     # Obstacle BC

```

```

193         if params.get("geo_type") == "step":
194             psi[obstacle_nodes] = 0.0
195         else:
196             # Cylinder/Rectangle/etc
197             psi[obstacle_nodes] = get_psi_inlet(cy)
198
199         dirichlet_psi = np.concatenate([inlet_nodes, wall_bottom
200             , wall_top, obstacle_nodes])
201         dirichlet_psi = np.unique(dirichlet_psi)
202
203         solid_walls = np.concatenate([wall_bottom, wall_top,
204             obstacle_nodes])
205         solid_walls = np.unique(solid_walls)
206
207         dirichlet_omega = np.concatenate([inlet_nodes,
208             solid_walls])
209         dirichlet_omega = np.unique(dirichlet_omega)
210
211         wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
212             solid_walls, triang)
213
214         # Stationary walls
215         wall_u_tan = np.zeros(len(solid_walls))
216
217         free_psi = np.setdiff1d(np.arange(n_nodes), dirichlet_psi)
218         free_omega = np.setdiff1d(np.arange(n_nodes),
219             dirichlet_omega)
220
221         K_free_psi = K[free_psi, :][:, free_psi]
222
223         dt = params["dt"]
224         Re = params["Re"]
225         steps_per_frame = params["steps_per_frame"]
226         total_steps = params["total_steps"]
227
228         current_step = 0
229
230         while current_step < total_steps and not STOP_SIMULATION:
231             for _ in range(steps_per_frame):

```

```

227     # A. Poisson Psi
228     rhs_psi = M @ omega
229     b_psi = rhs_psi[free_psi] - K[free_psi, :][:,
        dirichlet_psi] @ psi[dirichlet_psi]
230     psi[free_psi] = spsolve(K_free_psi, b_psi)
231
232     # B. Thom's Formula (Generalized for moving walls)
233     # Canonical form:  $\omega_w = -2 (\psi_{nb} - \psi_w)/h^2$ 
         $- 2 u_{tan}/h$ 
234     # Derivation (top lid, n inward = -y): Taylor at the
        wall with step -h gives
235     #  $\psi_{nb} = \psi_w - h (d\psi/dy)_w + (h^2/2) (d^2\psi/dy^2)_w$ 
236     # Since  $u = d\psi/dy$ ,  $(d\psi/dy)_w = u_{tan}$ . With  $\psi$ 
        constant along the wall,
237     #  $d^2\psi/dx^2 = 0$  there, so  $d^2\psi/dy^2 = -\omega_w$ .
        Solving:
238     #  $\omega_w = -2 (\psi_{nb} - \psi_w + h u_{tan}) / h^2$ 
239     # Validated against Ghia et al. (1982) at  $Re=100$  (
        RMS 0.065 vs. 0.93 with the
240     # wrong sign). See .agents/review/thom_validation/
        for the reproducible check.
241     psi_nb = psi[wall_neighbors]
242     psi_w = psi[solid_walls]
243     h_vals = np.sqrt(wall_h_sq)
244     omega[solid_walls] = -2.0 * (psi_nb - psi_w + h_vals
        * wall_u_tan) / wall_h_sq
245
246     # C. Velocities
247     elements = triang.triangles
248     mask = triang.mask if triang.mask is not None else
        np.zeros(len(elements), dtype=bool)
249     el_nodes = elements[~mask]
250     x_el = X[el_nodes]
251     y_el = Y[el_nodes]
252     p_el = psi[el_nodes]
253
254     b_coeff = np.column_stack([y_el[:,1]-y_el[:,2], y_el
       [:,2]-y_el[:,0], y_el[:,0]-y_el[:,1]])

```

```

255         c_coeff = np.column_stack([x_el[:,2]-x_el[:,1], x_el
           [:,0]-x_el[:,2], x_el[:,1]-x_el[:,0]])
256     area_el = 0.5 * np.abs(x_el[:,0]*(y_el[:,1]-y_el
           [:,2]) + x_el[:,1]*(y_el[:,2]-y_el[:,0]) + x_el
           [:,2]*(y_el[:,0]-y_el[:,1]))
257
258     u_vals = np.sum(p_el * c_coeff, axis=1) / (2 *
            area_el)
259     v_vals = -np.sum(p_el * b_coeff, axis=1) / (2 *
            area_el)
260
261     u_el_full = np.zeros(len(elements))
262     v_el_full = np.zeros(len(elements))
263     u_el_full[~mask] = u_vals
264     v_el_full[~mask] = v_vals
265
266     # D. Convection
267     C = assemble_convection(X, Y, triang, u_el_full,
            v_el_full)
268
269     # E. Vorticity Transport
270     A = M + (dt / Re) * K
271     rhs = M @ omega - dt * (C @ omega)
272
273     A_free = A[free_omega, :][:, free_omega]
274     b_omega = rhs[free_omega] - A[free_omega, :][:,
            dirichlet_omega] @ omega[dirichlet_omega]
275     omega[free_omega] = spsolve(A_free, b_omega)
276
277     current_step += 1
278
279     # Plot
280     tci = tri.LinearTriInterpolator(triang, psi)
281     (dpsi_dx, dpsi_dy) = tci.gradient(X, Y)
282     u = dpsi_dy
283     v = -dpsi_dx
284
285     fig = plt.figure(figsize=(10, 12))
286     gs = fig.add_gridspec(3, 1, height_ratios=[1, 1, 1])

```

```

287
288     ax1 = fig.add_subplot(gs[0])
289     ax2 = fig.add_subplot(gs[1])
290     ax3 = fig.add_subplot(gs[2])
291
292     # Smooth gradient for Psi using tripcolor with Gouraud
293     shading
294     # User requested "Blue to Red" (Jet)
295     contour_psi = ax1.tripcolor(triang, psi, shading='
296     gouraud', cmap='jet')
297     # Removed discrete contour lines for cleaner look
298     ax1.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
299     # Visible mesh
300     ax1.set_title(f"Navier-Stokes (Step {current_step}) - $
301     \\psi$")
302     ax1.set_aspect('equal')
303     ax1.set_xlabel("x [m]")
304     ax1.set_ylabel("y [m]")
305     fig.colorbar(contour_psi, ax=ax1)
306
307     # Smooth gradient for Omega
308     contour_omega = ax2.tripcolor(triang, omega, shading='
309     gouraud', cmap='jet')
310     ax2.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
311     ax2.set_title(f"Navier-Stokes (Step {current_step}) - $
312     \\omega$")
313     ax2.set_aspect('equal')
314     ax2.set_xlabel("x [m]")
315     ax2.set_ylabel("y [m]")
316     fig.colorbar(contour_omega, ax=ax2)
317
318     xi = np.linspace(0, params["L"], 100)
319     yi = np.linspace(0, params["H"], 100)
320     Xi, Yi = np.meshgrid(xi, yi)
321     (dpsi_dx_grid, dpsi_dy_grid) = tci.gradient(Xi, Yi)
322     u_grid = dpsi_dy_grid
323     v_grid = -dpsi_dx_grid
324     vel_mag_grid = np.sqrt(u_grid**2 + v_grid**2)
325

```

```

320     # Streamlines with 'jet'
321     strm = ax3.streamplot(Xi, Yi, u_grid, v_grid, color=
        vel_mag_grid, cmap='jet', density=1.5, linewidth=1,
        arrowsize=1.5)
322     ax3.set_title("Linhas de Corrente")
323     ax3.set_aspect('equal')
324     ax3.set_xlim(0, params["L"])
325     ax3.set_ylim(0, params["H"])
326     ax3.set_xlabel("x [m]")
327     ax3.set_ylabel("y [m]")
328     fig.colorbar(strm.lines, ax=ax3)
329
330
331
332     plt.tight_layout()
333
334     img_base64 = plot_to_base64(fig)
335     plot_div.innerHTML = f''
336     await asyncio.sleep(0.01)
337
338     if not STOP_SIMULATION:
339         plot_div.innerHTML += '<p class="text-center text-green
        -600 font-bold mt-2">Simulação Concluída!</p>'
340     else:
341         plot_div.innerHTML += '<p class="text-center text-red
        -600 font-bold mt-2">Simulação Parada pelo Usuário.</
        p>'
342
343     @when("click", "#run-btn")
344     async def run_handler(event=None):
345         global STOP_SIMULATION
346         STOP_SIMULATION = False
347
348         plot_div = document.getElementById("plot-output")
349         plot_div.innerHTML = "Iniciando..."
350         await asyncio.sleep(0.1)
351

```



```

352     # Show stop button
353     document.getElementById("run-btn").classList.add("hidden")
354     document.getElementById("run-btn").classList.remove("flex")
355     document.getElementById("stop-btn").classList.remove("hidden
        ")
356     document.getElementById("stop-btn").classList.add("flex")
357
358     try:
359         params = {
360             "L": float(document.getElementById("L").value),
361             "H": float(document.getElementById("H").value),
362             "cx": float(document.getElementById("cx").value),
363             "cy": float(document.getElementById("cy").value),
364             "scenario": document.getElementById("scenario").
                value,
365             "geo_type": document.getElementById("geo_type").
                value,
366             "r": float(document.getElementById("r").value),
367             "w": float(document.getElementById("w").value),
368             "h": float(document.getElementById("h").value),
369             "step_h": float(document.getElementById("step_h").
                value),
370             "step_l": float(document.getElementById("step_l").
                value),
371             "nx": int(document.getElementById("nx").value),
372             "ny": int(document.getElementById("ny").value),
373             "Re": float(document.getElementById("Re").value),
374             "dt": float(document.getElementById("dt").value),
375             "steps_per_frame": int(document.getElementById("
                steps_per_frame").value),
376             "total_steps": int(document.getElementById("
                total_steps").value)
377         }
378
379         await solve_navier_stokes(params, plot_div)
380
381     except Exception as e:
382         plot_div.innerHTML = f"Erro: {e}"
383         print(e)

```

```
384     finally:
385         document.getElementById("stop-btn").classList.add("
            hidden")
386         document.getElementById("stop-btn").classList.remove("
            flex")
387         document.getElementById("run-btn").classList.remove("
            hidden")
388         document.getElementById("run-btn").classList.add("flex")
```